STEP 机器人 SDK 使用说明书

适用版本：V1.8.1.20250213.ARM 以上版本
出版状态：正式
版本：C++_V2.4.0(SDK-V2.4.0.20250213)

中国上海新时达机器人有限公司全权负责本控制系统使用说明书的编制、印刷。

版权所有，保留一切权利。

没有得到上海新时达机器人有限公司的许可，任何单位和个人不得擅自摘抄、复制本书（软件等）的一部分或全部，不得以任何形式（包括资料和出版物）进行传播。
版权所有，侵权必究。内容如有改动，恕不另行通知。

All Copyright© reserved by Shanghai STEP Robot Co., Ltd., China.

Shanghai STEP Robot Co., Ltd. authorizes in the documenting, printing to this robot control manual.

All rights reserved.

The information in this document is subject to change withoutprior notice. No part of this document may in any form or by any means(electronic，mechanical，micro-coping，photocopying，recording or otherwise)be reproduced，stored in a retrial system or transmitted without prior written permission from Shanghai STEP Robotics Co.，Ltd.

目录

1 SDK 使用前准备 1

1.1 SDK 说明 1
1.2 控制器软件版本要求 1
1.3 上位机 VS 版本要求 1

2 SDK 使用说明 3

2.1 通讯 3
2.1.1 Lib 调用 3
2.1.2 定义 4
2.1.3 初始化 5
2.1.4 获取版本号 5
2.1.5 连接示例 6
2.2 状态反馈接口 7
2.2.1 获取当前工程名 7
2.2.2 获取当前程序名 7
2.2.3 获取用户坐标系名称 7
2.2.4 获取当前工具名称 7
2.2.5 获取当前错误信息 7
2.2.6 获取机器人 DI 8
2.2.7 获取机器人 DO 8
2.2.8 获取机器人 AI 8
2.2.9 获取机器人 AO 9
2.2.10 获取实际笛卡尔动态信息 9
2.2.11 获取当前运行模式 9
2.2.12 获取使能状态 10
2.2.13 获取当前点动坐标系 10
2.2.14 获取全局速度 10
2.2.15 获取程序运行状态 10
2.2.16 获取程序运行模式 11
2.2.17 获取当前所在行 11
2.2.18 获取当前关节位置 11
2.2.19 获取当前世界笛卡尔位置 12
2.2.20 获取当前基坐标笛卡尔位置 12
2.2.21 获取当前用户坐标笛卡尔位置 12
2.2.22 获取当前世界坐标笛卡尔位置和旋转矩阵 12
2.2.23 获取运行时间 13
2.2.24 获取开机时间 13
2.2.25 获取焊接模式 13
2.3 基础命令接口 14
2.3.1 获取机器人信息 14
2.3.2 获取控制柜信息 14
2.3.3 机器人上下使能（间隔 300ms） 15

2.3.4 设置操作模式（功能较多，可详细查看）.....16
2.3.5 加载程序.....21
2.3.6 卸载程序.....22
2.3.7 清除错误.....22
2.4 参数设置接口.....23
2.4.1 设置用户坐标系.....23
2.4.2 设置工具坐标系.....23
2.4.3 设置全局速度.....24
2.4.4 设置点动坐标系.....24
2.4.5 设置当前运行模式.....25
2.4.6 设置程序行号.....25
2.4.7 设置 DI.....26
2.4.8 设置 DO.....27
2.4.9 设置 AI.....28
2.4.10 设置 AO.....30
2.5 变量设置接口.....32
2.5.1 变量设置说明.....32
2.5.2 修改 BOOL 值.....32
2.5.3 在线修改 BOOL 值.....32
2.5.4 修改 INT 值.....33
2.5.5 在线修改 INT 值.....33
2.5.6 修改 REAL 值.....34
2.5.7 在线修改 REAL 值.....34
2.5.8 修改 STRING 值.....35
2.5.9 在线修改 STRING 值.....35
2.5.10 修改 AXISPOS 值.....36
2.5.11 在线修改 AXISPOS 值.....36
2.5.12 修改 CARTPOS 值.....37
2.5.13 在线修改 CARTPOS 值.....38
2.5.14 修改 ROBOTAXISPOS 值.....38
2.5.15 在线修改 ROBOTAXISPOS 值.....39
2.5.16 修改 ROBOTCARTPOS 值.....39
2.5.17 在线修改 ROBOTCARTPOS 值.....40
2.5.18 修改 DYNAMIC 值.....41
2.5.19 在线修改 DYNAMIC 值.....42
2.5.20 修改 ARCONDATA 值.....43
2.5.21 在线修改 ARCONDATA 值.....44
2.5.22 修改 ARCOFFDATA 值.....44
2.5.23 在线修改 ARCOFFDATA 值.....45
2.5.24 修改 ARCDATA 值.....45
2.5.25 在线修改 ARCDATA 值.....46
2.5.26 修改 ARCRETRYDATA 值.....47
2.5.27 在线修改 ARCRETRYDATA 值.....47
2.5.28 修改 ARCSEGDATA 值.....48

2.5.29 在线修改 ARCSEGDATA 值.....49
2.5.30 修改 ARCCONFIGDATA 值.....49
2.5.31 在线修改 ARCCONFIGDATA 值.....50
2.5.32 修改 WEAVEDATA 值.....50
2.5.33 在线修改 WEAVEDATA 值.....51
2.5.34 修改 CD 值.....52
2.5.35 在线修改 CD 值.....53
2.5.36 修改 CDS 值.....54
2.5.37 修改 PAT 值.....55
2.5.38 在线修改 PAT 值.....55
2.5.39 修改 TRACKDATA 值.....56
2.5.40 在线修改 TRACKDATA 值.....58
2.5.41 修改 LASERTRACKDATA 值.....59
2.5.42 在线修改 LASERTRACKDATA 值.....60
2.5.43 修改 PLASERFILTER 值.....61
2.5.44 在线修改 PLASERFILTER 值.....62
2.5.45 修改 LASERARCONDATA 值.....62
2.5.46 在线修改 LASERARCONDATA 值.....64
2.5.47 修改 LASERARCDATA 值.....64
2.5.48 在线修改 LASERARCDATA 值.....65
2.5.49 修改 LASERARCOFFDATA 值.....66
2.5.50 在线修改 LASERARCOFFDATA 值.....67
2.5.51 修改 AXISDIST 值.....68
2.5.52 在线修改 AXISDIST 值.....68
2.5.53 修改 CARTDIST 值.....69
2.5.54 在线修改 CARTDIST 值.....70
2.5.55 修改 AUXAXISPOS 值.....71
2.5.56 在线修改 AUXAXISPOS 值.....71
2.5.57 修改 PCARTPOS 值.....72
2.5.58 在线修改 PCARTPOS 值.....73
2.5.59 修改 OVERLAPREL 值.....74
2.5.60 在线修改 OVERLAPREL 值.....74
2.5.61 修改 OVERLAPABS 值.....75
2.5.62 在线修改 OVERLAPABS 值.....75
2.5.63 修改 RAMPTYPE 值.....76
2.5.64 在线修改 RAMPTYPE 值.....77
2.5.65 修改 BENDDATA 值.....77
2.5.66 在线修改 BENDDATA 值.....78
2.5.67 修改 BENDSYNDATA 值.....78
2.5.68 在线修改 BENDSYNDATA 值.....79
2.5.69 修改 POLISHDATA 值.....80
2.5.70 在线修改 POLISHDATA 值.....81
2.5.71 修改 TOOLLOAD 值.....81
2.5.72 修改 PAYLOAD 值.....82

2.5.73 修改 TOOL 值 83
2.5.74 修改 CARTREF 值 84
2.6 变量获取接口 85
2.6.1 变量获取说明 85
2.6.2 获取 BOOL 值 85
2.6.3 获取 INT 值 85
2.6.4 获取 REAL 值 86
2.6.5 获取 string 值 86
2.6.6 获取 Axispos 值 87
2.6.7 获取 CARTPOS 值 87
2.6.8 获取 ROBOTAXISPOS 值 88
2.6.9 获取 ROBOTCARTPOS 值 88
2.6.10 获取 DYNAMIC 值 89
2.6.11 获取 ARCONDATA 值 90
2.6.12 获取 ARCOFFDATA 值 90
2.6.13 获取 ARCDATA 值 91
2.6.14 获取 ARCRETRYDATA 值 91
2.6.15 获取 ARCSEGDATA 值 92
2.6.16 获取 ARCCONFIGDATA 值 93
2.6.17 获取 WEAVEDATA 值 94
2.6.18 获取 CD 值 95
2.6.19 获取 CDS 值 96
2.6.20 获取 PAT 值 97
2.6.21 获取 TRACKDATA 值 98
2.6.22 获取 LASERTRACKDATA 值 99
2.6.23 获取 PLASERFILTER 值 99
2.6.24 获取 LASERARCONDATA 值 100
2.6.25 获取 LASERARCDATA 值 101
2.6.26 获取 LASERARCOFFDATA 值 102
2.6.27 获取 AXISDIST 值 103
2.6.28 获取 CARTDIST 值 103
2.6.29 获取 AUXAXISPOS 值 104
2.6.30 获取 PCARTPOS 值 104
2.6.31 获取 OVERLAPREL 值 105
2.6.32 获取 OVERLAPABS 值 106
2.6.33 获取 RAMPTYPE 值 106
2.6.34 获取 BENDDATA 值 107
2.6.35 获取 BENDSYNDATA 值 107
2.6.36 获取 POLISHDATA 值 108
2.6.37 获取 TOOLLOAD 值 109
2.6.38 获取 PAYLOAD 值 110
2.6.39 获取 TOOL 值 110
2.6.40 获取 CARTREF 值 111
2.7 应用功能接口 112

2.7.1 关节绝对位置点动（手动模式可用）.....112
2.7.2 关节绝对位置点动（手动模式可用）.....112
2.7.3 设置焊接模式.....113
2.7.4 获取焊机电压电流.....114
2.7.5 关节坐标转笛卡尔坐标（正解）.....114
2.7.6 笛卡尔坐标转关节坐标（逆解）.....115
2.8 语句控制接口.....117
2.8.1 语句控制使用总览.....117
2.8.2 当前缓存区语句数量.....117
2.8.3 加载模拟程序.....118
2.8.4 卸载模拟程序.....119
2.8.5 发送语句.....119
2.8.6 写 EOF 语句.....119
2.8.7 写 SetTool 语句.....120
2.8.8 写 SetRefSys 语句.....120
2.8.9 写 PTP 语句.....121
2.8.10 写 Lin 语句.....122
2.8.11 写 Circ 语句.....124
2.8.12 写 Dyn 语句.....126
2.8.13 写 WaitTime 语句.....127
2.8.14 写 ArcOn 语句.....127
2.8.15 写 ArcOff 语句.....129
2.8.16 写 ArcSet 语句.....130
2.8.17 写 WLin 语句.....131
2.8.18 写 WCirc 语句.....135
2.8.19 写 ArcConfig 语句.....139
2.8.20 写 ArcJob 语句.....139
2.8.21 写 WireSet 语句.....139
2.8.22 写 DOSet 语句.....140
2.8.23 写 DOSetSyncPath 语句.....140
2.8.24 写 DOSetSyncTime 语句.....141
2.8.25 写 DOPulse 语句.....142
2.8.26 写 DOPulseSetSyncPath 语句.....142
2.8.27 写 DOPulseSetSyncTime 语句.....143
2.8.28 写 GDOSet 语句.....144
2.8.29 写 GDOSetSyncPath 语句.....144
2.8.30 写 GDOSetSyncTime 语句.....145
2.8.31 写 AOSet 语句.....146
2.8.32 写 AOSetSyncPath 语句.....146
2.8.33 写 AOSetSyncTime 语句.....147
2.8.34 写 DIWait 语句.....147
2.8.35 写 DIWaitSyncPath 语句.....148
2.8.36 写 DIWaitSyncTime 语句.....149
2.8.37 写 GDIWait 语句.....149

2.8.38 写 GDIWaitSyncPath 语句.....150
2.8.39 写 GDIWaitSyncTime 语句.....150
2.8.40 写 AIWait 语句.....151
2.8.41 写 AIWaitSyncPath 语句.....152
2.8.42 写 AIWaitSyncTime 语句.....152
2.8.43 写 CORR_Tool 语句.....153
2.8.44 写 CORR_Ref 语句.....154
2.8.45 写 CORR_OFF 语句.....155
2.8.46 写 Ovl 语句.....155
2.8.47 写 AxisSpaceActivate 语句.....156
2.8.48 写 CartSpaceActivate 语句.....156

3 SDK 使用案例.....158

3.1 如何发送点位让机器人运行.....158
3.2 如何发送多个点位让机器人运行.....159

版本修订说明

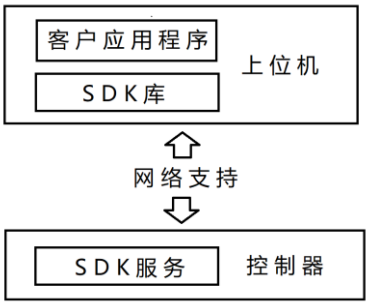
版本号	描　述	修订时间	修订人
V1.0	新增。	2023/2/24	Chenwy
V1.1	增加调用说明，修改部分错误描述。	2023/3/22	Chenwy
V1.2	增加笛卡坐标旋转矩阵输出接口； 解决初始化不完全问题。	2023/3/25	Chenwy
V1.3	增加工具坐标、用户坐标的读取和修改； 获取笛卡尔位置增加 mode。	2023/4/28	Chenwy
V1.4	增加 SDK 使用案例。	2023/6/5	Chenwy
V1.5	修复了一些 bug;； 新增在线修改变量接口。	2023/11/14	Chenwy
V2.0	新增绝对点动接口； 新增版本信息查看接口。	2024/2/19	Chenwy
V2.1	新增语句控制接口； 增加焊机模式状态反馈接口； 增加焊机电压电流获取接口。	2024/2/22	Chenwy
V2.2	增加焊接变量的修改和获取接口； 补充位置变量的修改和获取接口； 增加动态圆滑变量修改和获取接口； 增加折弯、打磨、动力学变量修改和获取接口； 增加上述变量的在线修改接口（除动力学变量）。	2024/3/29	Chenwy
V2.3	增加 linux 版本使用说明； 各设置接口增加输入变量限制。	2024/4/12	Chenwy
V2.3.1	修改部分变量接口和焊接语句接口； 增加 CDS 变量接口。	2024/7/15	Chenwy
V2.3.1	增加语句控制接口。	2024/11/11	Chenwy
V2.3.2	增加正逆解接口； 增加保持变量修改说明。	2024/12/19	Chenwy
V2.4.0	所有语句下发接口增加行号； 耗时命令接口增加阻塞选项。	2025/3/5	Chenwy

1 SDK使用前准备（必读）

1.1 SDK说明

SDK 库工作于上位机，本身不具备单独运行能力，仅提供用户二次开发以及运行依赖支持。**如不使用示教器，请在关机状态下拔出。**

用户基于 SDK 库完成自己的应用程序开发后，运行时会依赖 SDK 库对象，通过 SDK 库提供的接口，完成与运行在控制器上的服务端的数据交互，达到监控机器人的目的。



SDK 库架构示意图

SDK 库提供了多种开发语言支持，包括 C++，C#，适应不同客户的开发需求，本文为 C++库使用说明书，代码示例为 Windows 下代码，注意 Linux 下使用时的部分语法区别。

1.2 控制器软件版本要求

1）控制器软件版本必须为：V1.8.1.20240929 版本及以上

2）V1.8.1.20240929 以下版本使用：

A: V1.8.1.20240929 以上版本（不包含）需手动在该软件版本基础上升级非实时

B: V1.8.1.20240929 以前版本需使用一键安装包升级

1.3 上位机VS版本要求

Windows 下开发是在 VS2022（v143），NET7.0 版本上进行，.lib 是在 release、x86/x64 下，debug 模式不通用，根据需求配置对应版本 lib。如需其他版本，请与厂家联系。

Linux 下开发的 gcc 版本为 11.4.0，.so 是在 release、x86/x64 下，根据需求配置对应版本 lib。如需其他版本，请与厂家联系。

不同环境的 SDK 和对应的说明书均可在新时达官网下载，官网会不定期进

行更新和维护。

更新的库需与机器人软件版本配套使用，若自行更新，请注意更新后的新增功能在当前机器人版本上是否支持，若不支持需进行机器人软件版本升级。

新时达官网下载中心：https://www.stepelectric.com/Download/56.html

2 SDK使用说明

2.1 通讯（必读）

PC 软件（SDK 开发包）作为客户端，机器人控制器作为服务端。

连接时，**有且仅有一个客户端**可以与服务器连接，使用时需要注意。

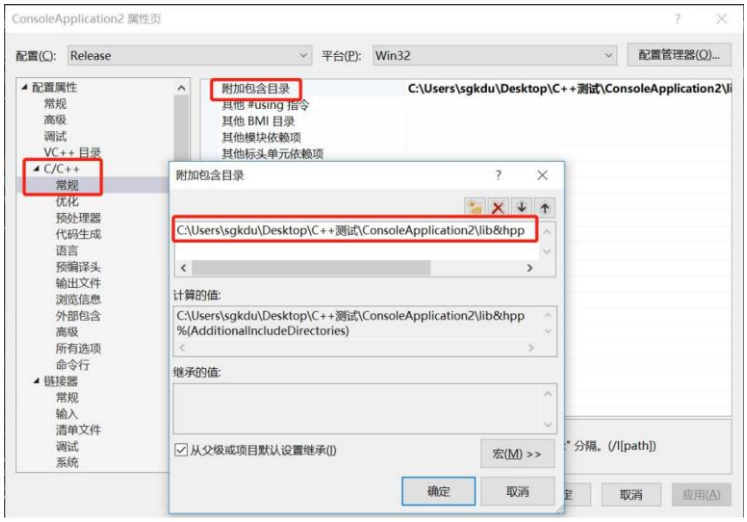
2.1.1 Lib调用

Windows 下调用：

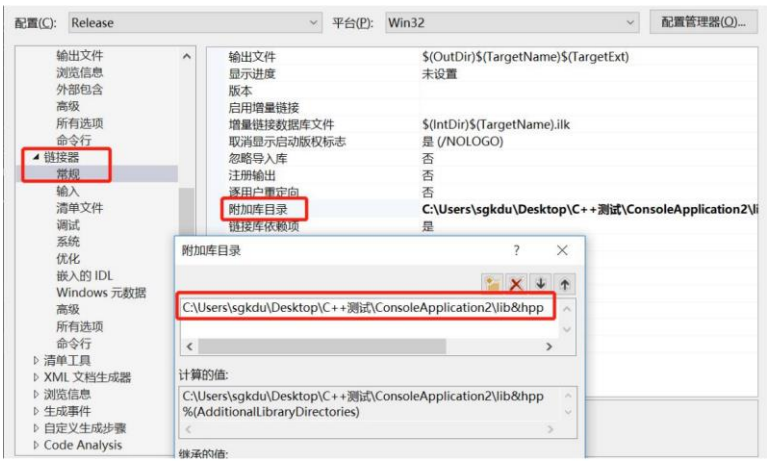
1）将以下文件放到一个文件夹里，如保存到“lib&hpp”文件夹中，

2）调用

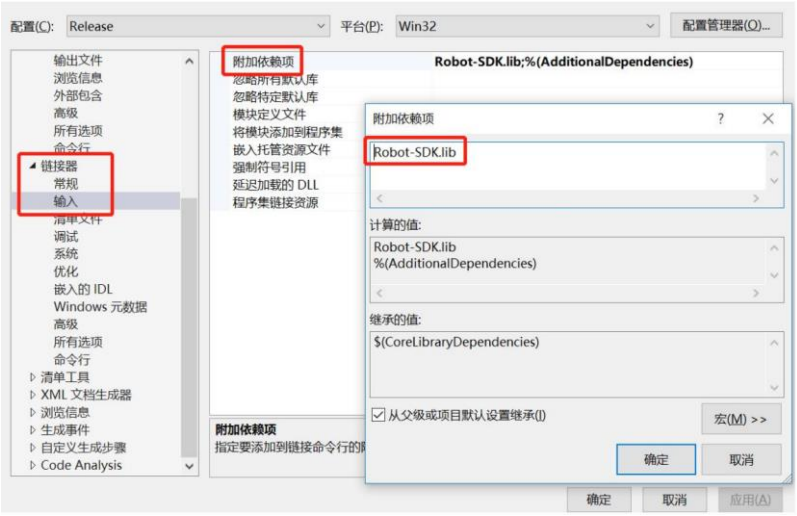
A．在 C++项目→属性→C/C++→常规→附加包含目录，添加“lib&hpp”文件夹路径



B．在 C++项目→属性→链接器→常规→附加库目录，添加“lib&hpp”文件夹路径



C. 在 C++项目→属性→链接器→输入→附加依赖项，添加“Robot-SDK.lib”



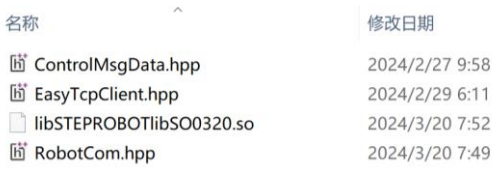
D. 全部设置好后，点击确定

E. 在程序中添加“RobotCom.hpp”头文件

注：如使用时报错，注意属性中的几处设置（x64 时选择 x64）：

Linux 下调用：

1）将以下文件放到一个文件夹里，如保存到“lib&hpp”文件夹中，



2）调用

不同的开发环境调用方法不同，大致思路是将库和头文件在项目中配置好，使得当前项目可以调用头文件和库。在此省略调用详细步骤。

注：以下以 Windows 下代码进行示例，请注意 Linux 下使用时部分语法区别。

2.1.2 定义

```
#include "RobotCom.hpp"
using namespace STEPROBOTSDK;
```

```
// 定义客户端（必须）
RobotComClient robotClient;
```

2.1.3 初始化

函数	int robotClient.init(ip,port)	
输入	ip	std::string，机器人地址
输入	port	std::string，机器人端口
返回值(int)	0：正常；-1：建立失败；-2：数据收发异常；-3：通讯断开	

机器人 ip 地址：192.168.39.220（若地址更改，使用更改后的地址）

端口号：30312（固定端口，不可更改）

示例：

```
int main() {
  int result = 0;
  //初始化，连接服务器
  result = robotClient.init("192.168.39.220", 30312);
  //检查初始化是否成功
  if (result != 0)
  {
    std::cout <<"初始化失败，结果为: "<< result << std::endl;
  }
}
```

注：同时只能有一个客户端连接 30312 端口！

连接 STEPStudio 时，ip 为 STEPStudio 所在电脑的 ip，如程序编写在同一台电脑，则地址为：127.0.0.1。

2.1.4 关闭连接

函数	int close()
返回值	0：关闭完成；

示例：

```
result = robotClient.close();
std::cout << "关闭连接完成。" << std::endl;
```

2.1.5 获取版本号

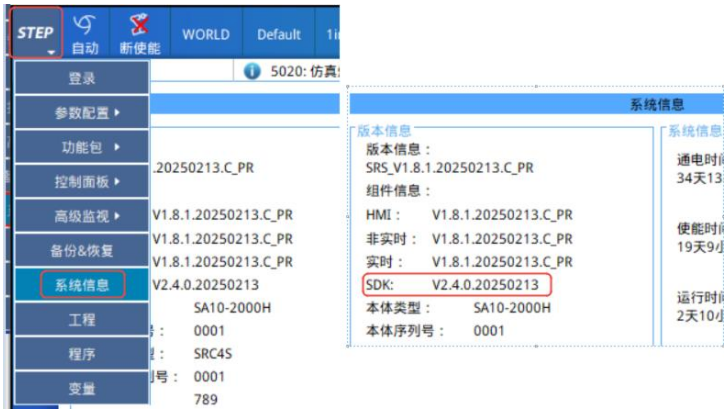
函数	std::string SDKVersion();
返回值	std::string，版本信息

示例：

```
std::string sdkversion;  //定义版本号变量
sdkversion = robotClient.SDKVersion();  //获取当前版本号
```

获取的版本号示例：SDK-V2.4.0.20250213，可与示教器系统信息上展示的版本对应查看，是否

可以配套使用



2.1.6 连接示例

```
#include "RobotCom.hpp"
using namespace STEPROBOTSDK;
// 定义客户端（必须）
RobotComClient robotClient;
int main() {
  int result = 0;
  std::string sdkversion;  //定义版本号变量
  std::cout << "当前 SDK 版本号为:" << sdkversion << std::endl;
  //可以检查一下版本是否和机器人版本匹配
  //初始化，连接服务器
  result = robotClient.init("192.168.39.220", 30312); //仿真软件时为“127.0.0.1”
  //检查初始化是否成功
  if (result != 0)
  {
    std::cout <<"初始化失败，结果为: "<< result << std::endl;
  }
  else
  {
    std::cout << "初始化完成！" << std::endl;
    sdkversion = robotClient.SDKVersion();  //获取当前版本号
  }
}
//使用完成，关闭连接
result = robotClient.close();
std::cout << "关闭连接完成。" << std::endl;
//需要使用时，重新初始化
```

2.2 状态反馈接口

2.2.1 获取当前工程名

函数	std::string getProjectName()
返回值	std::string，工程名

示例：

```
std::string projectname;                //定义变量
projectname = robotClient.getProjectName();  //获取当前工程名
```

2.2.2 获取当前程序名

函数	std::string getProgramName()
返回值	std::string，程序名

示例：

```
std::string programname;                //定义变量
programname = robotClient.getProgramName();  //获取当前程序名
```

2.2.3 获取用户坐标系名称

函数	std::string getTeachRefSysNam()
返回值	std::string，用户坐标系名称

示例：

```
std::string teachrefsysname;                //定义变量
teachrefsysname = robotClient.getTeachRefSysNam();  //获取用户坐标系名称
```

2.2.4 获取当前工具名称

函数	std::string getToolName()
返回值	std::string，程序名

示例：

```
std::string toolname;                //定义变量
toolname = robotClient.getToolName();  //获取当前工具名
```

2.2.5 获取当前错误信息

函数	MessageData getMessageData()
返回值	MessageData，错误信息

MessageData 说明：

MESSAGETYPE	m_MessageType;	错误类型(枚举)：eInfo,eWarning,eError
int	m_MessageID;	错误 ID
char	m_MessageString[96];	错误信息
double	m_MessageTime	错误时间，单位分钟，自 1970/1/1/8:00 算起
char	m_MessageSource[96]	错误来源

示例：

```
MessageData errormassage;                //定义变量
errormassage = robotClient.getMessageData();        //获取错误信息
```

2.2.6 获取机器人DI

函数	RobotDI getRobotDI();
返回值	RobotDI，机器人 DI

RobotDI 说明：

unsigned long int	m_DI[64];	按位取值，共 2048 个 DI， Linux 下数据类型为 unsigned int
unsigned long int	m_DIMask[64];	仿真状态值 Linux 下数据类型为 unsigned int

示例：

```
RobotDI robotdi = {};                //定义变量
for (int i = 0; i < 64; i++) {
    robotdi.m_DI[i] = robotClient.getRobotDI().m_DI[i]; //获取DI值
}
int di1 = robotdi.m_DI[0] & 0x01;     //读取DI1的值,1为TRUE，0为FALSE
int di10 = (robotdi.m_DI[0] >> 9) & 0x01;//读取DI10的值,1为TRUE，0为FALSE
int di20 = (robotdi.m_DI[0] >> 19) & 0x01;//读取DI20的值,1为TRUE，0为FALSE
int di33 = (robotdi.m_DI[1]) & 0x01;//读取 DI33 的值,1 为 TRUE，0 为 FALSE
```

2.2.7 获取机器人DO

函数	RobotDO getRobotDO();
返回值	RobotDO，机器人 DO

RobotDO 说明：

unsigned long int	m_DO[64];	按位取值，共 2048 个 DO Linux 下数据类型为 unsigned int
unsigned long int	m_ DOMask[64];	强制状态值 Linux 下数据类型为 unsigned int

示例：

```
RobotDO robotdo = {};                //定义变量
for (int i = 0; i < 64; i++) {
    robotdo.m_DO[i] = robotClient.getRobotDO().m_DO[i]; //获取DO值
}
int do1 = robotdo.m_DO[0] & 0x01;     //读取DO1的值,1为TRUE，0为FALSE
int do10 = (robotdo.m_DO[0] >> 9) & 0x01;//读取DO10的值,1为TRUE，0为FALSE
int do20 = (robotdo.m_DO[0] >> 19) & 0x01;//读取DO20的值,1为TRUE，0为FALSE
int do33 = (robotdo.m_DO[1]) & 0x01;//读取 DO33 的值,1 为 TRUE，0 为 FALSE
```

2.2.8 获取机器人AI

函数	RobotAI getRobotAI();

返回值	RobotAI，机器人 AI

RobotAI 说明：

short	m_AI[32];	32 个 AI
unsigned long int	m_AIMask [32];	仿真状态值 Linux 下数据类型为 unsigned int

示例：

```
RobotAI robotai = {};                //定义变量
for (int i = 0; i < 32; i++) {
    robotai.m_AI[i] = robotClient.getRobotAI().m_AI[i]; //获取AI值
}
short ai1 = robotai.m_AI[0];     //读取 AI1 的值
```

2.2.9 获取机器人AO

函数	RobotAO getRobotAO();
返回值	RobotAO，机器人 AO

RobotDO 说明：

short	m_AO[32];	32 个 AO
unsigned long int	m_AOMask [32];	仿真状态值 Linux 下数据类型为 unsigned int

示例：

```
RobotAO robotao = {};                //定义变量
for (int i = 0; i < 32; i++) {
    robotao.m_AO[i] = robotClient.getRobotAO().m_AO[i]; //获取AO值
}
short ao1 = robotao.m_AO[0];    //读取 AO1 的值
```

2.2.10 获取实际笛卡尔动态信息

函数	CartDynState getActCartDynState();
返回值	CartDynState，动态信息

CartDynState 说明：

double	m_Vel;	速度
double	m_Acc;	加速度
double	m_Dec;	减速度
double	m_Jerk;	加加速度

示例：

```
CartDynState cartdynstate;              //定义变量
cartdynstate = robotClient.getActCartDynState();        //获取动态信息
double vel = cartdynstate.m_Vel;         //读取速度值
```

2.2.11 获取当前运行模式

函数	OPERATIONMODE getOperationMode();
返回值	OPERATIONMODE，运行模式

OPERATIONMODE 说明：

OPERATIONMODE(enum）	m_OperationMode;	eManualHigh = 1,//高速手动模式 eAutomatic, // 自动模式 eAutomaticExternal // 外部自动，用于有上层控制系统的情况

示例：

```
OPERATIONMODE operationmode;                //定义变量
operationmode = robotClient.getOperationMode();          //获取运行模式
```

2.2.12 获取使能状态

函数	int getMotorEnableState();
返回值	int，使能信息，0:未上使能,1:上使能

示例：

```
int enablestate = robotClient.getMotorEnableState();// 获取使能状态
```

2.2.13 获取当前点动坐标系

函数	REFSYS getJogRefSys ();
返回值	REFSYS，运行模式

REFSYS 说明：

REFSYS(enum）	m_JogRefSys;	当前点动坐标系 eJoints, //关节坐标系 eWorld, //世界坐标系 eBase, //基坐标系 eTool, //工具坐标系 eCustom, //工作台坐标系/工件坐标系/用户坐标系 eGo, eTTS, //焊炬点动坐标系

示例：

```
REFSYS jogrefsys;                //定义变量
jogrefsys = robotClient.getJogRefSys();          //获取点动坐标系
```

2.2.14 获取全局速度

函数	double getOverride();
返回值	double，全局速度

示例：

```
double override = robotClient.getOverride();        //获取全局变量
```

2.2.15 获取程序运行状态

函数	PROGRAMSTATE getProgramState();
返回值	PROGRAMSTATE，运行状态

PROGRAMSTATE 说明

PROGRAMSTATE(enum）	m_ProgramState;	机器人状态 eRun,ePause,eStop

示例：

```
PROGRAMSTATE programstate;                //定义变量
programstate = robotClient.getProgramState();        //获取程序运行状态
```

2.2.16 获取程序运行模式

函数	PROGRAMMODE getProgramMode();
返回值	PROGRAMMODE，运行模式

PROGRAMMODE 说明

PROGRAMMODE(enum）	m_ProgramMode;	当前运行模式 eContinue,eStep,eMotionStep

示例：

```
PROGRAMMODE programmode;              //定义变量
programmode = robotClient.getProgramMode();        //获取程序运行模式
```

2.2.17 获取当前所在行

函数	int getCurrentLine();
返回值	int，当前所在行

示例：

```
int currentline = robotClient.getCurrentLine();      //获取当前所在行
```

2.2.18 获取当前关节位置

函数	AXISPOS getAxisPos();
返回值	AXISPOS，关节信息

AXISPOS 说明

JointsPos	m_Joint[6];	J1/J2/J3/J4/J5/J6
AuxPos	m_AuxPos[6];	附加轴位置

示例：

```
AXISPOS currentaxispos = robotClient.getAxisPos();   //获取当前笛卡尔位置
double posJ1 = currentaxispos.m_Joint[0];        //X坐标
double posJ2 = currentaxispos.m_Joint[1];        //Y坐标
double posJ3 = currentaxispos.m_Joint[2];        //Z坐标
double posJ4 = currentaxispos.m_Joint[3];        //A坐标
double posJ5 = currentaxispos.m_Joint[4];        //B坐标
double posJ6 = currentaxispos.m_Joint[5];        //C坐标
double posAux1 = currentaxispos.m_AuxJoint[0];        //附加轴1位置
double posAux2 = currentaxispos.m_AuxJoint[1];        //附加轴2位置
double posAux3 = currentaxispos.m_AuxJoint[2];        //附加轴3位置
double posAux4 = currentaxispos.m_AuxJoint[3];        //附加轴4位置
double posAux5 = currentaxispos.m_AuxJoint[4];        //附加轴5位置
double posAux6 = currentaxispos.m_AuxJoint[5];        //附加轴 6 位置
```

2.2.19 获取当前世界笛卡尔位置

函数	RobotCartPos getCartPosWorld();
返回值	RobotCartPos，笛卡尔位置信息

RobotCartPos 说明

double	cart[6];	坐标
unsigned int	m_Mode	模式

示例：

```
RobotCartPos cartposworld = robotClient.getCartPosWorld();
//获取当前世界笛卡尔位置
double posX = cartposworld.cart[0];     //X坐标
double posY = cartposworld.cart[1];     //Y坐标
double posZ = cartposworld.cart[2];     //Z坐标
double posA = cartposworld.cart[3];     //A坐标
double posB = cartposworld.cart[4];     //B坐标
double posC = cartposworld.cart[5];     //C坐标
unsigned int Mode = cartposworld.m_Mode; //模式
```

2.2.20 获取当前基坐标笛卡尔位置

函数	RobotCartPos getCartPosRobotBase();
返回值	RobotCartPos ，笛卡尔位置信息

示例：

```
RobotCartPos cartposbase = robotClient.getCartPosRobotBase();
//获取当前基坐标笛卡尔位置
```

2.2.21 获取当前用户坐标笛卡尔位置

函数	RobotCartPos getCartPosUserRef();
返回值	RobotCartPos ，笛卡尔位置信息

示例：

```
RobotCartPos cartposbase = robotClient.getCartPosUserRef();
//获取当前用户坐标笛卡尔位置
```

2.2.22 获取当前世界坐标笛卡尔位置和旋转矩阵

函数	HomoMatrix getCartPosR();
返回值	HomoMatrix，笛卡尔位置+旋转矩阵

HomoMatrix 说明

rot	double Matrix3[3][3]	转转矩阵
xyz	double Vector3[3]	世界坐标

示例：

```
HomoMatrix cartposr = robotClient.getCartPosR();
//获取当前带旋转矩阵的笛卡尔位置
```

2.2.23 获取运行时间

函数	unsigned int getRobotRunningTime();
返回值	unsigned int，机器人运行时间，单位：分钟

示例：

```
int runningtime = robotClient.getRobotRunningTime();  //获取运行时间
```

2.2.24 获取开机时间

函数	int getLifeTime();
返回值	int，开机时间，单位：分钟

示例：

```
int lifetime = robotClient.getLifeTime();    //获取开机时间
```

2.2.25 获取焊接模式

函数	ARCWELDINGMODE getWeldingMode();
返回值	ARCWELDINGMODE，焊接模式，枚举型

ARCWELDINGMODE 说明：

ARCWELDINGMODE (enum）	eSimuWelding, eHotWelding, eCheckPath	仿真焊 实焊 空跑

示例：

```
ARCWELDINGMODE arcmode == robotClient.getWeldingMode();
```

2.3 基础命令接口

2.3.1 获取机器人信息

函数	int RequestRobotInfoCmd(robotinfo)	
输出	robotinfo	RobotInfoStruct
返回值(int)	0：执行成功；-1：执行失败；-2：数据收发异常；-3：通讯断开	

RobotInfoStruct 说明：

char	m_Name[32];	机器人名称
char	m_RobotSNo[32];	机器人序列号

示例：

```
RobotInfoStruct robotinfo = {};          //定义机器人信息结构体变量
std::string robotname;                     //定义机器人名称变量
std::string robotNo;                       //定义机器人序列号变量
result = robotClient.RequestRobotInfoCmd(robotinfo); //获取机器人信息
if (result != 0) //检查是否成功
{
    std::cout << "获取机器人信息失败，结果为:" << result << std::endl;
}
else
{
    robotname = robotinfo.m_Name;        //机器人名称赋值（可直接调用）
    robotNo = robotinfo.m_RobotSNo;    //机器人序列号赋值（可直接调用）
}
```

2.3.2 获取控制柜信息

函数	int RequestElecBoxInfoCmd(elecboxinfo)	
输出	elecboxinfo	ElecBoxInfoStruct
返回值(int)	0：执行成功；-1：执行失败；-2：数据收发异常；-3：通讯断开	

ElecBoxInfoStruct 说明：

char	m_ElecBoxName [32];	控制柜型号
char	m_ElecBoxSNo [32];	控制柜序列号

示例：

```
int result= 0;
ElecBoxInfoStruct elecboxinfo = {};          //定义控制柜信息结构体变量
std::string elecboxname;                     //定义控制柜型号变量
std::string elecboxNo;                       //定义控制柜序列号变量
result = robotClient.RequestElecBoxInfoCmd(elecboxinfo); //获取机器人信息
if (result != 0) //检查是否成功
{
    std::cout << "获取控制柜信息失败，结果为:" << result << std::endl;
}
else
{
```

```
        elecboxname = elecboxinfo.m_ElecBoxName; //机器人名称赋值（可直接调用）
        elecboxNo = elecboxinfo.m_ElecBoxSNo;  //机器人序列号赋值（可直接调用）
    }
```

2.3.3 机器人上下使能（间隔300ms）

函数	int EnableMotorCmd();
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开

注：若机器人使用的是三代柜，使能与其他操作的间隔时间为 300ms，少于这个时间无法执行下一步

示例 1：上使能

```
    int result= 0;
    //当前机器人未使能状态
    if (robotClient.getMotorEnableState() == 0)
    {
        result = robotClient.EnableMotorCmd();    //上使能
        if (result != 0) //检查是否成功
        {
            std::cout << "上使能执行失败，结果为:" << result << std::endl;
        }
        else
        {
            //使能执行需要时间，等待上使能成功
            for (int i = 0; i < 20; i++)
            {
                if (robotClient.getMotorEnableState() == 1)
                {
                    break; //如果为使能状态，跳出for循环
                }
                Sleep(50); //50ms后再次获取状态
            }
            Sleep(300);//使能到其他操作的间隔时间为300ms，少于这个时间无法执行下一步
        }
    }
```

示例 2：断使能

```
    int result= 0;
    //当前机器人使能状态
    if (robotClient.getMotorEnableState() == 1)
    {
        result = robotClient.EnableMotorCmd();    //断使能
        if (result != 0) //检查是否成功
        {
            std::cout << "断使能执行失败，结果为:" << result << std::endl;
        }
```

```
        else
        {
            //断使能执行需要时间，等待断使能成功
            for (int i = 0; i < 20; i++)
            {
                if (robotClient.getMotorEnableState() == 0)
                {
                    break; //如果为未使能状态，跳出for循环
                }
                Sleep(50); //50ms后再次获取状态
            }
        }
    }
```

2.3.4 设置操作模式（功能较多，可详细查看）

函数	int SetModeCmd(mode, is_blocked);	
输入	mode	MODEKEY，模式设置
输入	is_blocked	bool，阻塞标志。默认 false，当前命令完成后给返回值，不用再进行状态判断，仅对手自动模式切换、开始停止有效（MANUAL、AUTO、AUTO_EXT、START、STOP）
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

MODEKEY 说明：

MANUAL	AUTO	AUTO_EXT	START	STOP	MSTOP
手动	自动	外部自动	开始	停止	停止点动
POSITIVE1	POSITIVE2	POSITIVE3	POSITIVE4	POSITIVE5	POSITIVE6
X/J1 +	Y/J2 +	Z/J3 +	A/J4 +	B/J5 +	C/J6 +
NEGATIVE1	NEGATIVE2	NEGATIVE3	NEGATIVE4	NEGATIVE5	NEGATIVE6
X/J1 -	Y/J2 -	Z/J3 -	A/J4 -	B/J5 -	C/J6 -
WIREOUT	WIREBACK	WIREGAS			
送丝	抽丝	送气			

示例 1：设置手动模式

```
int result= 0;
result = robotClient.SetModeCmd(MANUAL); //操作模式设置为手动
if (result != 0) //检查是否成功
{
    std::cout << "操作模式设置失败，结果为:" << result << std::endl;
}
else
{
    for (int i = 0; i < 20; i++)
    {
        if (robotClient.getOperationMode() == eManualHigh)
```

```
            {
                break; //如果为手动状态，跳出for循环
            }
            Sleep(50); //50ms后再次获取状态
        }
    }
```

示例 2：设置自动模式

```
    int result= 0;
    result = robotClient.SetModeCmd(AUTO); //操作模式设置为自动
    if (result != 0) //检查是否成功
    {
        std::cout << "操作模式设置失败，结果为:" << result << std::endl;
    }
    else
    {
        for (int i = 0; i < 20; i++)
        {
            if (robotClient.getOperationMode() == eAutomatic)
            {
                break; //如果为自动状态，跳出for循环
            }
            Sleep(50); //50ms后再次获取状态
        }
    }
```

示例 3：启动程序（非外部模式）

```
    int result= 0;
    if (robotClient.getOperationMode() != eAutomaticExternal)  //机器人非外部状态
    {
        if (robotClient.getMotorEnableState() == 1)  //机器人使能状态
        {
            result = robotClient.SetModeCmd(START); //启动加载的程序
            if (result != 0) //检查是否成功
            {
                std::cout << "程序启动执行失败，结果为:" << result << std::endl;
            }
            else
            {
                for (int i = 0; i < 20; i++)
                {
                    if (robotClient.getProgramState() == eRun)
                    {
                        break; //如果为运行状态，跳出for循环
                    }
                    Sleep(50); //50ms后再次获取状态
```

```
                }
            }
        }
    }
```

示例 4：停止程序（非外部模式）

```
int result= 0;
if (robotClient.getOperationMode() != eAutomaticExternal)  //机器人非外部状态
{
    if (robotClient.getProgramState() == eRun)  //机器人运行状态
    {
        result = robotClient.SetModeCmd(STOP); //停止加载的程序
        if (result != 0) //检查是否成功
        {
            std::cout << "程序停止执行失败，结果为:" << result << std::endl;
        }
        else
        {
            for (int i = 0; i < 20; i++)
            {
                if (robotClient.getProgramState() == eStop)
                {
                    break; //如果为停止状态，跳出for循环
                }
                Sleep(50); //50ms后再次获取状态
            }
        }
    }
}
```

示例 5：点动机器人，必须在手动模式下

```
int result= 0;
//如果不是手动状态，设置为手动
if (robotClient.getOperationMode() != eManualHigh)
{
    result = robotClient.SetModeCmd(MANUAL); //操作模式设置为手动
    if (result != 0) //检查是否成功
    {
        std::cout << "操作模式设置失败，结果为:" << result << std::endl;
    }
    else
    {
        for (int i = 0; i < 20; i++)
        {
            if (robotClient.getOperationMode() == eManualHigh)
            {
```

```
                    break; //如果为手动状态，跳出for循环
                }
                Sleep(50); //50ms后再次获取状态
            }
        }
    }
    //设置点动坐标系
    //如果不是关节点动坐标系（这里根据需求设置），设置为关节
    if (robotClient.getJogRefSys() != eJoints)
    {
        result = robotClient.JogRefSystemCmd("JOINTS"); //设置点动坐标系为：关节
        if (result != 0) //检查是否成功
        {
            std::cout << "点动坐标系设置失败，结果为:" << result << std::endl;
        }
        else
        {
            for (int i = 0; i < 20; i++)
            {
                if (robotClient.getJogRefSys() == eJoints)
                {
                    break; //如果为关节点动坐标系，跳出for循环
                }
                Sleep(50); //50ms后再次获取状态
            }
        }
    }
    //上使能
    //当前机器人未使能状态
    if (robotClient.getMotorEnableState() == 0)
    {
        result = robotClient.EnableMotorCmd();    //上使能
        if (result != 0) //检查是否成功
        {
            std::cout << "上使能执行失败，结果为:" << result << std::endl;
        }
        else
        {
            //使能执行需要时间，等待上使能成功
            for (int i = 0; i < 20; i++)
            {
                if (robotClient.getMotorEnableState() == 1)
                {
                    break; //如果为使能状态，跳出for循环
```

```
                }
                Sleep(50); //50ms后再次获取状态
            }
            Sleep(300);//使能到其他操作的间隔时间为300ms，少于这个时间无法执行下一步
        }
    }
    //点动机器人关节1，正方向运行1s后停止
    result = robotClient.SetModeCmd(POSITIVE1);
    if (result != 0) //检查是否成功
    {
        std::cout << "J1+失败，结果为:" << result << std::endl;
    }
    else
    {
        Sleep(1000); //运行一段时间，执行需要时间，不完全=1秒
        //停止点动
        result = robotClient.SetModeCmd(MSTOP);
        if (result != 0) //检查是否成功
        {
            std::cout << "停止点动失败，结果为:" << result << std::endl;
        }
        else
        {
            //使能执行需要时间，等待机器人停止
            for (int i = 0; i < 20; i++)
            {
                if (robotClient.getProgramState() == eStop)
                {
                    break; //如果为停止状态，跳出for循环
                }
                Sleep(50); //50ms后再次获取状态
            }
        }
    }
    //断使能（根据需求）
    if (robotClient.getMotorEnableState() == 1)
    {
        result = robotClient.EnableMotorCmd();    //断使能
        if (result != 0) //检查是否成功
        {
            std::cout << "断使能执行失败，结果为:" << result << std::endl;
        }
        else
        {
```

```
            //断使能执行需要时间，等待断使能成功
            for (int i = 0; i < 20; i++)
            {
                if (robotClient.getMotorEnableState() == 0)
                {
                    break; //如果为未使能状态，跳出for循环
                }
                Sleep(50); //50ms后再次获取状态
            }
        }
    }
```

2.3.5 加载程序

函数	int ProgramLoadCmd(project,program,errorline, is_blocked);	
输入	project	std::string，工程
输入	program	std::string，程序
输出	errorline	std::vector<int>，错误行号
输入	is_blocked	bool，阻塞标志。默认 false，当前命令完成后给返回值，不用再进行状态判断
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：执行加载命令时，当前需无程序加载

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::vector<int>  errorline;                    //新建错误行号变量
result = robotClient.ProgramLoadCmd(project, program, errorline);    //加载程序
if (result != 0) //检查是否成功
{
    std::cout << "加载程序失败，结果为:" << result << std::endl;
}
else
{
    //加载程序执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if (robotClient.getProgramName() == program)
        {
            break; //如果程序名为加载程序名状态，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
//如加载的程序有语法错误，返回具体错误行号
//行号为-1 时，错误信息较多，请到示教器上查看
```

2.3.6 卸载程序

函数	int ProgramKillCmd(project, program, is_blocked);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	is_blocked	bool，阻塞标志。默认 false，当前命令完成后给返回值，不用再进行状态判断
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例 1：

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::vector<int>  errorline;                    //新建错误行号变量
result = robotClient.ProgramKillCmd(project, program);    //卸载程序
if (result != 0) //检查是否成功
{
    std::cout << "卸载程序失败，结果为:" << result << std::endl;
}
else
{
    //卸载程序执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if (robotClient.getProgramName() == "")
        {
            break; //如果正确，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
```

2.3.7 清除错误

函数	int AllAlarmConfirmCmd();
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开

示例：

```
int result= 0;
result = robotClient.AllAlarmConfirmCmd();    //清错
if (result != 0) //检查是否成功
{
    std::cout << "清错失败，结果为:" << result << std::endl;
}
```

2.4 参数设置接口

2.4.1 设置用户坐标系

函数	int ReferenceSystemSetCmd(refName)	
输入	refName	std::string，例：“WORLD”“ROBOTBASE”
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result= 0;
std::string refName = "ROBOTBASE";      //示教器中已有该用户坐标系
result = robotClient.ReferenceSystemSetCmd(refName); //设置用户坐标系为：ROBOTBASE
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if (robotClient.getTeachRefSysNam() == refName)
        {
            break; //如果正确，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
```

2.4.2 设置工具坐标系

函数	int ToolSetCmd(tool)	
输入	tool	std::string，例：“Default”
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result= 0;
std::string tool = "Default";      //示教器中已有该工具
result = robotClient.ToolSetCmd(tool); //设置工具为：Default
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
```

```
        {
            if (robotClient.getToolName() == tool)
            {
                break; //如果正确，跳出for循环
            }
            Sleep(50); //50ms后再次获取状态
        }
    }
```

2.4.3 设置全局速度

函数	int OverrideSetCmd(ori);	
输入	ori	Double，速度百分比
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result= 0;
double ori = 10;     //ori为0.1~100的double值，1以上四舍五入成整
result = robotClient.OverrideSetCmd(ori); //设置全局速度为10%
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if (robotClient.getOverride() == ori)
        {
            break; //如果正确，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
```

2.4.4 设置点动坐标系

函数	int JogRefSystemCmd(refName);	
输入	refName	std::string ， "WORLD" "JOINTS" "UserRef" "TOOL" "TTS"
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result= 0;
result = robotClient.JogRefSystemCmd("JOINTS"); //设置全局速度为10%
if (result != 0) //检查是否成功
```

```
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if (robotClient.getJogRefSys() == eJoints)
        {
            break; //如果正确，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
```

2.4.5 设置当前运行模式

函数	int ProgramRunModeCmd(mode);	
输入	mode	int，0 连续、1 单步、2 运动单步
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result= 0;
int mode = 0;
result = robotClient.ProgramRunModeCmd(mode); //设置当前运行模式为：连续
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if (robotClient.getProgramMode() == eContinue)
        {
            break; //如果正确，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
```

2.4.6 设置程序行号

函数	int SetpcCmd(linNum);

输入	linNum	int，行号
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result= 0;
int linNum = 5;
result = robotClient.SetpcCmd(linNum); //设置行号为5
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if (robotClient.getCurrentLine() == linNum)
        {
            break; //如果正确，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
```

2.4.7 设置DI

1）强制 DI 状态：

函数	int DigitalInputSimulationCmd(port,data);	
输入	port	int，端口号 0~2047（对应 DI 1~2048）
输入	data	int，状态，0 为 FALSE，1 为 TRUE
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例 1：

```
int result= 0;
int port = 5;
int data = 1;
result = robotClient.DigitalInputSimulationCmd(port,data);//强制DI6为TRUE
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
```

```
            if ((robotClient.getRobotDI().m_DI[0] >> 5 & 0x01) == data)
            {
                break; //如果正确，跳出for循环
            }
            Sleep(50); //50ms后再次获取状态
        }
    }
```

2）取消 DI 强制选项（**会复位 DI**）：

函数	int DigitalInputSimulationRemovalCmd(port);	
输入	port	int，端口号 0~2047（对应 DI 1~2048）
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例 1：

```
int result= 0;
int port = 5;
result = robotClient.DigitalInputSimulationRemovalCmd(port);//取消DI6强制选项
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if ((robotClient.getRobotDI().m_DIMask[0] >> 5 & 0x01) == data)
        {
            break; //如果正确，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
```

2.4.8 设置DO

1）强制 DO 状态：

函数	int DigitalOutputEnforceCmd(port,data);	
输入	port	int，端口号 0~2047（对应 DO 1~2048）
输入	data	int，状态，0 为 FALSE，1 为 TRUE
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result= 0;
int port = 5;
int data = 1;
result = robotClient.DigitalOutputEnforceCmd(port,data);//强制DI6为TRUE
```

```
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if ((robotClient.getRobotDO().m_DO[0] >> 5 & 0x01) == data)
        {
            break; //如果正确，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
```

2）取消 DO 强制选项（**不改变 DO 状态**）：

函数	int DigitalOutputEnforceRemovalCmd(port);	
输入	port	int，端口号 0~2047（对应 DO 1~2048）
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result= 0;
result = robotClient.DigitalOutputEnforceRemovalCmd(port);//取消DI6强制选项
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if ((robotClient.getRobotDO().m_DOMask[0] >> 5 & 0x01) == 0)
        {
            break; //如果正确，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
```

2.4.9 设置AI

1）强制 AI 状态：

函数	int AnalogInputSimulationCmd(port,data);

输入	port	int，端口号 0~31（对应 AI 1~32）
输入	data	short
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例 1：

```
int result= 0;
int port = 5;
int data = 10;
result = robotClient.AnalogInputSimulationCmd(port,data);//强制AI6为10
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if (robotClient.getRobotAI().m_AI[port] == data)
        {
            break; //如果正确，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
```

2）取消 AI 强制选项（**会复位 AI**）：

函数	int AnalogInputSimulationRemovalCmd(port);	
输入	port	int，端口号 0~31（对应 AI 1~32）
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result= 0;
int port = 5;
result = robotClient.AnalogInputSimulationRemovalCmd(port);//取消AI6强制选项
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if (robotClient.getRobotAI().m_AIMask == 0)
        {
            break; //如果正确，跳出for循环
```

```
            }
            Sleep(50); //50ms后再次获取状态
        }
    }
```

2.4.10 设置AO

1）强制 AO 状态：

函数	int AnalogOutputEnforceCmd(port,data);	
输入	port	int，端口号 0~31（对应 AO 1~32）
输入	data	short
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例 1：

```
int result= 0;
int port = 5;
int data = 10;
result = robotClient.AnalogOutputEnforceCmd(port,data);//强制AO6为10
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if (robotClient.getRobotAO().m_AO[port] == data)
        {
            break; //如果正确，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
```

2）取消 AO 强制选项（**不改变 AO 状态**）：

函数	int AnalogOutputEnforceRemovalCmd (port);	
输入	port	int，端口号 0~31（对应 AO 1~32）
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例 1：

```
int result= 0;
int port = 5;
result = robotClient.AnalogOutputEnforceRemovalCmd(port);//取消AI6强制选项
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
```

```
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if (robotClient.getRobotAO().m_AOMask == 0)
        {
            break; //如果正确，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
```

2.5 变量设置接口

2.5.1 变量设置说明（必读）

1）所有**修改变量**接口使用，均要求**示教器程序处于非运行状态**，修改变量命令才可以**成功执行**；

2）**在线修改变量**接口可在**示教器程序运行中**使用，但使用时注意示教器程序的**预读功能**，否则可能会出现在线修改变量时，示教器程序已经预读过该变量，导致在线修改变量虽然成功了，但**没有达到期望的效果**；

3）修改全局变量时，工程名为“_global”，程序名为“_project”。

4）修改保持变量时，工程名为“_global”，程序名为“_record”。

2.5.2 修改BOOL值

函数	int VariableBoolModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	bool，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "bool0";          //示教器中已有该变量
bool value = TRUE;
result = robotClient.VariableBoolModifyCmd(project, program, variable, value);
//设置bool0为TRUE
if (result != 0) //检查是否成功
{
    std::cout << "修改bool值失败，结果为:" << result << std::endl;
}
```

2.5.3 在线修改BOOL值

函数	int ProcessVariableBoolModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	bool，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "bool0";          //示教器中已有该变量
bool value = TRUE;
result = robotClient.ProcessVariableBoolModifyCmd(project, program, variable,
value);
//设置bool0为TRUE
if (result != 0) //检查是否成功
{
    std::cout << "在线修改bool值失败，结果为:" << result << std::endl;
}
```

2.5.4 修改INT值

函数	int VariableIntModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	int，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "int0";          //示教器中已有该变量
int value = 10;
result = robotClient.VariableIntModifyCmd(project, program, variable, value);
//设置int0为10
if (result != 0) //检查是否成功
{
    std::cout << "修改int值失败，结果为:" << result << std::endl;
}
```

2.5.5 在线修改INT值

函数	int ProcessVariableIntModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	int，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result= 0;
```

```
    std::string project = "project0";        //示教器中已有该工程
    std::string program = "program0";      //示教器中已有该程序
    std::string variable = "int0";          //示教器中已有该变量
    int value = 10;
    result = robotClient.ProcessVariableIntModifyCmd(project, program, variable,
value);
    //设置int0为10
    if (result != 0) //检查是否成功
    {
        std::cout << "在线修改int值失败，结果为:" << result << std::endl;
    }
```

2.5.6 修改REAL值

函数	int VariableRealModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	float，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
    int result= 0;
    std::string project = "project0";        //示教器中已有该工程
    std::string program = "program0";      //示教器中已有该程序
    std::string variable = "real0";          //示教器中已有该变量
    float value = 10.01;
    result = robotClient.VariableRealModifyCmd(project, program, variable, value);
    //设置real0为10.01
    if (result != 0) //检查是否成功
    {
        std::cout << "修改real0值失败，结果为:" << result << std::endl;
    }
```

2.5.7 在线修改REAL值

函数	int ProcessVariableRealModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	float，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
    int result= 0;
    std::string project = "project0";        //示教器中已有该工程
```

```
    std::string program = "program0";      //示教器中已有该程序
    std::string variable = "real0";          //示教器中已有该变量
    float value = 10.01;
    result = robotClient.ProcessVariableRealModifyCmd(project, program, variable,
value);
    //设置real0为10.01
    if (result != 0) //检查是否成功
    {
        std::cout << "在线修改real0值失败，结果为:" << result << std::endl;
    }
```

2.5.8 修改STRING值

函数	int VariableStringModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	std::string，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
    int result= 0;
    std::string project = "project0";        //示教器中已有该工程
    std::string program = "program0";      //示教器中已有该程序
    std::string variable = "string0";          //示教器中已有该变量
    std::string value = "STEP Robot";
    result = robotClient.VariableStringModifyCmd(project, program, variable, value);
    //设置string0为STEP Robot
    if (result != 0) //检查是否成功
    {
        std::cout << "修改string值失败，结果为:" << result << std::endl;
    }
```

2.5.9 在线修改STRING值

函数	int ProcessVariableStringModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	std::string，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
    int result= 0;
```

```
    std::string project = "project0";        //示教器中已有该工程
    std::string program = "program0";      //示教器中已有该程序
    std::string variable = "string0";          //示教器中已有该变量
    std::string value = "STEP Robot";
    result = robotClient.ProcessVariableStringModifyCmd(project, program, variable,
value);
    //设置string0为STEP Robot
    if (result != 0) //检查是否成功
    {
        std::cout << "修改string值失败，结果为:" << result << std::endl;
    }
```

2.5.10 修改AXISPOS值

函数	int VariableAxisposModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	AXISPOS，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

AXISPOS 说明：

JointsPos	m_Joint;	关节位置
	JointsPos[6]	Double 数组 0~5 表示 1~6 轴关节位置
AuxPos	m_AuxJoint;	附加轴位置
	AuxPos[6];	Double 数组 0~5 表示 7~12 轴关节位置

示例 1：

```
    int result= 0;
    std::string project = "project0";        //示教器中已有该工程
    std::string program = "program0";      //示教器中已有该程序
    std::string variable = "ap0";          //示教器中已有该变量
    AXISPOS value = { 10,-10,10,-10,10,-10,0,0,0,0,0,0 };
    result = robotClient.VariableAxisposModifyCmd(project, program, variable, value);
    //设置ap0为J1=10°,J2=-10°,J3=10°,J4=-10°,J5=10°,J6=-10°,附加轴均为0°
    if (result != 0) //检查是否成功
    {
        std::cout << "修改ap0值失败，结果为:" << result << std::endl;
    }
```

2.5.11 在线修改AXISPOS值

函数	int ProcessVariableAxisposModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	AXISPOS，变量值

返 回 值 (int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开

AXISPOS 说明：

JointsPos	m_Joint;	关节位置
	JointsPos[6]	Double 数组 0~5 表示 1~6 轴关节位置
AuxPos	m_AuxJoint;	附加轴位置
	AuxPos[6];	Double 数组 0~5 表示 7~12 轴关节位置

示例：

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "ap0";          //示教器中已有该变量
AXISPOS value = { 10,-10,10,-10,10,-10,0,0,0,0,0,0 };
result = robotClient.ProcessVariableAxisposModifyCmd(project, program, variable,
value);
//设置ap0为J1=10°,J2=-10°,J3=10°,J4=-10°,J5=10°,J6=-10°,附加轴均为0°
if (result != 0) //检查是否成功
{
    std::cout << "在线修改ap0值失败，结果为:" << result << std::endl;
}
```

2.5.12 修改CARTPOS值

函数	int VariableCartposModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	CARTPOS，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

CARTPOS 说明：

RobotCartPos	m_CartPos;	笛卡尔位置
	cart[6]	Double 数组 0~5 表示 1~6 轴笛卡尔位置
	m_Mode	int，位置模式
AuxPos	m_AuxJoint;	附加轴位置
	AuxPos[6];	Double 数组 0~5 表示 7~12 轴关节位置

示例：

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "cp0";          //示教器中已有该变量
CARTPOS value = { 300,0,500,-180,0,0,0,0,0,0,0,0 };
result = robotClient.VariableCartposModifyCmd(project, program, variable, value);
//设置cp0为X = 300,Y = 0,Z = 500,A = -180,B = 0,C = 0,附加轴均为0°,mode=0
if (result != 0) //检查是否成功
```

```
{
    std::cout << "修改cp0值失败，结果为:" << result << std::endl;
}
```

2.5.13 在线修改CARTPOS值

函数	int ProcessVariableCartposModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	CARTPOS，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

CARTPOS 说明：

RobotCartPos	m_CartPos;	笛卡尔位置
	cart[6]	Double 数组 0~5 表示 1~6 轴笛卡尔位置
	m_Mode	int，位置模式
AuxPos	m_AuxJoint;	附加轴位置
	AuxPos[6];	Double 数组 0~5 表示 7~12 轴关节位置

示例：

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "cp0";          //示教器中已有该变量
CARTPOS value = { 300,0,500,-180,0,0,0,0,0,0,0,0,0 };
result = robotClient.ProcessVariableCartposModifyCmd(project, program, variable,
value);
//设置cp0为X = 300,Y = 0,Z = 500,A = -180,B = 0,C = 0,附加轴均为0°,mode=0
if (result != 0) //检查是否成功
{
    std::cout << "在线修改cp0值失败，结果为:" << result << std::endl;
}
```

2.5.14 修改ROBOTAXISPOS值

函数	Int VariableRobotAxisposModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	JointsPos，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

JointsPos 说明：

double[]	JointsPos[6]	0~5 表示 1~6 轴关节位置

示例 1：

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "rap0";          //示教器中已有该变量
JointsPos value = { 10, -10, 10, -10, 10, -10 };
result = robotClient.VariableRobotAxisposModifyCmd(project, program, variable,
value);
//设置rap0为J1=10°,J2=-10°,J3=10°,J4=-10°,J5=10°,J6=-10°
if (result != 0) //检查是否成功
{
    std::cout << "修改rap0值失败，结果为:" << result << std::endl;
}
```

2.5.15 在线修改ROBOTAXISPOS值

函数	int ProcessVariableRobotAxisposModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	JointsPos，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

JointsPos 说明：

double[]	JointsPos[6]	0~5 表示 1~6 轴关节位置

示例：

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "rap0";          //示教器中已有该变量
JointsPos value = { 10, -10, 10, -10, 10, -10 };
result = robotClient.ProcessVariableRobotAxisposModifyCmd(project, program,
variable, value);
//设置rap0为J1=10°,J2=-10°,J3=10°,J4=-10°,J5=10°,J6=-10°
if (result != 0) //检查是否成功
{
    std::cout << "在线修改rap0值失败，结果为:" << result << std::endl;
}
```

2.5.16 修改ROBOTCARTPOS值

函数	int VariableRobotCartposModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名

输入	value	RobotCartPos，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

RobotCartPos 说明：

double[]	cart[6]	0~5 表示 1~6 轴笛卡尔位置
int	m_Mode	位置模式

示例：

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "rcp0";          //示教器中已有该变量
RobotCartPos value = { 300,0,500,-180,0,0,0 };
result = robotClient.VariableRobotCartposModifyCmd(project, program, variable,
value);
//设置rcp0为X=300,Y=0,Z=500,A=-180,B=0,C=0,mode=0
if (result != 0) //检查是否成功
{
    std::cout << "修改rcp0值失败，结果为:" << result << std::endl;
}
```

2.5.17 在线修改ROBOTCARTPOS值

函数	int ProcessVariableRobotCartposModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	RobotCartPos，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

RobotCartPos 说明：

double[]	cart[6]	0~5 表示 1~6 轴笛卡尔位置
int	m_Mode	位置模式

示例：

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "rcp0";          //示教器中已有该变量
RobotCartPos value = { 300,0,500,-180,0,0,0 };
result = robotClient.ProcessVariableRobotCartposModifyCmd(project, program,
variable, value);
//设置rcp0为X=300,Y=0,Z=500,A=-180,B=0,C=0,mode=0
if (result != 0) //检查是否成功
{
    std::cout << "在线修改rcp0值失败，结果为:" << result << std::endl;
}
```

2.5.18 修改DYNAMIC值

函数	int VariableDynamicModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	SDynamicPercent，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

SDynamicPercent 说明：

m_SegmentDynamic	double m_Vel;	位置速度
	double m_Acc;	位置加速度
	double m_Dec;	位置减速度
	double m_Jerk;	位置加加速度
	double m_Tjolt;	时间，未使用，默认-1
m_OriDynamic	double m_Vel;	旋转速度
	double m_Acc;	旋转加速度
	double m_Dec;	旋转减速度
	double m_Jerk;	旋转加加速度
	double m_Tjolt;	时间，未使用，默认-1
m_JointPercent	double m_Vel;	关节速度
	double m_Acc;	关节加速度
	double m_Dec;	关节减速度
	double m_Jerk;	关节加加速度
	double m_Tjolt;	时间，未使用，默认-1

示例：

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "dyn0";          //示教器中已有该变量
SDynamicPercent value ;
value.m_SegmentDynamic.m_Vel = 2000;
value.m_SegmentDynamic.m_Acc = 6000;
value.m_SegmentDynamic.m_Dec = 6000;
value.m_SegmentDynamic.m_Jerk = 60000;

value.m_OriDynamic.m_Vel = 200;
value.m_OriDynamic.m_Acc = 750;
value.m_OriDynamic.m_Dec = 750;
value.m_OriDynamic.m_Jerk = 7500;

value.m_JointPercent.m_Vel = 100;
value.m_JointPercent.m_Acc = 200;
value.m_JointPercent.m_Dec = 200;
value.m_JointPercent.m_Jerk = 200;
```

```
result = robotClient.VariableDynamicModifyCmd(project, program, variable, value);
if (result != 0) //检查是否成功
{
    std::cout << "修改dyn0值失败，结果为:" << result << std::endl;
}
```

2.5.19 在线修改DYNAMIC值

函数	int ProcessVariableDynamicModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	SDynamicPercent，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

SDynamicPercent 说明：

m_SegmentDynamic	double m_Vel;	位置速度
	double m_Acc;	位置加速度
	double m_Dec;	位置减速度
	double m_Jerk;	位置加加速度
	double m_Tjolt;	时间，未使用，默认-1
m_OriDynamic	double m_Vel;	旋转速度
	double m_Acc;	旋转加速度
	double m_Dec;	旋转减速度
	double m_Jerk;	旋转加加速度
	double m_Tjolt;	时间，未使用，默认-1
m_JointPercent	double m_Vel;	关节速度
	double m_Acc;	关节加速度
	double m_Dec;	关节减速度
	double m_Jerk;	关节加加速度
	double m_Tjolt;	时间，未使用，默认-1

示例：

```
int result= 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "dyn0";          //示教器中已有该变量
SDynamicPercent value ;
value.m_SegmentDynamic.m_Vel = 2000;
value.m_SegmentDynamic.m_Acc = 6000;
value.m_SegmentDynamic.m_Dec = 6000;
value.m_SegmentDynamic.m_Jerk = 60000;

value.m_OriDynamic.m_Vel = 200;
```

```
    value.m_OriDynamic.m_Acc = 750;
    value.m_OriDynamic.m_Dec = 750;
    value.m_OriDynamic.m_Jerk = 7500;

    value.m_JointPercent.m_Vel = 100;
    value.m_JointPercent.m_Acc = 200;
    value.m_JointPercent.m_Dec = 200;
    value.m_JointPercent.m_Jerk = 200;
    result = robotClient.ProcessVariableDynamicModifyCmd(project, program, variable,
value);
    if (result != 0) //检查是否成功
    {
        std::cout << "在线修改dyn0值失败，结果为:" << result << std::endl;
    }
```

2.5.20 修改ARCONDATA值

函数	int VariableArcOnDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	ArcOnData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

ArcOnData 说明：

unsigned int m_preFlowtime;	预送气时间
float m_arconCurrent;	起弧电流
float m_arconVoltage;	起弧电压
unsigned int m_arconTime;	起弧时间
unsigned int m_BeforeArcOn;	预起弧时间
unsigned int m_BeforeWireOut;	预送丝时间

示例：

```
    int result = 0;
    std::string project = "project0";        //示教器中已有该工程
    std::string program = "program0";      //示教器中已有该程序
    std::string variable = "arcon0";
    ArcOnData arcondata;
    arcondata.m_preFlowtime = 100;
    arcondata.m_arconCurrent = 2;
    arcondata.m_arconVoltage = 5;
    arcondata.m_arconTime = 100;
    arcondata.m_BeforeArcOn = 10;
    arcondata.m_BeforeWireOut = 20;
    result = robotClient.VariableArcOnDataModifyCmd(project, program, variable,
    arcondata);
```

```
if (result != 0) //检查是否成功
{
    std::cout << "修改arcon0值失败，结果为:" << result << std::endl;
}
```

2.5.21 在线修改ARCONDATA值

函数	int ProcessVariableArcOnDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	ArcOnData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "arcon0";
ArcOnData arcondata;
arcondata.m_preFlowtime = 100;
arcondata.m_arconCurrent = 2;
arcondata.m_arconVoltage = 5;
arcondata.m_arconTime = 100;
arcondata.m_BeforeArcOn = 10;
arcondata.m_BeforeWireOut = 20;
result = robotClient.ProcessVariableArcOnDataModifyCmd(project, program, variable,
arcondata);
if (result != 0) //检查是否成功
{
    std::cout << "修改arcon0值失败，结果为:" << result << std::endl;
}
```

2.5.22 修改ARCOFFDATA值

函数	int VariableArcOffDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	ArcOffData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

ArcOffData 说明：

unsigned int m_burnBacktime;	回烧时间

unsigned int m_postFlowtime;	滞后送气时间
float m_arcoffCurrent;	收弧电流
float m_arcoffVoltage;	收弧电压
unsigned int m_arcoffTime;	收弧时间

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "arcoff0";          //示教器中已有该变量
ArcOffData arcoffdata = {1,2,3,4,5};
result = robotClient.VariableArcOffDataModifyCmd(project, program, variable,
arcoffdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改arcoff0值失败，结果为:" << result << std::endl;
}
```

2.5.23 在线修改ARCOFFDATA值

函数	int ProcessVariableArcOnDataModifyCmd (project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	ArcOffData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "arcoff0";          //示教器中已有该变量
ArcOffData arcoffdata = {1,2,3,4,5};
result = robotClient.ProcessVariableArcOffDataModifyCmd(project, program, variable,
arcoffdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改arcoff0值失败，结果为:" << result << std::endl;
}
```

2.5.24 修改ARCDATA值

函数	int VariableArcDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序

输入	variable	std::string，变量名
输入	value	ArcData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

ArcData 说明：

float m_weldVoltage;	焊接电压
float m_weldCurrent;	焊接电流
float m_weldSpeed;	焊接速度
double m_endSpeed;	焊接结束速度

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "arc0";
ArcData arcdata ={ 1,2,3,4};
result = robotClient.VariableArcDataModifyCmd(project, program, variable, arcdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改arc0值失败，结果为:"<< result << std::endl;
}
```

2.5.25 在线修改ARCDATA值

函数	int ProcessVariableArcDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	ArcData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "arc0";
ArcData arcdata ={ 1,2,3,4};
result = robotClient.ProcessVariableArcDataModifyCmd(project, program, variable,
arcdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改arc0值失败，结果为:"<< result << std::endl;
}
```

2.5.26 修改ARCRETRYDATA值

函数	int VariableArcRetryDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	ArcRetryData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

ArcRetryData 说明：

int m_DetectTime;	起弧失败检测时间
int m_RetryTimes;	起弧失败后，重试的次数
int m_RetractTime;	焊丝回抽时间
int m_RetractWaitTime;	回抽等待时间
double m_CurrentInc;	再起弧电流增量
double m_VoltageInc;	再起弧电压增量
char m_ScratchArcFlag[32]	擦线起弧标志
double m_ScratchMaxDistance;	擦线距离
double m_ScratchStepLength;	擦线步长，每次移动距离
double m_ReturnSpeed;	擦线返回速度

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "retry0";
ArcRetryData arcretrydata = { 1000, 0,1000,50,0, 0,"TRUE",7,50,9};
result = robotClient.VariableArcRetryDataModifyCmd(project, program, variable,
arcretrydata);
if (result != 0) //检查是否成功
{
    std::cout << "修改retry0值失败，结果为:" << result << std::endl;
}
```

2.5.27 在线修改ARCRETRYDATA值

函数	int ProcessVariableArcRetryDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	ArcRetryData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
```

```
std::string program = "program0";      //示教器中已有该程序
std::string variable = "retry0";
ArcRetryData arcretrydata = { 1000, 0,1000,50,0, 0,"TRUE",7,50,9};
result = robotClient.ProcessVariableArcRetryDataModifyCmd(project, program,
variable, arcretrydata);
if (result != 0) //检查是否成功
{
    std::cout << "修改retry0值失败，结果为:" << result << std::endl;
}
```

2.5.28 修改ARCSEGDATA值

函数	int VariableArcSegDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	ArcSegData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

ArcSegData 说明：

double m_StartArcOnCurrent;	开始起弧电流
double m_StartArcOnVoltage;	开始起弧电压
int m_DecaySeg;	渐变段数
int m_PreflowTime;	预送气时间
double m_ArcOnCurrent;	起弧电流
double m_ArcOnVoltage;	起弧电压
int m_ArcOnWaitTime;	等待焊机起弧成功的时间
int m_ArcOnDecayTime;	起弧渐变时间
int m_FailDetectTime;	起弧失败检测时间
int m_RetryTimes;	起弧失败后，重试的次数
int m_RetractTime;	焊丝回抽时间
int m_RetractWaitTime;	回抽等待时间
double m_CurrentInc;	再起弧电流增量
double m_VoltageInc;	再起弧电压增量
int m_BurnbackTime;	回烧时间
int m_PostflowTime;	滞后送气时间
double m_ArcOffCurrent;	收弧电流
double m_ArcOffVoltage;	收弧电压
int m_ArcOffWaitTime;	等待焊机息弧成功的时间
int m_ArcOffDecayTime;	渐变时间
int m_StickCheckDelayTime;	粘丝检测延时时间

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
```

```
std::string program = "program0";      //示教器中已有该程序
std::string variable = "arcsegdata0";
ArcSegData arcsegdata = {1, 2,3,4,5, 6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21};
result = robotClient.VariableArcSegDataModifyCmd(project, program, variable,
arcsegdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改arcsegdata0值失败，结果为:" << result << std::endl;
}
```

2.5.29 在线修改ARCSEGDATA值

函数	int ProcessVariableArcSegDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	ArcSegData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "arcsegdata0";
ArcSegData arcsegdata = {1, 2,3,4,5, 6,7,8,9,10,11,12,13,14,15,16,17,18,19,20,21};
result = robotClient.ProcessVariableArcSegDataModifyCmd(project, program, variable,
arcsegdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改arcsegdata0值失败，结果为:" << result << std::endl;
}
```

2.5.30 修改ARCCONFIGDATA值

函数	int VariableArcConfigDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	ArcConfigData，double[14]，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
```

```
std::string variable = "conf0";
double arcconfigdata[14] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 };
result = robotClient.VariableArcConfigDataModifyCmd(project, program, variable,
arcconfigdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改conf0值失败，结果为:" << result << std::endl;
}
```

2.5.31 在线修改ARCCONFIGDATA值

函数	int ProcessVariableArcConfigDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	ArcConfigData，double[14]，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "conf0";
double arcconfigdata[14] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14 };
result = robotClient.ProcessVariableArcConfigDataModifyCmd(project, program,
variable, arcconfigdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改conf0值失败，结果为:" << result << std::endl;
}
```

2.5.32 修改WEAVEDATA值

函数	int VariableWeaveDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	WeaveData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

WeaveData 说明：

WeaveShape m_waveShape;	摆弧形状(枚举型)
WeaveType m_waveType;	摆弧类型(枚举型)
float m_weaveLength;	摆弧长度
float m_weaveWidth;	摆弧幅值

double m_SwingDirection;	摆弧倾斜角（对 sin 摆无效）
double m_WeaveAngle;	摆弧平面倾斜角
double m_SpaceAngle;	空间摆弧夹角（对 L 型摆和空间三角摆有效）
unsigned int m_PauseTime_1;	1/4 处停留时间
unsigned int m_PauseTime_2;	2/4 处停留时间
unsigned int m_PauseTime_3;	3/4 处停留时间
unsigned int m_PauseTime_4;	4/4 处停留时间
int m_PauseContinue;	停留连续
double m_EndLength;	结束长度
double m_EndWidth;	结束宽度
double m_CenterHigh;	中心高度

WeaveShape 说明：

eNoWeave= 0,	无摆弧
eSin = 5,	正弦摆
eSpiral,	斜椭圆摆
eObliqueTriangle,	斜三角摆
eSpaceTriangle,	空间三角摆
eLTriangle,	空间 L 型摆
eBackForward,	前后摆
eConstPoint	定点圆摆

WeaveType 说明：

eTCPWeave = 0,	TCP 摆
eWristWeave,	腕关节摆
e45JointWeave,	关节摆

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "wd0";
WeaveData weavedata = { eSin, e45JointWeave, 1, 2, 3, 4, 5, 6, 7, 8, 9, 1, 1, 2,
3 };
result = robotClient.VariableWeaveDataModifyCmd(project, program, variable,
weavedata);
if (result != 0) //检查是否成功
{
    std::cout << "修改wd0值失败，结果为:" << result << std::endl;
}
```

2.5.33 在线修改WEAVEDATA值

函数	int ProcessVariableWeaveDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序

输入	variable	std::string，变量名
输入	value	WeaveData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";          //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "wd0";
WeaveData weavedata = { eSin, e45JointWeave, 1, 2, 3, 4, 5, 6, 7, 8, 9, 1, 1, 2,
3 };
result = robotClient.ProcessVariableWeaveDataModifyCmd(project, program, variable,
weavedata);
if (result != 0) //检查是否成功
{
    std::cout << "修改wd0值失败，结果为:" << result << std::endl;
}
```

2.5.34 修改CD值

函数	int VariableCDDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	CDData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

CDData 说明：

int m_Calibrate;	是否标定，0 为 FALSE，1 为 TRUE
CartPos m_CartPos ;	机器人基准位置
AuxPos m_AuxPos ;	附加轴基准位置
CartPos m_CurrentPos ;	当前位置
CDDirection m_Direction;	寻位方向
char m_RefName[32]	参考坐标系

CartPos 说明：

double m_X;	X 值
double m_Y;	Y 值
double m_Z;	Z 值
double m_A;	A 值
double m_B;	B 值
double m_C;	C 值
unsigned int m_Mode;	Mode 值

AuxPos 说明：

double[6]	附加轴值

CDDirection 说明：

eCD_X_Positive = 1,	X+方向
eCD_X_Negative,	X-方向
eCD_Y_Positive,	Y+方向
eCD_Y_Negative,	Y-方向
eCD_Z_Positive,	Z+方向
eCD_Z_Negative	Z-方向

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "cd0";
CDData cd = {0,1,2,3,4,5,6,0,7,8,9,10,11,12 ,21,22,23,24,25, 26,0,eCD_Y_Positive,
"WORLD"};
result = robotClient.VariableCDModifyCmd(project, program, variable, cd);
if (result != 0) //检查是否成功
{
    std::cout << "修改cd0值失败，结果为:" << result << std::endl;
}
```

2.5.35 在线修改CD值

函数	int ProcessVariableCDModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	CDData，变量值
返回值 (int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "cd0";
CDData cd = {0,1,2,3,4,5,6,0,7,8,9,10,11,12 ,21,22,23,24,25, 26,0,eCD_Y_Positive,
"WORLD"};
result = robotClient.ProcessVariableCDModifyCmd(project, program, variable, cd);
if (result != 0) //检查是否成功
{
    std::cout << "修改cd0值失败，结果为:" << result << std::endl;
}
```

2.5.36 修改CDS值

函数	int VariableCDSModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	ArcCDSData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

ArcCDSData 说明：

CORRECTTYPE m_Dimension;	寻位方式，枚举型
int m_temp1;	预留
int m_temp2;	预留
double m_temp3;	预留
double m_temp4;	预留
double m_temp5;	预留
CDData m_CD[7];	CDData 数组，7 个 CDData

CORRECTTYPE 说明：

e1D_CR = 1,	1D_CR
e2D_CR = 2,	2D_CR
e3D_CR = 3,	3D_CR
e6D_CR = 6,	6D_CR
e2D1R_CR = 7,	2D1R_CR
e3D1R_CR = 8,	3D1R_CR
eCircle_CR = 9,	Circle_ CR
e3D3R_CR = 10	3D3R_CR

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "cds0";
ArcCDSData cds0;
cds0.m_Dimension = e1D_CR;
cds0.m_temp1 = 0;
cds0.m_temp2 = 0;
cds0.m_temp3 = 0;
cds0.m_temp4 = 0;
cds0.m_temp5 = 0;
for (int i = 0; i < 7; i++)
{
    cds0.m_CD[i] = { 0,1,2,3,4,5,6,0,7,8,9,10,11,12 ,21,22,23,24,25,
26,0,eCD_Y_Positive, "WORLD" };
}
result = robotClient.VariableCDSModifyCmd(project, program, variable, cds0);
```

```
if (result != 0) //检查是否成功
{
    std::cout << "修改cds0值失败，结果为:" << result << std::endl;
}
```

2.5.37 修改PAT值

函数	int VariablePATModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	PAT，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

PAT 说明：

TouchMode m_TouchMode;	寻位模式（枚举型）
double m_Tolerance;	容忍偏差
double m_SearchDistance;	寻位距离
double m_SearchSpeed;	寻位速度
unsigned int m_ReturnAcc;	返回加速度
unsigned int m_ReturnSpeed;	返回速度
double m_ReturnDistance;	回退距离

TouchMode 说明：

eSingleTouch,	单接触寻位

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "pat0";
PAT pat ;
pat.m_TouchMode = eSingleTouch;
pat.m_Tolerance = 20;
pat.m_SearchDistance = 3;
pat.m_SearchSpeed = 1;
pat.m_ReturnAcc = 10;
pat.m_ReturnSpeed = 20;
result = robotClient.VariablePATModifyCmd(project, program, variable, pat);
if (result != 0) //检查是否成功
{
    std::cout << "修改pat0值失败，结果为:" << result << std::endl;
}
```

2.5.38 在线修改PAT值

函数	int ProcessVariablePATModifyCmd(project,program,variable,value);

输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	PAT，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "pat0";
PAT pat ;
pat.m_TouchMode = eSingleTouch;
pat.m_Tolerance = 20;
pat.m_SearchDistance = 3;
pat.m_SearchSpeed = 1;
pat.m_ReturnAcc = 10;
pat.m_ReturnSpeed = 20;
result = robotClient.ProcessVariablePATModifyCmd(project, program, variable, pat);
if (result != 0) //检查是否成功
{
    std::cout << "修改pat0值失败，结果为:" << result << std::endl;
}
```

2.5.39 修改TRACKDATA值

函数	int VariableTrackDataModifyCmd(project,program,variab1le,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	TrackData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

TrackData 说明：

int m_LateralBeginCycle;	横向纠偏开始周期[3,1000]
double Kpl;	横向比例增益,不是比例增益百分比[0.100]
double m_LeftCoefficient;	横向左边偏移系数(0,100]
double m_RightCoefficient;	横向右边偏移系数(0,100]
int m_VerticalModeFlag;	纵向模式标志位，为 0 则是常数模式,为 1 是反馈模式;
double m_Iref_Vertical;	纵向纠偏基准电流（常数模式）
int m_VerticalBeginCycle;	纵向取样开始周期[2,50]
int m_VerticalSustainCycle;	纵向取样持续周期[1,50]
double m_VerticalLength;	纵向周期长度[2,10]

double Kpv;	纵向比例增益，不是比例增益百分比[0,10]
int m_TypeofTimeOrDist;	未使用，赋值 0
unsigned int m_TimeInterval;	未使用，赋值 200
unsigned int m_DistInterval;	未使用，赋值 2
double m_YBias;	未使用，赋值 1
double m_ZBias;	未使用，赋值 1
double L_MinComp;	横向最小补偿[0,1000]
double L_MaxComp;	横向最大补偿[0,1000]
double L_TotalMaxComp;	横向最大补偿总[0,1000]
double L_RightComp;	横向不对称调整系数[-99.9,99.9]
doubletemp Temp_L;	double[6]数组，备用
double V_MinComp;	纵向最小补偿
double V_MaxComp;	纵向最大补偿
double V_TotalMaxComp;	纵向最大补偿总
double V_TopComp;	纵向不对称调整系数
doubletemp Temp_V;	double[6]数组，备用

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "trackdata0";
TrackData trackdata;
trackdata.m_LateralBeginCycle = 100;
trackdata.Kpl = 0;
trackdata.m_LeftCoefficient = 100;
trackdata.m_RightCoefficient = 100;
trackdata.m_VerticalModeFlag = 1;
trackdata.m_Iref_Vertical = 0;
trackdata.m_VerticalBeginCycle = 7;
trackdata.m_VerticalSustainCycle = 1;
trackdata.m_VerticalLength = 3;
trackdata.Kpv = 0;
trackdata.m_TypeofTimeOrDist = 0;
trackdata.m_TimeInterval = 200;
trackdata.m_DistInterval = 2;
trackdata.m_YBias = 1;
trackdata.m_ZBias = 1;
trackdata.L_MaxComp = 1;
trackdata.L_MinComp = 1;
trackdata.L_TotalMaxComp = 1;
trackdata.L_RightComp = 1;
for (int i = 0; i < 6;i++)
{
```

```
        trackdata.Temp_L[i] = i + 1;
    }
    trackdata.V_MaxComp = 1;
    trackdata.V_MinComp = 1;
    trackdata.V_TopComp = 1;
    trackdata.V_TotalMaxComp = 1;
    for (int j = 0; j < 6; j++)
    {
        trackdata.Temp_V[j] = j + 1;
    }
    result = robotClient.VariableTrackDataModifyCmd(project, program, variable,
    trackdata);
    if (result != 0) //检查是否成功
    {
        std::cout << "修改trackdata0值失败，结果为:" << result << std::endl;
    }
```

2.5.40 在线修改TRACKDATA值

函数	int ProcessVariableTrackDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	TrackData，变量值
返回值 (int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
    int result = 0;
    std::string project = "project0";        //示教器中已有该工程
    std::string program = "program0";      //示教器中已有该程序
    std::string variable = "trackdata0";
    TrackData trackdata;
    trackdata.m_LateralBeginCycle = 100;
    trackdata.Kpl = 0;
    trackdata.m_LeftCoefficient = 100;
    trackdata.m_RightCoefficient = 100;
    trackdata.m_VerticalModeFlag = 1;
    trackdata.m_Iref_Vertical = 0;
    trackdata.m_VerticalBeginCycle = 7;
    trackdata.m_VerticalSustainCycle = 1;
    trackdata.m_VerticalLength = 3;
    trackdata.Kpv = 0;
    trackdata.m_TypeofTimeOrDist = 0;
    trackdata.m_TimeInterval = 200;
```

```
    trackdata.m_DistInterval = 2;
    trackdata.m_YBias = 1;
    trackdata.m_ZBias = 1;
    trackdata.L_MaxComp = 1;
    trackdata.L_MinComp = 1;
    trackdata.L_TotalMaxComp = 1;
    trackdata.L_RightComp = 1;
    for (int i = 0; i < 6;i++)
    {
        trackdata.Temp_L[i] = i + 1;
    }
    trackdata.V_MaxComp = 1;
    trackdata.V_MinComp = 1;
    trackdata.V_TopComp = 1;
    trackdata.V_TotalMaxComp = 1;
    for (int j = 0; j < 6; j++)
    {
        trackdata.Temp_V[j] = j + 1;
    }
    result = robotClient.ProcessVariableTrackDataModifyCmd(project, program, variable,
trackdata);
    if (result != 0) //检查是否成功
    {
        std::cout << "修改trackdata0值失败，结果为:" << result << std::endl;
    }
```

2.5.41 修改LASERTRACKDATA值

函数	int VariableLaserTrackDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	LaserTrackData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

LaserTrackData 说明：

double m_EndSpeed;	最大速度
double m_EndCurrent;	最大电流
double m_YBias;	Y 方向偏置距离
double m_ZBias;	Z 方向偏置距离
double m_MaxWidth;	最大坡口宽度
char m_AdapLength[32];	宽度自适应开关
double m_StandardWidth;	标准坡口宽度
double m_WidthRatio;	宽度比例

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "lasertrackdata0";
LaserTrackData lasertrackdata;
lasertrackdata.m_EndSpeed = 1;
lasertrackdata.m_EndCurrent = 2;
lasertrackdata.m_YBias = 3;
lasertrackdata.m_ZBias = 4;
lasertrackdata.m_MaxWidth = 5;
memcpy(lasertrackdata.m_AdapLength, "FALSE", (strlen("FALSE") + 1));
lasertrackdata.m_StandardWidth = 6;
lasertrackdata.m_WidthRatio = 7;
result = robotClient.VariableLaserTrackDataModifyCmd(project, program, variable,
lasertrackdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改trackdata0值失败，结果为:" << result << std::endl;
}
```

2.5.42 在线修改LASERTRACKDATA值

函数	int ProcessVariableLaserTrackDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	LaserTrackData，变量值
返回值 (int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "lasertrackdata0";
LaserTrackData lasertrackdata;
lasertrackdata.m_EndSpeed = 1;
lasertrackdata.m_EndCurrent = 2;
lasertrackdata.m_YBias = 3;
lasertrackdata.m_ZBias = 4;
lasertrackdata.m_MaxWidth = 5;
memcpy(lasertrackdata.m_AdapLength, "FALSE", (strlen("FALSE") + 1));
lasertrackdata.m_StandardWidth = 6;
lasertrackdata.m_WidthRatio = 7;
result = robotClient.ProcessVariableLaserTrackDataModifyCmd(project, program,
```

```
variable, lasertrackdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改trackdata0值失败，结果为:" << result << std::endl;
}
```

2.5.43 修改PLASERFILTER值

函数	int VariablePLaserFilterModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	PLaserFilter，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

PLaserFilter 说明：

FilterType m_FilterType;	滤波器类型（枚举型）
PointType m_PointType;	目标点类型（枚举型）
int m_FilterParam;	滤波器参数
int m_FilterLength;	滤波长度
double m_Featurethreshold;	特征点阈值

FilterType 说明：

eNoFilter,	无滤波
eMedianFilter,	中值滤波
eMeanFilter,	均值滤波
eGaussFilter	高斯滤波

PointType 说明：

eMax,	第一处极大值
eMin,	第一处极小值
eUpLeft,	第一处上升沿左侧
eUpRight,	第一处上升沿右侧
eDownLeft,	第一处下降沿左侧
eDownRight,	第一处下降沿右侧

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "plaserfilter0";
PLaserFilter plaserfilter;
plaserfilter.m_FilterType = eNoFilter;
plaserfilter.m_PointType = eMax;
plaserfilter.m_FilterLength = 1;
plaserfilter.m_FilterParam = 1;
plaserfilter.m_Featurethreshold = 0;
```

```
result = robotClient.VariablePLaserFilterModifyCmd(project, program, variable,
plaserfilter);
if (result != 0) //检查是否成功
{
    std::cout << "修改plaserfilter0值失败，结果为:" << result << std::endl;
}
```

2.5.44 在线修改PLASERFILTER值

函数	int ProcessVariablePLaserFilterModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	PLaserFilter，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "plaserfilter0";
PLaserFilter plaserfilter;
plaserfilter.m_FilterType = eNoFilter;
plaserfilter.m_PointType = eMax;
plaserfilter.m_FilterLength = 1;
plaserfilter.m_FilterParam = 1;
plaserfilter.m_Featurethreshold = 0;
result = robotClient.ProcessVariablePLaserFilterModifyCmd(project, program,
variable, plaserfilter);
if (result != 0) //检查是否成功
{
    std::cout << "修改plaserfilter0值失败，结果为:" << result << std::endl;
}
```

2.5.45 修改LASERARCONDATA值

函数	int VariableLaserArcOnDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	LaserArcOnNewData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

LaserArcOnNewData 说明：

LaserArcOnData m_LaserOnData;	滤波器类型（枚举型）

LaserArcData m_LaserData;	目标点类型（枚举型）

LaserArcOnData 说明：

int m_PreFlowTime;	预送气时间
int m_PreWireTime;	预送丝时间
int m_DelayLightTime;	延迟开光时间
double m_WireSpeed;	送丝速度
double m_Temp4;	预留
double m_Temp3;	预留
double m_Temp2;	预留
double m_Temp1;	预留

LaserArcData 说明：

double m_LaserPower;	激光功率
int m_PWM;	占空比
int m_Frequency;	频率
double m_WireSpeed;	送丝速度
double m_Temp4;	预留
double m_Temp3;	预留
double m_Temp2;	预留
double m_Temp1;	预留

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "laserarcondata0";
LaserArcOnNewData laserarcondata;
laserarcondata.m_LaserOnData.m_PreFlowTime = 1;
laserarcondata.m_LaserOnData.m_PreWireTime = 2;
laserarcondata.m_LaserOnData.m_DelayLightTime = 3;
laserarcondata.m_LaserOnData.m_WireSpeed = 4;
laserarcondata.m_LaserOnData.m_Temp4 = 0;
laserarcondata.m_LaserOnData.m_Temp3 = 0;
laserarcondata.m_LaserOnData.m_Temp2 = 0;
laserarcondata.m_LaserOnData.m_Temp1 = 0;
laserarcondata.m_LaserData.m_LaserPower = 5;
laserarcondata.m_LaserData.m_PWM = 6;
laserarcondata.m_LaserData.m_Frequency = 7;
laserarcondata.m_LaserData.m_WireSpeed = 8;
laserarcondata.m_LaserData.m_Temp4 = 9;
laserarcondata.m_LaserData.m_Temp3 = 10;
laserarcondata.m_LaserData.m_Temp2 = 11;
laserarcondata.m_LaserData.m_Temp1 = 12;
result = robotClient.VariableLaserArcOnDataModifyCmd(project, program, variable,
laserarcondata);
```

```
if (result != 0) //检查是否成功
{
    std::cout << "修改laserarcondata0值失败，结果为:" << result << std::endl;
}
```

2.5.46 在线修改LASERARCONDATA值

函数	int ProcessVariableLaserArcOnDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	LaserArcOnNewData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "laserarcondata0";
LaserArcOnNewData laserarcondata;
laserarcondata.m_LaserOnData.m_PreFlowTime = 1;
laserarcondata.m_LaserOnData.m_PreWireTime = 2;
laserarcondata.m_LaserOnData.m_DelayLightTime = 3;
laserarcondata.m_LaserOnData.m_WireSpeed = 4;
laserarcondata.m_LaserOnData.m_Temp4 = 0;
laserarcondata.m_LaserOnData.m_Temp3 = 0;
laserarcondata.m_LaserOnData.m_Temp2 = 0;
laserarcondata.m_LaserOnData.m_Temp1 = 0;
laserarcondata.m_LaserData.m_LaserPower = 5;
laserarcondata.m_LaserData.m_PWM = 6;
laserarcondata.m_LaserData.m_Frequency = 7;
laserarcondata.m_LaserData.m_WireSpeed = 8;
laserarcondata.m_LaserData.m_Temp4 = 9;
laserarcondata.m_LaserData.m_Temp3 = 10;
laserarcondata.m_LaserData.m_Temp2 = 11;
laserarcondata.m_LaserData.m_Temp1 = 12;
result = robotClient.ProcessVariableLaserArcOnDataModifyCmd(project, program,
variable, laserarcondata);
if (result != 0) //检查是否成功
{
    std::cout << "修改laserarcondata0值失败，结果为:" << result << std::endl;
}
```

2.5.47 修改LASERARCDATA值

函数	int VariableLaserArcDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	LaserArcData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

LaserArcData 说明：

double m_LaserPower;	激光功率
int m_PWM;	占空比
int m_Frequency;	频率
double m_WireSpeed;	送丝速度
double m_Temp4;	预留
double m_Temp3;	预留
double m_Temp2;	预留
double m_Temp1;	预留

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "laserarcdata0";
LaserArcData laserarcdata;
laserarcdata.m_LaserPower = 1;
laserarcdata.m_PWM = 2;
laserarcdata.m_Frequency = 3;
laserarcdata.m_WireSpeed = 4;
laserarcdata.m_Temp4 = 0;
laserarcdata.m_Temp3 = 0;
laserarcdata.m_Temp2 = 0;
laserarcdata.m_Temp1 = 0;
result = robotClient.VariableLaserArcDataModifyCmd(project, program, variable,
laserarcdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改laserarcdata0值失败，结果为:" << result << std::endl;
}
```

2.5.48 在线修改LASERARCDATA值

函数	int ProcessVariableLaserArcDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	LaserArcData，变量值

返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "laserarcdata0";
LaserArcData laserarcdata;
laserarcdata.m_LaserPower = 1;
laserarcdata.m_PWM = 2;
laserarcdata.m_Frequency = 3;
laserarcdata.m_WireSpeed = 4;
laserarcdata.m_Temp4 = 0;
laserarcdata.m_Temp3 = 0;
laserarcdata.m_Temp2 = 0;
laserarcdata.m_Temp1 = 0;
result = robotClient.ProcessVariableLaserArcDataModifyCmd(project, program,
variable, laserarcdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改laserarcdata0值失败，结果为:" << result << std::endl;
}
```

2.5.49 修改LASERARCOFFDATA值

函数	int VariableLaserArcOffDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	LaserArcOffData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

LaserArcOffData 说明：

int m_DelayFlowTime;	滞后送气时间
int m_PreCloseWireTime;	提前关丝时间
int m_WireRetractTime;	退丝时间
int m_WireReFeedTime;	退丝再进时间
double m_WireSpeed;	送丝速度
double m_Temp5;	预留
double m_Temp4;	预留
double m_Temp3;	预留
double m_Temp2;	预留
double m_Temp1;	预留

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "laserarcoffdata0";
LaserArcOffData laserarcoffdata;
laserarcoffdata.m_DelayFlowTime = 1;
laserarcoffdata.m_PreCloseWireTime = 2;
laserarcoffdata.m_WireRetractTime = 3;
laserarcoffdata.m_WireReFeedTime = 4;
laserarcoffdata.m_WireSpeed = 10;
laserarcoffdata.m_Temp5 = 0;
laserarcoffdata.m_Temp4 = 0;
laserarcoffdata.m_Temp3 = 0;
laserarcoffdata.m_Temp2 = 0;
laserarcoffdata.m_Temp1 = 0;
result = robotClient.VariableLaserArcOffDataModifyCmd(project, program, variable,
laserarcoffdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改laserarcoffdata0值失败，结果为:" << result << std::endl;
}
```

2.5.50 在线修改LASERARCOFFDATA值

函数	int ProcessVariableLaserArcOffDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	LaserArcOffData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "laserarcoffdata0";
LaserArcOffData laserarcoffdata;
laserarcoffdata.m_DelayFlowTime = 1;
laserarcoffdata.m_PreCloseWireTime = 2;
laserarcoffdata.m_WireRetractTime = 3;
laserarcoffdata.m_WireReFeedTime = 4;
laserarcoffdata.m_WireSpeed = 10;
laserarcoffdata.m_Temp5 = 0;
laserarcoffdata.m_Temp4 = 0;
```

```
laserarcoffdata.m_Temp3 = 0;
laserarcoffdata.m_Temp2 = 0;
laserarcoffdata.m_Temp1 = 0;
result = robotClient.ProcessVariableLaserArcOffDataModifyCmd(project, program,
variable, laserarcoffdata);
if (result != 0) //检查是否成功
{
    std::cout << "修改laserarcoffdata0值失败，结果为:" << result << std::endl;
}
```

2.5.51 修改AXISDIST值

函数	int VariableAxisDistModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	AxisPos，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

AXISPOS 说明

JointsPos	double m_JointsPos[6];	J1/J2/J3/J4/J5/J6
AuxPos	double m_AuxPos[6];	附加轴位置

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "apd0";
AxisPos apd;
for (int i = 0; i < 6;i++)
{
    apd.m_JointsPos[i] = i + 1;
}
for (int j = 0; j < 6; j++)
{
    apd.m_AuxPos[j] = j + 7;
}
result = robotClient.VariableAxisDistModifyCmd(project, program, variable, apd);
if (result != 0) //检查是否成功
{
    std::cout << "修改apd0值失败，结果为:" << result << std::endl;
}
```

2.5.52 在线修改AXISDIST值

函数	int ProcessVariableAxisDistModifyCmd(project,program,variable,value);

输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	AxisDist，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "apd0";
AxisPos apd;
for (int i = 0; i < 6;i++)
{
    apd.m_JointsPos[i] = i + 1;
}
for (int j = 0; j < 6; j++)
{
    apd.m_AuxPos[j] = j + 7;
}

result = robotClient.ProcessVariableAxisDistModifyCmd(project, program, variable,
apd);
if (result != 0) //检查是否成功
{
    std::cout << "修改apd0值失败，结果为:" << result << std::endl;
}
```

2.5.53 修改CARTDIST值

函数	int VariableCartDistModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	CartDist，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

CartDist 说明

double	m_X	X
double	m_Y	Y
double	m_Z	Z
double	m_A	A
double	m_B	B
double	m_C	C

AuxPos	double m_AuxPos[6];	附加轴位置
int	m_Mode	模式

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "cpd0";
CartDist cpd0;
cpd0.m_X = 1;
cpd0.m_Y = 2;
cpd0.m_Z = 3;
cpd0.m_A = 4;
cpd0.m_B = 5;
cpd0.m_C = 6;
cpd0.m_Mode = 0;
for (int i = 0; i < 6; i++)
{
    cpd0.m_AuxPos[i] = i + 7;
}
result = robotClient.VariableCartDistModifyCmd(project, program, variable, cpd0);
if (result != 0) //检查是否成功
{
    std::cout << "修改cpd0值失败，结果为:" << result << std::endl;
}
```

2.5.54 在线修改CARTDIST值

函数	int ProcessVariableCartDistModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	CartDist，变量值
返回值 (int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "cpd0";
CartDist cpd0;
cpd0.m_X = 1;
cpd0.m_Y = 2;
cpd0.m_Z = 3;
cpd0.m_A = 4;
```

```
cpd0.m_B = 5;
cpd0.m_C = 6;
cpd0.m_Mode = 0;
for (int i = 0; i < 6; i++)
{
    cpd0.m_AuxPos[i] = i + 7;
}
result = robotClient.ProcessVariableCartDistModifyCmd(project, program, variable,
cpd0);
if (result != 0) //检查是否成功
{
    std::cout << "修改cpd0值失败，结果为:" << result << std::endl;
}
```

2.5.55 修改AUXAXISPOS值

函数	int VariableAuxAxisPosModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	AuxAxisPos，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

AuxAxisPos 说明

AuxPos	double[6];	附加轴位置

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "aap0";
AuxAxisPos aap0;
for (int i = 0; i < 6; i++)
{
    aap0[i] = i + 1;
}
result = robotClient.VariableAuxAxisPosModifyCmd(project, program, variable, aap0);
if (result != 0) //检查是否成功
{
    std::cout << "修改aap0值失败，结果为:" << result << std::endl;
}
```

2.5.56 在线修改AUXAXISPOS值

函数	int ProcessVariableAuxAxisPosModifyCmd(project,program,variable,value);	
输入	project	std::string，工程

输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	AuxAxisPos，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "aap0";
AuxAxisPos aap0;
for (int i = 0; i < 6; i++)
{
    aap0[i] = i + 1;
}
result = robotClient.ProcessVariableAuxAxisPosModifyCmd(project, program, variable,
aap0);
if (result != 0) //检查是否成功
{
    std::cout << "修改aap0值失败，结果为:" << result << std::endl;
}
```

2.5.57 修改PCARTPOS值

函数	int VariablePCartPosModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	PalletPosition，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

PalletPosition 说明

char m_PalName[48];	码垛变量名称
unsigned int m_LayerNum;	层号
unsigned int m_PartNum;	件号
CartPos m_CartPos;	笛卡尔坐标
AuxPos m_AuxPos;	附加轴坐标

CartPos 说明：

double m_X;	X 值
double m_Y;	Y 值
double m_Z;	Z 值
double m_A;	A 值
double m_B;	B 值
double m_C;	C 值

unsigned int m_Mode;	Mode 值

AuxPos 说明

AuxPos	m_AuxPos[6];	附加轴位置

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "pcartpos0";
PalletPosition pcartpos0;
memcpy(pcartpos0.m_PalName, "null", (strlen("null") + 1));
pcartpos0.m_LayerNum = 1;
pcartpos0.m_PartNum = 2;
pcartpos0.m_CartPos.m_X = 926.331260;
pcartpos0.m_CartPos.m_Y = -0.685060;
pcartpos0.m_CartPos.m_Z = 1206.394710;
pcartpos0.m_CartPos.m_A = 0.000000;
pcartpos0.m_CartPos.m_B = 90.000000;
pcartpos0.m_CartPos.m_C = -180.00000;
pcartpos0.m_CartPos.m_Mode = 3;
for (int i = 0; i < 6; i++)
{
    pcartpos0.m_AuxPos[i] = i + 1;
}
result = robotClient.VariablePCartPosModifyCmd(project, program, variable,
pcartpos0);
if (result != 0) //检查是否成功
{
    std::cout << "修改pcartpos0值失败，结果为:" << result << std::endl;
}
```

2.5.58 在线修改PCARTPOS值

函数	int ProcessVariablePCartPosModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	PalletPosition，变量值
返回值 (int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "pcartpos0";
```

```
PalletPosition pcartpos0;
memcpy(pcartpos0.m_PalName, "null", (strlen("null") + 1));
pcartpos0.m_LayerNum = 1;
pcartpos0.m_PartNum = 2;
pcartpos0.m_CartPos.m_X = 926.331260;
pcartpos0.m_CartPos.m_Y = -0.685060;
pcartpos0.m_CartPos.m_Z = 1206.394710;
pcartpos0.m_CartPos.m_A = 0.000000;
pcartpos0.m_CartPos.m_B = 90.000000;
pcartpos0.m_CartPos.m_C = -180.00000;
pcartpos0.m_CartPos.m_Mode = 3;
for (int i = 0; i < 6; i++)
{
    pcartpos0.m_AuxPos[i] = i + 1;
}
result = robotClient.ProcessVariablePCartPosModifyCmd(project, program, variable,
pcartpos0);
if (result != 0) //检查是否成功
{
    std::cout << "修改pcartpos0值失败，结果为:" << result << std::endl;
}
```

2.5.59 修改OVERLAPREL值

函数	int VariableOverlapRELModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	int，变量值，0~100
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "olr0";
int olr0 = 20;
result = robotClient.VariableOverlapRELModifyCmd(project, program, variable, olr0);
if (result != 0) //检查是否成功
{
    std::cout << "修改olr0值失败，结果为:" << result << std::endl;
}
```

2.5.60 在线修改OVERLAPREL值

函数	int ProcessVariableOverlapRELModifyCmd(project,program,variable,value);

输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	int，变量值，0~100
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "olr0";
int olr0 = 20;
result = robotClient.ProcessVariableOverlapRELModifyCmd(project, program, variable,
olr0);
if (result != 0) //检查是否成功
{
    std::cout << "修改olr0值失败，结果为:" << result << std::endl;
}
```

2.5.61 修改OVERLAPABS值

函数	int VariableOverlapABSModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	int，变量值，0~200
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "ola0";
int ola0 = 10;
result = robotClient.VariableOverlapABSModifyCmd(project, program, variable, ola0);
if (result != 0) //检查是否成功
{
    std::cout << "修改ola0值失败，结果为:" << result << std::endl;
}
```

2.5.62 在线修改OVERLAPABS值

函数	int ProcessVariableOverlapABSModifyCmd(project,program,variable,value);	
输入	project	std::string，工程

输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	int，变量值，0~200
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "ola0";
int ola0 = 10;
result = robotClient.ProcessVariableOverlapABSModifyCmd(project, program, variable,
ola0);
if (result != 0) //检查是否成功
{
    std::cout << "修改ola0值失败，结果为:" << result << std::endl;
}
```

2.5.63 修改RAMPTYPE值

函数	int VariableRampTypeModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	RampType，变量值（枚举型）
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

RampType 说明

eTran,	T 型加速度
eSShape,	S 型速度
eSine,	Sin 型速度
eSShape_new	新的 7 段 s 型速度规划

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "ramp0";
RampType ramp0 = eSShape;
result = robotClient.VariableRampTypeModifyCmd(project, program, variable, ramp0);
if (result != 0) //检查是否成功
{
    std::cout << "修改ramp0值失败，结果为:" << result << std::endl;
}
```

2.5.64 在线修改RAMPTYPE值

函数	int ProcessVariableRampTypeModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	RampType，变量值（枚举型）
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "ramp0";
RampType ramp0 = eSShape;
result = robotClient.ProcessVariableRampTypeModifyCmd(project, program, variable,
ramp0);
if (result != 0) //检查是否成功
{
    std::cout << "修改ramp0值失败，结果为:" << result << std::endl;
}
```

2.5.65 修改BENDDATA值

函数	int VariableBendDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	BendTargetData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

BendTargetData 说明

double m_BendAngle;	折弯角度
double m_BendVel;	折弯速度
double m_BendAccTime;	折弯加速度
double m_BendDelayTime;	折弯延迟时间
double m_BendThickness;	板厚

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "benddata0";
BendTargetData benddata0;
benddata0.m_BendAngle = 90;
benddata0.m_BendVel = 5;
```

```
benddata0.m_BendAccTime = 200;
benddata0.m_BendDelayTime = 0;
benddata0.m_BendThickness = 1;
result = robotClient.VariableBendDataModifyCmd(project, program, variable,
benddata0);
if (result != 0) //检查是否成功
{
    std::cout << "修改benddata0值失败，结果为:" << result << std::endl;
}
```

2.5.66 在线修改BENDDATA值

函数	int ProcessVariableBendDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	BendTargetData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "benddata0";
BendTargetData benddata0;
benddata0.m_BendAngle = 90;
benddata0.m_BendVel = 5;
benddata0.m_BendAccTime = 200;
benddata0.m_BendDelayTime = 0;
benddata0.m_BendThickness = 1;
result = robotClient.ProcessVariableBendDataModifyCmd(project, program, variable,
benddata0);
if (result != 0) //检查是否成功
{
    std::cout << "修改benddata0值失败，结果为:" << result << std::endl;
}
```

2.5.67 修改BENDSYNDATA值

函数	int VariableBendSynDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	BendSynTargetData，变量值

返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开

BendSynTargetData 说明

double m_SynDistance;	同步距离
double m_SynVel;	同步速度
double m_SynAccTime;	同步加速时间
double m_SynDelayTime;	同步延迟时间

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "bendsyndata0";
BendSynTargetData bendsyndata0;
bendsyndata0.m_SynDistance = 100;
bendsyndata0.m_SynVel = 50;
bendsyndata0.m_SynAccTime = 200;
bendsyndata0.m_SynDelayTime = 0;
result = robotClient.VariableBendSynDataModifyCmd(project, program, variable,
bendsyndata0);
if (result != 0) //检查是否成功
{
    std::cout << "修改bendsyndata0值失败，结果为:" << result << std::endl;
}
```

2.5.68 在线修改BENDSYNDATA值

函数	int ProcessVariableBendSynDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	BendSynTargetData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "bendsyndata0";
BendSynTargetData bendsyndata0;
bendsyndata0.m_SynDistance = 100;
bendsyndata0.m_SynVel = 50;
bendsyndata0.m_SynAccTime = 200;
bendsyndata0.m_SynDelayTime = 0;
result = robotClient.ProcessVariableBendSynDataModifyCmd(project, program,
```

```
variable, bendsyndata0);
if (result != 0) //检查是否成功
{
    std::cout << "修改bendsyndata0值失败，结果为:" << result << std::endl;
}
```

2.5.69 修改POLISHDATA值

函数	int VariablePolishDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	PolishData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

PolishData 说明

PolishType m_PolishShape;	打磨类型，枚举型
unsigned int m_PolishNumber;	打磨个数，>=1
double m_Radius;	圆半径
double m_SemimajorAxle;	椭圆长半轴
double m_SemiminorAxle;	椭圆短半轴
double m_Angle;	椭圆倾斜角 0-180
double m_Amplitude;	sin 幅值
double m_PressAmount;	打磨下压量

PolishType 说明

ePolishCircle = 0,	eCircle
ePolishEllipse,	eEllipse,
ePolishSin	eSin

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "polishdata0";
PolishData polishdata0;
polishdata0.m_PolishShape = ePolishEllipse;
polishdata0.m_PolishNumber = 1;
polishdata0.m_Radius = 0;
polishdata0.m_SemimajorAxle = 3;
polishdata0.m_SemiminorAxle = 4;
polishdata0.m_Angle = 5;
polishdata0.m_Amplitude = 0;
polishdata0.m_PressAmount = 0;
result = robotClient.VariablePolishDataModifyCmd(project, program, variable,
polishdata0);
```

```
if (result != 0) //检查是否成功
{
    std::cout << "修改polishdata0值失败，结果为:" << result << std::endl;
}
```

2.5.70 在线修改POLISHDATA值

函数	int ProcessVariablePolishDataModifyCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	PolishData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "polishdata0";
PolishData polishdata0;
polishdata0.m_PolishShape = ePolishEllipse;
polishdata0.m_PolishNumber = 1;
polishdata0.m_Radius = 0;
polishdata0.m_SemimajorAxle = 3;
polishdata0.m_SemiminorAxle = 4;
polishdata0.m_Angle = 5;
polishdata0.m_Amplitude = 0;
polishdata0.m_PressAmount = 0;
result = robotClient.ProcessVariablePolishDataModifyCmd(project, program, variable,
polishdata0);
if (result != 0) //检查是否成功
{
    std::cout << "修改polishdata0值失败，结果为:" << result << std::endl;
}
```

2.5.71 修改TOOLLOAD值

函数	int VariableToolLoadModifyCmd(project,program,variable,value);	
输入	project	std::string，工程，"_global"，全局
输入	program	std::string，程序，"_project"，全局
输入	variable	std::string，变量名
输入	value	PayloadData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

PayloadData 说明

double Weight;	质量
double x;	质心 x
double y;	质心 y
double z;	质心 z
double ixx;	转动惯量 ixx
double iyy;	转动惯量 iyy
double izz;	转动惯量 izz
double ixy;	转动惯量 ixy
double ixz;	转动惯量 ixz
double iyz;	转动惯量 iyz

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "toolload0";
PayloadData toolload0;
toolload0.Weight = 1;
toolload0.x = 2;
toolload0.y = 3;
toolload0.z = 4;
toolload0.ixx = 5;
toolload0.iyy = 6;
toolload0.izz = 7;
toolload0.ixy = 8;
toolload0.ixz = 9;
toolload0.iyz = 10;
result = robotClient.VariableToolLoadModifyCmd(project, program, variable,
toolload0);
if (result != 0) //检查是否成功
{
    std::cout << "修改toolload0值失败，结果为:" << result << std::endl;
}
```

2.5.72 修改PAYLOAD值

函数	int VariablePayLoadModifyCmd(project,program,variable,value);	
输入	project	std::string，工程，"_global"，全局
输入	program	std::string，程序，"_project"，全局
输入	variable	std::string，变量名
输入	value	PayloadData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

PayloadData 说明

double Weight;	质量

double x;	质心 x
double y;	质心 y
double z;	质心 z
double ixx;	转动惯量 ixx
double iyy;	转动惯量 iyy
double izz;	转动惯量 izz
double ixy;	转动惯量 ixy
double ixz;	转动惯量 ixz
double iyz;	转动惯量 iyz

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "payload0";
PayloadData payload0;
payload0.Weight = 1;
payload0.x = 2;
payload0.y = 3;
payload0.z = 4;
payload0.ixx = 5;
payload0.iyy = 6;
payload0.izz = 7;
payload0.ixy = 8;
payload0.ixz = 9;
payload0.iyz = 10;
result = robotClient.VariablePayLoadModifyCmd(project, program, variable,
payload0);
if (result != 0) //检查是否成功
{
    std::cout << "修改payload0值失败，结果为:" << result << std::endl;
}
```

2.5.73 修改TOOL值

函数	int VariableToolModifyCmd(variable, value);	
输入	variable	std::string，变量名
输入	value	Tool，工具坐标变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

Tool 说明：

bool Fixed;	是否固定工具
double X;	工具 X
double Y;	工具 Y
double Z;	工具 Z
double A;	工具 A

double B;	工具 B
double C;	工具 C

示例：

```
int result = 0;
std::string variable = "tool1";          //示教器中已有该工具
Tool value = { false,100,100,100,0,0,0 };
result = robotClient.VariableToolModifyCmd(variable, value);
if (result != 0) //检查是否成功
{
    std::cout << "修改tool1值失败，结果为:" << result << std::endl;
}
```

2.5.74 修改CARTREF值

函数	int VariableRefModifyCmd(variable, value);	
输入	variable	std::string，变量名
输入	value	CartRefSys，用户坐标变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

CartRefSys 说明：

double X;	工具 X
double Y;	工具 Y
double Z;	工具 Z
double A;	工具 A
double B;	工具 B
double C;	工具 C
char m_BaseRef[32];	基座标

示例：

```
int result = 0;
std::string variable = "ref1";          //示教器中已有该用户坐标
CartRefSys value = { 10,0,0,0,0,0,"WORLD" };
result = robotClient.VariableRefModifyCmd(variable, value);
if (result != 0) //检查是否成功
{
    std::cout << "修改ref1值失败，结果为:" << result << std::endl;
}
```

注：用户坐标的 WORLD 和 ROBOTBASE 是不支持修改的。

2.6 变量获取接口

2.6.1 变量获取说明（必读）

1）变量获取接口为命令接口，不建议一直进行获取，在需要判断时获取判断即可；

2）获取全局变量时，工程名为“_global”，程序名为“_project”。

3）获取保持变量时，工程名为“_global”，程序名为“_record”。

2.6.2 获取BOOL值

函数	int VariableBoolReadCmd(project,program,variable,(ref)value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	bool，变量值 C#加 ref
返回值(int)	0：获取成功；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "bool0";
bool bool0 = false;
result = robotClient.VariableBoolReadCmd(project, program, variable, bool0);
if (result != 0) //检查是否成功
{
    std::cout << "获取bool0值失败，结果为:" << result << std::endl;
}
```

2.6.3 获取INT值

函数	int VariableIntReadCmd(project,program,variable, (ref)value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	int，变量值 C#加 ref
返回值(int)	0：获取成功；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "int0";
int int0 = 0;
result = robotClient.VariableIntReadCmd(project, program, variable, int0);
```

```
if (result != 0) //检查是否成功
{
    std::cout << "获取int0值失败，结果为:" << result << std::endl;
}
```

2.6.4 获取REAL值

函数	int VariableRealReadCmd(project,program,variable, (ref)value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	float，变量值 C#加 ref
返回值(int)	0：获取成功；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "real0";
float real0 = 0;
result = robotClient.VariableRealReadCmd(project, program, variable, real0);
if (result != 0) //检查是否成功
{
    std::cout << "获取real0值失败，结果为:" << result << std::endl;
}
```

2.6.5 获取string值

函数	int VariableStringReadCmd(project,program,variable, (ref)value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	std::string，变量值 C#加 ref
返回值(int)	0：获取成功；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";     //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "string0";
std::string string0 = "";
result = robotClient.VariableStringReadCmd(project, program, variable, string0);
if (result != 0) //检查是否成功
{
    std::cout << "获取string0值失败，结果为:" << result << std::endl;
```

```
}
```

2.6.6 获取Axispos值

函数	int VariableAxisposReadCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	AXISPOS，变量值
返回值(int)	0：获取成功；-1：执行失败；-2：数据收发异常；-3：通讯断开	

AXISPOS 说明：

JointsPos	m_Joint;	关节位置
	JointsPos[6]	Double 数组 0~5 表示 1~6 轴关节位置
AuxPos	m_AuxJoint;	附加轴位置
	AuxPos[6];	Double 数组 0~5 表示 7~12 轴关节位置

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "ap0";
AXISPOS ap0 = {0,0,0,0,0,0,0,0,0,0,0,0};
result = robotClient.VariableAxisposReadCmd(project, program, variable, ap0);
if (result != 0) //检查是否成功
{
    std::cout << "获取ap0值失败，结果为:" << result << std::endl;
}
```

2.6.7 获取CARTPOS值

函数	int VariableCartposReadCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	CARTPOS，变量值
返回值(int)	0：获取成功；-1：执行失败；-2：数据收发异常；-3：通讯断开	

CARTPOS 说明：

RobotCartPos	m_CartPos;	笛卡尔位置
	cart[6]	Double 数组 0~5 表示 1~6 轴笛卡尔位置
	m_Mode	int，位置模式
AuxPos	m_AuxJoint;	附加轴位置
	AuxPos[6];	Double 数组 0~5 表示 7~12 轴关节位置

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
```

```
std::string program = "program0";      //示教器中已有该程序
std::string variable = "cp0";
CARTPOS cp0 = {0,0,0,0,0,0,0,0,0,0,0,0};
result = robotClient.VariableCartposReadCmd(project, program, variable, cp0);
if (result != 0) //检查是否成功
{
    std::cout << "获取cp0值失败，结果为:" << result << std::endl;
}
```

2.6.8 获取ROBOTAXISPOS值

函数	Int VariableRobotAxisposReadCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	JointsPos，变量值
返回值(int)	0：获取成功；-1：执行失败；-2：数据收发异常；-3：通讯断开	

JointsPos 说明：

double[]	JointsPos[6]	0~5 表示 1~6 轴关节位置

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "rap0";
//定义机器人关节坐标变量并赋值
JointsPos rap0 = {0,0,0,0,0,0};
result = robotClient.VariableRobotAxisposReadCmd(project, program, variable, rap0);
if (result != 0) //检查是否成功
{
    std::cout << "获取rap0值失败，结果为:" << result << std::endl;
}
```

2.6.9 获取ROBOTCARTPOS值

函数	int VariableRobotCartposReadCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	RobotCartPos，变量值
返回值(int)	0：获取成功；-1：执行失败；-2：数据收发异常；-3：通讯断开	

RobotCartPos 说明：

double[]	cart[6]	0~5 表示 1~6 轴笛卡尔位置
int	m_Mode	位置模式

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "rcp0";
RobotCartPos rcp0 = {0,0,0,0,0,0,0};
result = robotClient.VariableRobotCartposReadCmd(project, program, variable, rcp0);
if (result != 0) //检查是否成功
{
    std::cout << "获取rcp0值失败，结果为:" << result << std::endl;
}
```

2.6.10 获取DYNAMIC值

函数	int VariableDynamicReadCmd(project,program,variable,value);	
输入	project	std::string，工程
输入	program	std::string，程序
输入	variable	std::string，变量名
输入	value	SDynamicPercent，变量值
返回值(int)	0：获取成功；-1：执行失败；-2：数据收发异常；-3：通讯断开	

SDynamicPercent 说明：

m_SegmentDynamic	double m_Vel;	位置速度
	double m_Acc;	位置加速度
	double m_Dec;	位置减速度
	double m_Jerk;	位置加加速度
	double m_Tjolt;	时间，未使用，默认-1
m_OriDynamic	double m_Vel;	旋转速度
	double m_Acc;	旋转加速度
	double m_Dec;	旋转减速度
	double m_Jerk;	旋转加加速度
	double m_Tjolt;	时间，未使用，默认-1
m_JointPercent	double m_Vel;	关节速度
	double m_Acc;	关节加速度
	double m_Dec;	关节减速度
	double m_Jerk;	关节加加速度
	double m_Tjolt;	时间，未使用，默认-1

示例：

```
int result = 0;
std::string project = "project0";      //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "dyn0";
//根据需求将其他值都赋值
SDynamicPercent dyn0 = {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0};
result = robotClient.VariableDynamicReadCmd(project, program, variable, dyn0);
if (result != 0) //检查是否成功
```

```
{
    std::cout << "获取dyn0值失败，结果为:" << result << std::endl;
}
```

2.6.11 获取ARCONDATA值

函数	int VariableArcOnDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	ArcOnData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

ArcOnData 说明：

unsigned int m_preFlowtime;	预送气时间
float m_arconCurrent;	起弧电流
float m_arconVoltage;	起弧电压
unsigned int m_arconTime;	起弧时间
unsigned int m_BeforeArcOn;	预起弧时间
unsigned int m_BeforeWireOut;	预送丝时间

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "arcon0";
ArcOnData arcondata = {0,0,0,0,0,0};
result = robotClient.VariableArcOnDataReadCmd(project, program, variable,
arcondata);
if (result != 0) //检查是否成功
{
    std::cout << "获取arcondata值失败，结果为:" << result << std::endl;
}
```

2.6.12 获取ARCOFFDATA值

函数	int VariableArcOffDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	ArcOffData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

ArcOffData 说明：

unsigned int m_burnBacktime;	回烧时间
unsigned int m_postFlowtime;	滞后送气时间

float m_arcoffCurrent;	收弧电流
float m_arcoffVoltage;	收弧电压
unsigned int m_arcoffTime;	收弧时间

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "arcoff0";
ArcOffData arcoff0 = {0,0,0,0,0};
result = robotClient.VariableArcOffDataReadCmd(project, program, variable,
arcoff0);
if (result != 0) //检查是否成功
{
    std::cout << "获取arcoff0值失败，结果为:" << result << std::endl;
}
```

2.6.13 获取ARCDATA值

函数	int VariableArcDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	ArcData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

ArcData 说明：

float m_weldVoltage;	焊接电压
float m_weldCurrent;	焊接电流
float m_weldSpeed;	焊接速度
double m_endSpeed;	焊接结束速度

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "arc0";
ArcData arc0 = {0,0,0,0};
result = robotClient.VariableArcDataReadCmd(project, program, variable, arc0);
if (result != 0) //检查是否成功
{
    std::cout << "获取arc0值失败，结果为:" << result << std::endl;
}
```

2.6.14 获取ARCRETRYDATA值

函数	int VariableArcRetryDataReadCmd(project,program,variable, ref value);

输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	ArcRetryData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

ArcRetryData 说明：

int m_DetectTime;	起弧失败检测时间
int m_RetryTimes;	起弧失败后，重试的次数
int m_RetractTime;	焊丝回抽时间
int m_RetractWaitTime;	回抽等待时间
double m_CurrentInc;	再起弧电流增量
double m_VoltageInc;	再起弧电压增量
String m_ScratchArcFlag	擦线起弧标志
double m_ScratchMaxDistance;	擦线距离
double m_ScratchStepLength;	擦线步长，每次移动距离
double m_ReturnSpeed;	擦线返回速度

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "retry0";
ArcRetryData retry0 = {0,0,0,0,0,0,"FALSE",0,0,0};
result = robotClient.VariableArcRetryDataReadCmd(project, program, variable,
retry0);
if (result != 0) //检查是否成功
{
    std::cout << "获取retry0值失败，结果为:" << result << std::endl;
}
```

2.6.15 获取ARCSEGDATA值

函数	int VariableArcSegDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	ArcSegData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

ArcSegData 说明：

double m_StartArcOnCurrent;	开始起弧电流
double m_StartArcOnVoltage;	开始起弧电压
int m_DecaySeg;	渐变段数
int m_PreflowTime;	预送气时间
double m_ArcOnCurrent;	起弧电流

double m_ArcOnVoltage;	起弧电压
int m_ArcOnWaitTime;	等待焊机起弧成功的时间
int m_ArcOnDecayTime;	起弧渐变时间
int m_FailDetectTime;	起弧失败检测时间
int m_RetryTimes;	起弧失败后，重试的次数
int m_RetractTime;	焊丝回抽时间
int m_RetractWaitTime;	回抽等待时间
double m_CurrentInc;	再起弧电流增量
double m_VoltageInc;	再起弧电压增量
int m_BurnbackTime;	回烧时间
int m_PostflowTime;	滞后送气时间
double m_ArcOffCurrent;	收弧电流
double m_ArcOffVoltage;	收弧电压
int m_ArcOffWaitTime;	等待焊机息弧成功的时间
int m_ArcOffDecayTime;	渐变时间
int m_StickCheckDelayTime;	粘丝检测延时时间

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "arcsegdata0";
ArcSegData arcsegdata0 = {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0};
result = robotClient.VariableArcSegDataReadCmd(project, program, variable,
arcsegdata0);
if (result != 0) //检查是否成功
{
    std::cout << "获取arcsegdata0值失败，结果为:" << result << std::endl;
}
```

2.6.16 获取ARCCONFIGDATA值

函数	int VariableArcConfigDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	double[14]，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "conf0";
double conf0[14] = { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 };
```

```
result = robotClient.VariableArcConfigDataReadCmd(project, program, variable,
conf0);
if (result != 0) //检查是否成功
{
    std::cout << "获取conf0值失败，结果为:" << result << std::endl;
}
```

2.6.17 获取WEAVEDATA值

函数	int VariableWeaveDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	WeaveData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

WeaveData 说明：

WeaveShape m_waveShape;	摆弧形状(枚举型)
WeaveType m_waveType;	摆弧类型(枚举型)
float m_weaveLength;	摆弧长度
float m_weaveWidth;	摆弧幅值
double m_SwingDirection;	摆弧倾斜角（对 sin 摆无效）
double m_WeaveAngle;	摆弧平面倾斜角
double m_SpaceAngle;	空间摆弧夹角（对 L 型摆和空间三角摆有效）
unsigned int m_PauseTime_1;	1/4 处停留时间
unsigned int m_PauseTime_2;	2/4 处停留时间
unsigned int m_PauseTime_3;	3/4 处停留时间
unsigned int m_PauseTime_4;	4/4 处停留时间
int m_PauseContinue;	停留连续
double m_EndLength;	结束长度
double m_EndWidth;	结束宽度
double m_CenterHigh;	中心高度

WeaveShape 说明：

eNoWeave= 0,	无摆弧
eSin = 5,	正弦摆
eSpiral,	斜椭圆摆
eObliqueTriangle,	斜三角摆
eSpaceTriangle,	空间三角摆
eLTriangle,	空间 L 型摆
eBackForward,	前后摆
eConstPoint	定点圆摆

WeaveType 说明：

eTCPWeave = 0,	TCP 摆
eWristWeave,	腕关节摆

e45JointWeave,	关节摆

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "wd0";
WeaveData wd0 = {eNoWeave,eTCPWeave,0,0,0,0,0,0,0,0,0,0,0,0,0};
result = robotClient.VariableWeaveDataReadCmd(project, program, variable, wd0);
if (result != 0) //检查是否成功
{
    std::cout << "获取wd0值失败，结果为:" << result << std::endl;
}
```

2.6.18 获取CD值

函数	int VariableCDReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	CDData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

CDData 说明：

int m_Calibrate;	是否标定，0 为 FALSE，1 为 TRUE
CartPos m_CartPos ;	机器人基准位置
AuxPosStrcut m_AuxPos ;	附加轴基准位置
CartPos m_CurrentPos ;	当前位置
CDDirection m_Direction;	寻位方向
String m_RefName	参考坐标系

CartPos 说明：

double m_X;	X 值
double m_Y;	Y 值
double m_Z;	Z 值
double m_A;	A 值
double m_B;	B 值
double m_C;	C 值
unsigned int m_Mode;	Mode 值

AuxPosStrcut 说明：

double[6]	m_jointData;	附加轴值

CDDirection 说明：

eCD_X_Positive = 1,	X+方向
eCD_X_Negative,	X-方向
eCD_Y_Positive,	Y+方向

eCD_Y_Negative,	Y-方向
eCD_Z_Positive,	Z+方向
eCD_Z_Negative	Z-方向

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "cd0";
CDData cd0 = {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,eCD_X_Positive,"WORLD"};
result = robotClient.VariableCDReadCmd(project, program, variable, cd0);
if (result != 0) //检查是否成功
{
    std::cout << "获取cd0值失败，结果为:" << result << std::endl;
}
```

2.6.19 获取CDS值

函数	int VariableCDSReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	ArcCDSData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

ArcCDSData 说明：

CORRECTTYPE m_Dimension;	寻位方式，枚举型
int m_temp1;	预留
int m_temp2;	预留
double m_temp3;	预留
double m_temp4;	预留
double m_temp5;	预留
CDData m_CD[7];	CDData 数组，7 个 CDData

CORRECTTYPE 说明：

e1D_CR = 1,	1D_CR
e2D_CR = 2,	2D_CR
e3D_CR = 3,	3D_CR
e6D_CR = 6,	6D_CR
e2D1R_CR = 7,	2D1R_CR
e3D1R_CR = 8,	3D1R_CR
eCircle_CR = 9,	Circle_ CR
e3D3R_CR = 10	3D3R_CR

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
```

```
std::string program = "program0";      //示教器中已有该程序
std::string variable = "cds0";
ArcCDSData cds0 = { e1D_CR,0,0,0,0,0,
0,0,0,0,0,
```

```
    std::cout << "获取pat0值失败，结果为:" << result << std::endl;
}
```

2.6.21 获取TRACKDATA值

函数	int VariableTrackDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	TrackData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

TrackData 说明：

int m_LateralBeginCycle;	横向纠偏开始周期[3,1000]
double Kpl;	横向比例增益,不是比例增益百分比[0.100]
double m_LeftCoefficient;	横向左边偏移系数(0,100]
double m_RightCoefficient;	横向右边偏移系数(0,100]
int m_VerticalModeFlag;	纵向模式标志位，为 0 则是常数模式,为 1 是反馈模式;
double m_Iref_Vertical;	纵向纠偏基准电流（常数模式）
int m_VerticalBeginCycle;	纵向取样开始周期[2,50]
int m_VerticalSustainCycle;	纵向取样持续周期[1,50]
double m_VerticalLength;	纵向周期长度[2,10]
double Kpv;	纵向比例增益，不是比例增益百分比[0,10]
int m_TypeofTimeOrDist;	未使用，赋值 0
unsigned int m_TimeInterval;	未使用，赋值 200
unsigned int m_DistInterval;	未使用，赋值 2
double m_YBias;	未使用，赋值 1
double m_ZBias;	未使用，赋值 1
double L_MinComp;	横向最小补偿[0,1000]
double L_MaxComp;	横向最大补偿[0,1000]
double L_TotalMaxComp;	横向最大补偿总[0,1000]
double L_RightComp;	横向不对称调整系数[-99.9,99.9]
doubletemp Temp_L;	double[6]数组，备用
double V_MinComp;	纵向最小补偿
double V_MaxComp;	纵向最大补偿
double V_TotalMaxComp;	纵向最大补偿总
double V_TopComp;	纵向不对称调整系数
doubletemp Temp_V;	double[6]数组，备用

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "trackdata0";
```

```
TrackData trackdata0 =
{0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0}；
result = robotClient.VariableTrackDataReadCmd(project, program, variable,
trackdata0)；
if (result != 0) //检查是否成功
{
    std::cout << "获取trackdata0值失败，结果为:" << result << std::endl;
}
```

2.6.22 获取LASERTRACKDATA值

函数	int VariableLaserTrackDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	LaserTrackData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

LaserTrackData 说明：

double m_EndSpeed;	最大速度
double m_EndCurrent;	最大电流
double m_YBias;	Y 方向偏置距离
double m_ZBias;	Z 方向偏置距离
double m_MaxWidth;	最大坡口宽度
String m_AdapLength	宽度自适应开关
double m_StandardWidth;	标准坡口宽度
double m_WidthRatio;	宽度比例

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "lasertrackdata0";
LaserTrackData lasertrackdata0 = {0,0,0,0,0,"FALSE",0,0}；
result = robotClient.VariableLaserTrackDataReadCmd(project, program, variable,
lasertrackdata0)；
if (result != 0) //检查是否成功
{
    std::cout << "获取lasertrackdata0值失败，结果为:" << result << std::endl;
}
```

2.6.23 获取PLASERFILTER值

函数	int VariablePlaserFilterReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序

输入	variable	string，变量名
输入	value	PLaserFilter，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

PLaserFilter 说明：

FilterType m_FilterType;	滤波器类型（枚举型）
PointType m_PointType;	目标点类型（枚举型）
int m_FilterParam;	滤波器参数
int m_FilterLength;	滤波长度
double m_Featurethreshold;	特征点阈值

FilterType 说明：

eNoFilter,	无滤波
eMedianFilter,	中值滤波
eMeanFilter,	均值滤波
eGaussFilter	高斯滤波

PointType 说明：

eMax,	第一处极大值
eMin,	第一处极小值
eUpLeft,	第一处上升沿左侧
eUpRight,	第一处上升沿右侧
eDownLeft,	第一处下降沿左侧
eDownRight,	第一处下降沿右侧

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "plaserfilter0";
PLaserFilter plaserfilter0 = {eNoFilter,eMax,1,1,0};
result = robotClient.VariablePLaserFilterReadCmd(project, program, variable,
plaserfilter0);
if (result != 0) //检查是否成功
{
    std::cout << "获取plaserfilter0值失败，结果为:" << result << std::endl;
}
```

2.6.24 获取LASERARCONDATA值

函数	int VariableLaserArcOnDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	LaserArcOnNewData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

LaserArcOnNewData 说明：

LaserArcOnData m_LaserOnData;	滤波器类型（枚举型）
LaserArcData m_LaserData;	目标点类型（枚举型）

LaserArcOnData 说明：

int m_PreFlowTime;	预送气时间
int m_PreWireTime;	预送丝时间
int m_DelayLightTime;	延迟开光时间
double m_WireSpeed;	送丝速度
double m_Temp4;	预留
double m_Temp3;	预留
double m_Temp2;	预留
double m_Temp1;	预留

LaserArcData 说明：

double m_LaserPower;	激光功率
int m_PWM;	占空比
int m_Frequency;	频率
double m_WireSpeed;	送丝速度
double m_Temp4;	预留
double m_Temp3;	预留
double m_Temp2;	预留
double m_Temp1;	预留

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "laserarcondata0";
LaserArcOnNewData laserarcondata0 = {0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0};
result = robotClient.VariableLaserArcOnDataReadCmd(project, program, variable,
laserarcondata0);
if (result != 0) //检查是否成功
{
    std::cout << "获取laserarcondata0值失败，结果为:" << result << std::endl;
}
```

2.6.25 获取LASERARCDATA值

函数	int VariableLaserArcDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	LaserArcData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

LaserArcData 说明：

double m_LaserPower;	激光功率
int m_PWM;	占空比
int m_Frequency;	频率
double m_WireSpeed;	送丝速度
double m_Temp4;	预留
double m_Temp3;	预留
double m_Temp2;	预留
double m_Temp1;	预留

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "laserarcdata0";
LaserArcData laserarcdata0 = {0,0,0,0,0,0,0,0};
result = robotClient.VariableLaserArcDataReadCmd(project, program, variable,
laserarcdata0);
if (result != 0) //检查是否成功
{
    std::cout << "获取laserarcdata0值失败，结果为:" << result << std::endl;
}
```

2.6.26 获取LASERARCOFFDATA值

函数	int VariableLaserArcOffDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	LaserArcOffData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

LaserArcOffData 说明：

double m_LaserPower;	激光功率
int m_PWM;	占空比
int m_Frequency;	频率
double m_WireSpeed;	送丝速度
double m_Temp4;	预留
double m_Temp3;	预留
double m_Temp2;	预留
double m_Temp1;	预留

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
```

```
std::string program = "program0";      //示教器中已有该程序
std::string variable = "laserarcoffdata0";
LaserArcOffData laserarcoffdata0 = {0,0,0,0,0,0,0,0,0,0};
result = robotClient.VariableLaserArcOffDataReadCmd(project, program, variable,
laserarcoffdata0);
if (result != 0) //检查是否成功
{
    std::cout << "获取laserarcoffdata0值失败，结果为:" << result << std::endl;
}
```

2.6.27 获取AXISDIST值

函数	int VariableAxisDistReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	AxisPos，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

AXISPOS 说明

JointsPos	double m_Joint[6];	J1/J2/J3/J4/J5/J6
AuxPos	double m_AuxPos[6];	附加轴位置

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "apd0";
AxisPos apd0 = {0,0,0,0,0,0,0,0,0,0,0,0};
result = robotClient.VariableAxisDistReadCmd(project, program, variable, apd0);
if (result != 0) //检查是否成功
{
    std::cout << "获取apd0值失败，结果为:" << result << std::endl;
}
```

2.6.28 获取CARTDIST值

函数	int VariableCartDistReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	CartDist，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

CartDist 说明

double	m_X	X
double	m_Y	Y

double	m_Z	Z
double	m_A	A
double	m_B	B
double	m_C	C
AuxPos	double m_AuxPos[6];	附加轴位置

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "cpd0";
CartDist cpd0 = {0,0,0,0,0,0,0,0,0,0,0,0};
result = robotClient.VariableCartDistReadCmd(project, program, variable, cpd0);
if (result != 0) //检查是否成功
{
    std::cout << "获取cpd0值失败，结果为:" << result << std::endl;
}
```

2.6.29 获取AUXAXISPOS值

函数	int VariableAuxAxisPosReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	AuxAxisPos，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

AuxAxisPos 说明

AuxPos	double[6];	附加轴位置

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "aap0";
AuxAxisPos aap0 = {0,0,0,0,0,0};
result = robotClient.VariableAuxAxisPosReadCmd(project, program, variable, aap0);
if (result != 0) //检查是否成功
{
    std::cout << "获取aap0值失败，结果为:" << result << std::endl;
}
```

2.6.30 获取PCARTPOS值

函数	int VariablePCartPosReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序

输入	variable	string，变量名
输入	value	PalletPosition，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

PalletPosition 说明

String m_PalName;	码垛变量名称
unsigned int m_LayerNum;	层号
unsigned int m_PartNum;	件号
CartPos m_CartPos;	笛卡尔坐标
AuxPosStrcut m_AuxPos;	附加轴坐标

CartPos 说明：

double m_X;	X 值
double m_Y;	Y 值
double m_Z;	Z 值
double m_A;	A 值
double m_B;	B 值
double m_C;	C 值
unsigned int m_Mode;	Mode 值

AuxAxisPos 说明

AuxPos	double[6];	附加轴位置

示例：

```
int result = 0;
std::string project = "project0";          //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "pcartpos0";
PalletPosition pcartpos0 = {"null",0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0};
result = robotClient.VariablePCartPosReadCmd(project, program, variable,
pcartpos0);
if (result != 0) //检查是否成功
{
    std::cout << "获取pcartpos0值失败，结果为:" << result << std::endl;
}
```

2.6.31 获取OVERLAPREL值

函数	int VariableOverlapRELReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	int，变量值，0~100
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
```

```
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "olr0";
int olr0 = 20;
result = robotClient.VariableOverlapRELReadCmd(project, program, variable, olr0);
if (result != 0) //检查是否成功
{
    std::cout << "获取olr0值失败，结果为:" << result << std::endl;
}
```

2.6.32 获取OVERLAPABS值

函数	int VariableOverlapABSReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	int，变量值，0~200
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "ola0";
int ola0 = 20;
result = robotClient.VariableOverlapABSReadCmd(project, program, variable, ola0);
if (result != 0) //检查是否成功
{
    std::cout << "获取ola0值失败，结果为:" << result << std::endl;
}
```

2.6.33 获取RAMPTYPE值

函数	int VariableRampTypeReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	RampType，变量值（枚举型）
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

RampType 说明

eTran,	T 型加速度
eSShape,	S 型速度
eSine,	Sin 型速度
eSShape_new	新的 7 段 s 型速度规划

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "ramp0";
RampType ramp0 = eSShape;
result = robotClient.VariableRampTypeReadCmd(project, program, variable, ramp0);
if (result != 0) //检查是否成功
{
    std::cout << "获取ramp0值失败，结果为:" << result << std::endl;
}
```

2.6.34 获取BENDDATA值

函数	int VariableBendDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	BendTargetData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

BendTargetData 说明

double m_BendAngle;	折弯角度
double m_BendVel;	折弯速度
double m_BendAccTime;	折弯加速度
double m_BendDelayTime;	折弯延迟时间
double m_BendThickness;	板厚

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "benddata0";
BendTargetData benddata0 = {0,0,0,0,0};
result = robotClient.VariableBendDataReadCmd(project, program, variable,
benddata0);
if (result != 0) //检查是否成功
{
    std::cout << "获取benddata0值失败，结果为:" << result << std::endl;
}
```

2.6.35 获取BENDSYNDATA值

函数	int VariableBendSynDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名

输入	value	BendSynTargetData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

BendSynTargetData 说明

double m_SynDistance;	同步距离
double m_SynVel;	同步速度
double m_SynAccTime;	同步加速时间
double m_SynDelayTime;	同步延迟时间

示例：

```
int result = 0;
std::string project = "project0";          //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "bendsyndata0";
BendTargetData bendsyndata0 = {0,0,0,0,0};
result = robotClient.VariableBendDataReadCmd(project, program, variable,
bendsyndata0);
if (result != 0) //检查是否成功
{
    std::cout << "获取bendsyndata0值失败，结果为:" << result << std::endl;
}
```

2.6.36 获取POLISHDATA值

函数	int VariablePolishDataReadCmd(project,program,variable, ref value);	
输入	project	string，工程
输入	program	string，程序
输入	variable	string，变量名
输入	value	PolishData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

PolishData 说明

PolishType m_PolishShape;	打磨类型，枚举型
unsigned int m_PolishNumber;	打磨个数，>=1
double m_Radius;	圆半径
double m_SemimajorAxle;	椭圆长半轴
double m_SemiminorAxle;	椭圆短半轴
double m_Angle;	椭圆倾斜角 0-180
double m_Amplitude;	sin 幅值
double m_PressAmount;	打磨下压量

PolishType 说明

ePolishCircle = 0,	eCircle
ePolishEllipse,	eEllipse,
ePolishSin	eSin

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "polishdata0";
PolishData polishdata0 = {ePolishEllipse,0,0,0,0,0,0,0};
result = robotClient.VariablePolishDataReadCmd(project, program, variable,
polishdata0);
if (result != 0) //检查是否成功
{
    std::cout << "获取polishdata0值失败，结果为:" << result << std::endl;
}
```

2.6.37 获取TOOLLOAD值

函数	int VariableToolLoadReadCmd(project,program,variable, ref value);	
输入	project	string，工程，"_global"，全局
输入	program	string，程序，"_project"，全局
输入	variable	string，变量名
输入	value	PayloadData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

PayloadData 说明

double Weight;	质量
double x;	质心 x
double y;	质心 y
double z;	质心 z
double ixx;	转动惯量 ixx
double iyy;	转动惯量 iyy
double izz;	转动惯量 izz
double ixy;	转动惯量 ixy
double ixz;	转动惯量 ixz
double iyz;	转动惯量 iyz

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";     //示教器中已有该程序
std::string variable = "toolload0";
PayloadData toolload0 = {0,0,0,0,0,0,0,0,0,0};
result = robotClient.VariableToolLoadReadCmd(project, program, variable,
toolload0);
if (result != 0) //检查是否成功
{
    std::cout << "获取toolload0值失败，结果为:" << result << std::endl;
}
```

2.6.38 获取PAYLOAD值

函数	int VariablePayLoadReadCmd(project,program,variable, ref value);	
输入	project	string，工程，"_global"，全局
输入	program	string，程序，"_project"，全局
输入	variable	string，变量名
输入	value	PayloadData，变量值
返回值(int)	0：执行完成；-1：执行失败；-2：数据收发异常；-3：通讯断开	

PayloadData 说明

double Weight;	质量
double x;	质心 x
double y;	质心 y
double z;	质心 z
double ixx;	转动惯量 ixx
double iyy;	转动惯量 iyy
double izz;	转动惯量 izz
double ixy;	转动惯量 ixy
double ixz;	转动惯量 ixz
double iyz;	转动惯量 iyz

示例：

```
int result = 0;
std::string project = "project0";        //示教器中已有该工程
std::string program = "program0";      //示教器中已有该程序
std::string variable = "payload0";
PayloadData payload0 = {0,0,0,0,0,0,0,0,0,0};
result = robotClient.VariablePayLoadReadCmd(project, program, variable, payload0);
if (result != 0) //检查是否成功
{
    std::cout << "获取payload0值失败，结果为:" << result << std::endl;
}
```

2.6.39 获取TOOL值

函数	int VariableToolReadCmd(variable, value);	
输入	variable	std::string，变量名
输入	value	Tool，工具坐标变量值
返回值(int)	0：获取成功；-1：执行失败；-2：数据收发异常；-3：通讯断开	

Tool 说明：

bool Fixed;	是否固定工具
double X;	工具 X
double Y;	工具 Y
double Z;	工具 Z
double A;	工具 A
double B;	工具 B

double C;	工具 C

示例：

```
int result = 0;
std::string variable = "tool0";
Tool tool0 = {false,0,0,0,0,0,0};
result = robotClient.VariableToolReadCmd( variable, tool0);
if (result != 0) //检查是否成功
{
    std::cout << "获取tool0值失败，结果为:" << result << std::endl;
}
```

2.6.40 获取CARTREF值

函数	int VariableRefReadCmd(variable, value);	
输入	variable	std::string，变量名
输入	value	CartRefSys，用户坐标变量值
返回值(int)	0：获取成功；-1：执行失败；-2：数据收发异常；-3：通讯断开	

CartRefSys 说明：

double X;	工具 X
double Y;	工具 Y
double Z;	工具 Z
double A;	工具 A
double B;	工具 B
double C;	工具 C
char m_BaseRef[32];	基座标

示例：

```
int result = 0;
std::string variable = "ref0";
CartRefSys ref0 = {0,0,0,0,0,0,"WORLD"};
result = robotClient.VariableRefReadCmd( variable, ref0);
if (result != 0) //检查是否成功
{
    std::cout << "获取ref0值失败，结果为:" << result << std::endl;
}
```

2.7 应用功能接口

2.7.1 关节绝对位置点动（手动模式可用）

函数	int JogToRobotAxisposCmd(type,value,toolname,refname);	
输入	type	JogPositionType，点动类型，PTP 或 Lin
输入	value	JointsPos，关节位置
输入	toolname	std::string，工具坐标
输入	refname	std::string，用户坐标
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

JogPositionType 说明：

	ePTP = 0,	点到点运行
	eLine = 1	直线运行

JointsPos 说明：

double[6]	关节值

示例：

```
int result = 0;
std::string tool1 = "Default";
std::string ref1 = "WORLD";
//新建关节位置，并赋初始值
JointsPos posvalue = { 40,40,50,60,50,50 };
//关节绝对点动
result = robotClient.JogToRobotAxisposCmd(ePTP, posvalue, tool1, ref1);
if (result != 0)
{
    std::cout << "点动失败，结果为:"<< std::endl;
}
std::string input;
std::cin >> input; //等待到位，可替换其他判断条件，如当前机器人是否运行中
result = robotClient.JogToPositionExitCmd();//关闭点动接口
if (result != 0)
{
    std::cout << "关闭点动失败，结果为:" << std::endl;
}
```

2.7.2 关节绝对位置点动（手动模式可用）

函数	int JogToRobotCartposCmd(type,value,toolname,refname);	
输入	type	JogPositionType，点动类型，PTP 或 Lin
输入	value	CartPos，笛卡尔位置
输入	toolname	std::string，工具坐标
输入	refname	std::string，用户坐标
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

JogPositionType 说明：

	ePTP = 0,	点到点运行
	eLine = 1	直线运行

CartPos 说明：

double	m_X;	X 值
	m_Y;	Y 值
	m_Z;	Z 值
	m_A;	A 值
	m_B;	B 值
	m_C;	C 值
	m_Mode;	Mode 值

示例：

```
int result = 0;
std::string tool1 = "Default";
std::string ref1 = "WORLD";
//新建笛卡尔位置，并赋初始值
CartPos posvalue = { 40,40,50,60,50,50,0 };
//笛卡尔绝对点动
result = robotClient.JogToRobotCartposCmd(eLine, posvalue, tool1, ref1);
if (result != 0)
{
    std::cout << "点动失败，结果为:"<< std::endl;
}
std::string input;
std::cin >> input; //等待到位，可替换其他判断条件，如当前机器人是否运行中
result = robotClient.JogToPositionExitCmd();//关闭点动接口
if (result != 0)
{
    std::cout << "关闭点动失败，结果为:" << std::endl;
}
```

2.7.3 设置焊接模式

函数	int SetArcWeldingModeCmd(value);	
输入	value	int, 0 是仿真，1 是实焊，2 是空跑
返回值(int)	0：执行中；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
//设置焊接模式为仿真焊接
result = robotClient.SetArcWeldingModeCmd(0);
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
```

```
}
else
{
    //执行需要时间，等待成功
    for (int i = 0; i < 20; i++)
    {
        if (eSimuWelding == robotClient.getWeldingMode())
        {
            break; //如果正确，跳出for循环
        }
        Sleep(50); //50ms后再次获取状态
    }
}
```

2.7.4 获取焊机电压电流

函数	int GetWeldingInfoCmd(Value);	
输出	Value	WeldingParamFeedBack，焊机反馈电压电流
返回值(int)	0：正常；-1：执行失败；-2：数据收发异常；-3：通讯断开	

WeldingParamFeedBack 说明：

double	m_SetCurrent;	设置电流
double	m_SetVoltage;	设置电压
double	m_RealCurrent;	实际电流
double	m_RealVoltage;	实际电压

示例：

```
int result = 0;
//新建焊机数据变量，并赋初始值
WeldingParamFeedBack weldvalue = { 0,0,0,0 };
//获取焊机数据
result = robotClient.GetWeldingInfoCmd(weldvalue);
if (result != 0) //检查是否成功
{
    std::cout << "设置失败，结果为:" << result << std::endl;
}
else
{
    std::cout << "m_SetVoltage:" << weldvalue.m_SetVoltage << std::endl;
    std::cout << "m_RealVoltage:" << weldvalue.m_RealVoltage << std::endl;
    std::cout << "m_SetCurrent:" << weldvalue.m_SetCurrent << std::endl;
    std::cout << "m_RealCurrent:" << weldvalue.m_RealCurrent << std::endl;
}
```

2.7.5 关节坐标转笛卡尔坐标（正解）

函数	int TransSDKGetKinematics(jointpos, cartpos);	
输入	jointpos	SDKInstructKinematics
输出	cartpos	KinematicsInfo，笛卡尔坐标
返回值(int)	0：正常；-1：执行失败；-2：数据收发异常；-3：通讯断开	

SDKInstructKinematics 说明：

JointsPos	joint;	关节坐标值
char	m_ToolName[32];	工具坐标
char	m_RefName[32];	用户坐标

JointsPos 说明：

double[]	JointsPos[6]	0~5 表示 1~6 轴关节位置

KinematicsInfo 说明：

CartSys	xyzabc;	笛卡尔坐标值
unsigned int	modetmp;	模式

CartSys 说明：

double	m_X;	mm
double	m_Y;	mm
double	m_Z;	mm
double	m_A;	°
double	m_B;	°
double	m_C;	°

示例：

```
int result = 0;
//正解
SDKInstructKinematics jointpos1 = { 0,0,0,0,90.2,0.0,"Default","WORLD" };
KinematicsInfo cartpos1;
result = robotClient.TransSDKGetKinematics(jointpos1, cartpos1);
if (result == 0) {
    std::cout << "cartpos1 m_X is:" << cartpos1.xyzabc.m_X << std::endl;
    std::cout << "cartpos1 m_Y is:" << cartpos1.xyzabc.m_Y << std::endl;
    std::cout << "cartpos1 m_Z is:" << cartpos1.xyzabc.m_Z << std::endl;
    std::cout << "cartpos1 m_A is:" << cartpos1.xyzabc.m_A << std::endl;
    std::cout << "cartpos1 m_B is:" << cartpos1.xyzabc.m_B << std::endl;
    std::cout << "cartpos1 m_C is:" << cartpos1.xyzabc.m_C << std::endl;
    std::cout << "cartpos1 modetmp is:" << cartpos1.modetmp << std::endl;
}
```

2.7.6 笛卡尔坐标转关节坐标（逆解）

函数	int TransSDKGetInvKinematics(cartpos, jointpos);	
输入	cartpos	SDKInstructInvKinematics
输出	jointpos	JointsPos，关节坐标
返回值(int)	0：正常；-1：执行失败；-2：数据收发异常；-3：通讯断开	

SDKInstructInvKinematics 说明：

CartSys	xyzabc;	笛卡尔坐标值
unsigned int	modetmp;	模式
char	m_ToolName[32];	工具坐标
char	m_RefName[32];	用户坐标

CartSys 说明：

double	m_X;	mm
double	m_Y;	mm
double	m_Z;	mm
double	m_A;	°
double	m_B;	°
double	m_C;	°

JointsPos 说明：

double[]	JointsPos[6]	0~5 表示 1~6 轴关节位置

示例：

```
int result = 0;
//逆解
SDKInstructInvKinematics cartpos = { 1089.3466,190.0992,-1139.8677,-118.1903,-51.7915,52.9105,8,"Default","WORLD" };
JointsPos jointpos;
result = robotClient.TransSDKGetInvKinematics(cartpos, jointpos);
if (result == 0) {
    std::cout << "jointpos j1 is:" << jointpos[0] << std::endl;
    std::cout << "jointpos j2 is:" << jointpos[1] << std::endl;
    std::cout << "jointpos j3 is:" << jointpos[2] << std::endl;
    std::cout << "jointpos j4 is:" << jointpos[3] << std::endl;
    std::cout << "jointpos j5 is:" << jointpos[4] << std::endl;
    std::cout << "jointpos j6 is:" << jointpos[5] << std::endl;
}
```

2.8 语句控制接口

2.8.1 语句控制使用总览（必读）

使用流程说明：

语句控制，后台模拟加载一个程序，并向这个程序里写语句实现对机器人的语句控制。新增了行号设置，可以在每条语句中选择输入行号，通过获取行号来判断当前程序运行到哪一条语句（不设置时默认为 0）。

使用逻辑：

1.使能（EnableMotorCmd）→

//判断使能状态后进行下一步（getMotorEnableState）

2.加载模拟程序（SDKSimulateProgramLoad）→

//加载前，判断当前加载工程名、程序名，若有程序加载中，建议先进行总卸载（ProgramKillCmd）

3.开始运行（SetModeCmd(START)）→

//判断加载状态后，开始

4.写语句（SDKSimulate…ProCmd）→

//根据需求写语句，单次发送语句数量最长为 10 条，可通过 NumOfPro 查看当前已写语句数量

5.发送语句（SDKSimulateProgramSend）→

//写语句仅将语句保存在缓存区，调用发送语句后，将程序发给机器人运行

6.停止运行（SetModeCmd(STOP)）→

//程序停止，若程序运行完停止，可在程序最后一行写 EOF 语句，发送语句，判断程序运行状态（getProgramState），之后停止程序或者做其他操作

7.卸载模拟程序（SDKSimulateProgramKill）.

//判断程序运行状态（getProgramState），卸载模拟程序

2.8.2 当前缓存区语句数量

变量	NumOfPro

示例：

```
if (robotClient.NumOfPro > 10) //当前设置的最大条数为10
{
    std::cout << "缓存区满啦，不能写语句了，先把当前程序发送给机器人跑吧！" <<
std::endl;
}
```

2.8.3 加载模拟程序

函数	int SDKSimulateProgramLoad(is_blocked);	
输入	is_blocked	bool，阻塞标志。默认 false，当前命令完成后给返回值，不用再进行状态判断
返回值(int)	0：正常；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
//卸载，加载程序
if (robotClient.getProgramName() != "SDK")//若当前程序不是“SDK”
{
    if (robotClient.getProgramName() != "")//若当前程序不是“”（无程序）
    { //kill
        result = robotClient.ProgramKillCmd(robotClient.getProjectName(),
robotClient.getProgramName());//卸载当前程序
        if (result != 0)
        {//若卸载失败，打印卸载结果
            std::cout << "卸载失败，卸载结果:" << result << std::endl;
        }
        while (robotClient.getProgramName() != "") //等待卸载成功
        {
            Sleep(10);
        }
    }
    result = robotClient.SDKSimulateProgramLoad();//加载模拟程序
    if (result != 0)
    {//若加载失败，打印加载结果
        std::cout << "加载失败，加载结果:" << result << std::endl;
    }
    while (robotClient.getProgramName() != "SDK")//等待加载成功
    {
        Sleep(10);
    }
}
else
{   //卸载
    result = robotClient.SDKSimulateProgramKill();
    if (result != 0)
    {
        std::cout << "卸载结果:" << result << std::endl;
        return;
    }
    while (robotClient.getProgramName() != "")
    {
```

```
            Sleep(10);
        }
        //加载
        result = robotClient.SDKSimulateProgramLoad();
        if (result != 0)
        {
            std::cout << "加载结果:" << result << std::endl;
            return;
        }
        while (robotClient.getProgramName() != "SDK")
        {
            Sleep(10);
        }
    }
    //根据异常进行处理
```

2.8.4 卸载模拟程序

函数	int SDKSimulateProgramKill(is_blocked);	
输入	is_blocked	bool，阻塞标志。默认 false，当前命令完成后给返回值，不用再进行状态判断
返回值(int)	0：正常；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例见上条。

2.8.5 发送语句

函数	int SDKSimulateProgramSend(line_number);	
输出	line_number	int，语句错误行号
返回值(int)	0：正常；-1：执行失败；-2：数据收发异常；-3：通讯断开	

示例：

```
int result = 0;
//新建错误行号，并赋初始值
int linenumber = 0;
//发送语句
result = robotClient.SDKSimulateProgramSend(linenumber);
//检查语句是否发送成功
if (result != 0)
{
    std::cerr << "语句写入失败，结果为:" << result << std::endl;
}
```

注：写语句后需要发送语句，机器人才开始运行发送的程序。

2.8.6 写EOF语句

函数	int SDKSimulateEOFCmd(linenum);

输入	linenum	int，语句行号，无输入时默认为 1
返回值(int)	0：正常；-1：执行失败；	

示例：

```
int result = 0;
//写EOF语句
result = robotClient.SDKSimulateEOFCmd();
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "EOF写入失败，结果为:" << result << std::endl;
}
```

注：该语句为程序结束语句，示教器程序为自动升成，虚拟程序需要在程序结束时写入该语句。

2.8.7 写SetTool语句

函数	int SDKSimulateSetToolCmd(toolname,linenum);	
输入	toolname	std::string，工具坐标
输入	linenum	int，语句行号，无输入时默认为 1
返回值(int)	0：正常；-1：执行失败	

示例：

```
int result = 0;
//设置工具为“Default”
std::string tool1 = "Default";
//写SetTool语句
result = robotClient.SDKSimulateSetToolCmd(tool1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "SetTool写入失败，结果为:" << result << std::endl;
}
```

2.8.8 写SetRefSys语句

函数	int SDKSimulateSetRefSysCmd(refname, linenum);	
输入	refname	Std::string，用户坐标
输入	linenum	int，语句行号，无输入时默认为 1
返回值(int)	0：正常；-1：执行失败	

示例：

```
int result = 0;
//设置工具为“WORLD”
std::string ref1 = "WORLD";
//写SetRefSys语句
```

```
result = robotClient.SDKSimulateSetRefSysCmd(ref1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "SetRefSys写入失败，结果为:" << result << std::endl;
}
```

2.8.9 写PTP语句

函数	int SDKSimulatePTPProCmd(endpos,dyn,ovl,toolname,refname, linenum);	
输入	endpos	HMIPos，目标点
输入	dyn	int，动态特性，只可以设置位置速度，其他参数利用 Dyn 语句设置;NULL 可以设置-1
输入	ovl	SDKOverlap，圆滑数据，NULL 时为-1
输入	toolname	Std::string，工具名
输入	refname	Std::string，用户名
输入	linenum	int，语句行号，无输入时默认为 1
返回值(int)	0：正常；-1：执行失败	

HMIPos 说明：

HMIPOSTYPE	m_PosType	点位类型
	eAxisPos = 1,	关节坐标类型（含附加轴）
	eCartPos,	笛卡尔坐标类型（含附加轴）
	eRobotAxisPos,	机器人关节坐标类型
	eRobotCartPos,	机器人笛卡尔坐标类型
int	m_Port	未使用，无需赋值，默认为 0
RcPos	m_Pos	点位数据
	CartPos m_CartPos;	笛卡尔坐标，xyzabc+mode
	AuxPos m_AuxPos;	附加轴信息
	JointsPos m_Joint;	关节部分
	POSTYPE m_postyp;	位置类型，未使用，无需赋值

SDKOverlap 说明：

double	m_tcp_distance	绝对距离
double	m_Percent	%百分比
OVERLAPTYPE	m_OveTyp	圆滑给定方式
	eOVLABS = 1	绝对值给定
	eOVLREL	百分比给定

示例：

```
int result = 0;
//新建目标点
HMIPos endpos ;
//设置点位类型为机器人关节坐标
```

```
endpos.m_PosType = eRobotAxisPos;
//给机器人6个关节赋值
endpos.m_Pos.m_Joint[0] = 0;
endpos.m_Pos.m_Joint[1] = 0;
endpos.m_Pos.m_Joint[2] = 0;
endpos.m_Pos.m_Joint[3] = 0;
endpos.m_Pos.m_Joint[4] = 90;
endpos.m_Pos.m_Joint[5] = 0;
//设置动态特性为NULL
int dyn = -1;
//新建圆滑特性
SDKOverlap ovl;
//设置圆滑特性类型为相对圆滑
ovl.m_OveTyp = eOVLREL;
//设置圆滑参数为100%
ovl.m_Percent = 100;
//设置工具为“Default”
std::string tool1 = "Default";
//设置工具为“WORLD”
std::string ref1 = "WORLD";
//写PTP语句
result = robotClient.SDKSimulatePTPProCmd(endpos, dyn, ovl, tool1, ref1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "PTP写入失败，结果为:" << result << std::endl;
}
```

2.8.10 写Lin语句

函数	int SDKSimulateLinProCmd (endpos,dyn,ovl,ori,toolname,refname, linenum);	
输入	endpos	HMIPos，目标点
输入	dyn	int，动态特性，只可以设置位置速度，其他参数利用 Dyn 语句设置;NULL 设置-1
输入	ovl	SDKOverlap，圆滑数据，NULL 时为-1
输入	ori	ORITYPE，姿态参数，枚举，NULL 为 0
输入	toolname	std::string，工具名
输入	refname	std::string，用户名
输入	linenum	int，语句行号，无输入时默认为 1
返回值(int)	0：正常；-1：执行失败	

HMIPos 说明：

HMIPOSTYPE	m_PosType	点位类型
	eAxisPos = 1,	关节坐标类型（含附加轴）

	eCartPos,	笛卡尔坐标类型（含附加轴）
	eRobotAxisPos,	机器人关节坐标类型
	eRobotCartPos,	机器人笛卡尔坐标类型
int	m_Port	未使用，无需赋值，默认为 0
RcPos	m_Pos	点位数据
	CartPos m_CartPos;	笛卡尔坐标，xyzabc+mode
	AuxPos m_AuxPos;	附加轴信息
	JointsPos m_Joint;	关节部分
	POSTYPE m_postyp;	位置类型，未使用，无需赋值

SDKOverlap 说明：

double	m_tcp_distance	绝对距离
double	m_Percent	%百分比
OVERLAPTYPE	m_OveTyp	圆滑给定方式
	eOVLABS = 1	绝对值给定
	eOVLREL	百分比给定

ORITYPE 说明：

ORITYPE	eOriNULL = 0,	NULL
	eVar = 1,	根据示教的姿态信息进行四元数姿态插补
	eConst,	恒定，始终保持起始位置的姿态
	eWrist,	腕关节

示例：

```
int result = 0;
//新建目标点
HMIPos endpos;
//设置点位类型为机器人笛卡尔坐标
endpos.m_PosType = eRobotCartPos;
//给机器人笛卡尔点位赋值
endpos.m_Pos.m_CartPos.m_X = 1000;
endpos.m_Pos.m_CartPos.m_Y = 0;
endpos.m_Pos.m_CartPos.m_Z = 1000;
endpos.m_Pos.m_CartPos.m_A = -180;
endpos.m_Pos.m_CartPos.m_B = 0;
endpos.m_Pos.m_CartPos.m_C = 0;
endpos.m_Pos.m_CartPos.m_Mode = 0;
//设置动态特性为NULL
int dyn = -1;
//新建圆滑特性
SDKOverlap ovl;
//设置圆滑特性类型为相对圆滑
ovl.m_OveTyp = eOVLREL;
//设置圆滑参数为100%
ovl.m_Percent = 100;
//新建姿态参数
```

```
ORITYPE ori;
//设置姿态参数为NULL
ori = eOriNULL;
//设置工具为“Default”
std::string tool1 = "Default";
//设置工具为“WORLD”
std::string ref1 = "WORLD";
//写Lin语句
result = robotClient.SDKSimulateLinProCmd(endpos, dyn, ovl, ori, tool1, ref1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "Lin写入失败，结果为:" << result << std::endl;
}
```

2.8.11 写Circ语句

函数	int SDKSimulateCircProCmd(endpos,helppos,dyn,ovl,ori,toolname,refname,linenum);	
输入	endpos	HMIPos，目标点
输入	helppos	HMIPos，过度点
输入	dyn	int，动态特性，只可以设置位置速度，其他参数利用 Dyn 语句设置;NULL 设置-1
输入	ovl	SDKOverlap，圆滑数据，NULL 时为-1
输入	ori	ORITYPE，姿态参数，枚举，NULL 为 0
输入	toolname	Std::string，工具名
输入	refname	Std::string，用户名
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

HMIPos 说明：

HMIPOSTYPE	m_PosType	点位类型
	eAxisPos = 1,	关节坐标类型（含附加轴）
	eCartPos,	笛卡尔坐标类型（含附加轴）
	eRobotAxisPos,	机器人关节坐标类型
	eRobotCartPos,	机器人笛卡尔坐标类型
int	m_Port	未使用，无需赋值，默认为 0
RcPos	m_Pos	点位数据
	CartPos m_CartPos;	笛卡尔坐标，xyzabc+mode
	AuxPos m_AuxPos;	附加轴信息
	JointsPos m_Joint;	关节部分
	POSTYPE m_postyp;	位置类型，未使用，无需赋值

SDKOverlap 说明：

double	m_tcp_distance	绝对距离

double	m_Percent	%百分比
OVERLAPTYPE	m_OveTyp	圆滑给定方式
	eOVLABS = 1	绝对值给定
	eOVLREL	百分比给定

ORITYPE 说明：

ORITYPE	eOriNULL = 0,	NULL
	eVar = 1,	根据示教的姿态信息进行四元数姿态插补
	eConst,	恒定，始终保持起始位置的姿态
	eWrist,	腕关节

示例：

```
int result = 0;
//新建目标点
HMIPos endpos;
//设置点位类型为机器人笛卡尔坐标
endpos.m_PosType = eRobotCartPos;
//给机器人笛卡尔点位赋值
endpos.m_Pos.m_CartPos.m_X = 1000;
endpos.m_Pos.m_CartPos.m_Y = 0;
endpos.m_Pos.m_CartPos.m_Z = 1000;
endpos.m_Pos.m_CartPos.m_A = -180;
endpos.m_Pos.m_CartPos.m_B = 0;
endpos.m_Pos.m_CartPos.m_C = 0;
endpos.m_Pos.m_CartPos.m_Mode = 0;
//新建过度点
HMIPos helppos;
//设置点位类型为机器人笛卡尔坐标
helppos.m_PosType = eRobotCartPos;
//给机器人笛卡尔点位赋值
helppos.m_Pos.m_CartPos.m_X = 800;
helppos.m_Pos.m_CartPos.m_Y = 200;
helppos.m_Pos.m_CartPos.m_Z = 1000;
helppos.m_Pos.m_CartPos.m_A = -180;
helppos.m_Pos.m_CartPos.m_B = 0;
helppos.m_Pos.m_CartPos.m_C = 0;
helppos.m_Pos.m_CartPos.m_Mode = 0;
//设置动态特性为NULL
int dyn = -1;
//新建圆滑特性
SDKOverlap ovl;
//设置圆滑特性类型为相对圆滑
ovl.m_OveTyp = eOVLREL;
//设置圆滑参数为100%
ovl.m_Percent = 100;
//新建姿态参数
```

```
ORITYPE ori;
//设置姿态参数为NULL
ori = eOriNULL;
//设置工具为“Default”
std::string tool1 = "Default";
//设置工具为“WORLD”
std::string ref1 = "WORLD";
//写Circ语句
result = robotClient.SDKSimulateCircProCmd(endpos, helppos, dyn, ovl, ori, tool1,
ref1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "Circ写入失败，结果为:" << result << std::endl;
}
```

2.8.12 写Dyn语句

函数	int SDKSimulateDynProCmd(m_DynPerc, linenum);	
输入	m_DynPerc	SDynamicPercent，动态特性
输入	linenum	int，语句行号，无输入时默认为 1
返回值(int)	0：正常；-1：执行失败	

SDynamicPercent 说明：

SubDynamic	m_SegmentDynamic	路径动态
SubDynamic	m_OriDynamic	TCP姿态动态
SubDynamic	m_JointPercent	关节动态

SubDynamic 说明：

double	m_Vel;	速度
double	m_Acc;	加速度
double	m_Dec;	减速度
double	m_Jerk;	加加速度
double	m_Tjolt;	时间,未用

示例：

```
int result = 0;
//新建dyn
SDynamicPercent dyn0;
//给dyn赋值
dyn0.m_SegmentDynamic.m_Vel = 2000;
dyn0.m_SegmentDynamic.m_Acc = 6000;
dyn0.m_SegmentDynamic.m_Dec = 6000;
dyn0.m_SegmentDynamic.m_Jerk = 60000;
dyn0.m_OriDynamic.m_Vel = 90;
dyn0.m_OriDynamic.m_Acc = 270;
```

```
dyn0.m_OriDynamic.m_Dec = 270;
dyn0.m_OriDynamic.m_Jerk = 2700;
dyn0.m_JointPercent.m_Vel = 100;
dyn0.m_JointPercent.m_Acc = 200;
dyn0.m_JointPercent.m_Dec = 200;
dyn0.m_JointPercent.m_Jerk = 200;
//写Dyn语句
result = robotClient.SDKSimulateDynProCmd(dyn0);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "Dyn写入失败，结果为:" << result << std::endl;
}
```

2.8.13 写WaitTime语句

函数	int SDKSimulateDynProCmd(waittime, linenum);	
输入	waittime	int，时间 ms
输入	linenum	int，语句行号，无输入时默认为 1
返回值(int)	0：正常；-1：执行失败	

示例：

```
int result = 0;
//新建WaitTime并赋值
int waittime = 200;
//写WaitTime语句
result = robotClient.SDKSimulateWaitTimeProCmd(waittime);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "eWaitTime写入失败，结果为:" << result << std::endl;
}
```

2.8.14 写ArcOn语句

函数	int SDKSimulateArcOnProCmd(arconvalue,arcsetvalue,decaytime,retrydata, linenum);	
输入	arconvalue	SDKArcOnVauleStruct，起弧变量
输入	arcsetvalue	SDKArcSetVauleStruct,焊接变量
输入	decaytime	int,渐变时间
输入	retrydata	HMIArcRetryData，再起弧变量
输入	backdistance	double回退距离
输入	linenum	int，语句行号，无输入时默认为1
返回值	(int) 0：正常；-1：执行失败	

SDKArcOnVauleStruct 说明：

int	m_PreflowTime;	预送气时间,ms
int	m_PreArcingTime;	预起弧时间,A
int	m_PreWireFeedTime;	预送丝时间,
double	m_ArcOnCurrent;	起弧电流
double	m_ArcOnVoltage;	起弧电压
int	m_ArcOnWaitTime;	等待焊机起弧成功的时间

SDKArcSetVauleStruct 说明：

double	m_ArcSetCurrent;	焊接电流
double	m_ArcSetVoltage;	焊接电压
double	m_ArcSetSpeed;	机器人焊接速度
double	m_ArcSetEndSpeed;	机器人焊接结束速度

SDKArcOnVauleStruct 说明：

int	m_Valid;	是否有再起弧，无时赋值 0
int	m_DetectTime;	起弧失败检测时间
int	m_RetryTimes;	起弧失败后，重试的次数
int	m_RetractTime;	焊丝回抽时间
int	m_RetractWaitTime;	回抽等待时间
double	m_CurrentInc;	再起弧电流增量
double	m_VoltageInc;	再起弧电压增量
ScratchArcData	m_ScratchArcData	擦线起弧设置参数

ScratchArcData 说明：

int	m_ScratchArcFlag;	擦线起弧标志，1 是启用，不启用时赋值 0
double	m_ScratchMaxDistance;	擦线距离
double	m_ScratchStepLength;	擦线步长，每次移动距离
double	m_ReturnSpeed;	擦线返回速度
int	m_ScratchSegType;	1 是 Lin，2 是 Circ，CircAngle
HMIPos	m_EndPos	目标点
HMIPos	m_HelpPos	过度点
Std::string	m_ToolName	工具名，必须赋值
Std::string	m_RefName	用户名，必须赋值

HMIPos 说明：

HMIPOSTYPE	m_PosType	点位类型
	eAxisPos = 1,	关节坐标类型（含附加轴）
	eCartPos,	笛卡尔坐标类型（含附加轴）
	eRobotAxisPos,	机器人关节坐标类型
	eRobotCartPos,	机器人笛卡尔坐标类型
int	m_Port	未使用，无需赋值，默认为 0
RcPos	m_Pos	点位数据
	CartPos m_CartPos;	笛卡尔坐标，xyzabc+mode
	AuxPos m_AuxPos;	附加轴信息
	JointsPos m_Joint;	关节部分

	POSTYPE m_postyp;	位置类型，未使用，无需赋值

示例：

```
int result = 0;
std::string tool1 = "Default";
std::string ref1 = "WORLD";
//新建起弧变量SDKArcOnVauleStruct
SDKArcOnVauleStruct arconvalue;
//给起弧变量赋值
arconvalue.m_PreflowTime = 500;
arconvalue.m_PreArcingTime = 200;
arconvalue.m_PreWireFeedTime = 0;
arconvalue.m_ArcOnCurrent = 100;
arconvalue.m_ArcOnVoltage = 100;
arconvalue.m_ArcOnWaitTime = 100;
//新建焊接变量SDKArcSetVauleStruct
SDKArcSetVauleStruct arcsetvalue;
//给焊接变量赋值
arcsetvalue.m_ArcSetCurrent = 250;
arcsetvalue.m_ArcSetVoltage = 1;
arcsetvalue.m_ArcSetSpeed = 10;
arcsetvalue.m_ArcSetEndSpeed = 0;
//新建渐变时间并赋值
int decaytime = 0;
//新建再起弧变量HMIArcRetryData
SDKArcRetryData retrydata;
//给再起弧变量赋值，未启用，给必须赋值的变量赋值
retrydata.m_Valid = 0;
retrydata.m_ScratchArcData.m_ScratchArcFlag = 0;
strcpy_s(retrydata.m_ScratchArcData.m_ToolName, tool1.c_str());
strcpy_s(retrydata.m_ScratchArcData.m_RefSysName, ref1.c_str());
//写ArcOn语句
result = robotClient.SDKSimulateArcOnProCmd(arconvalue, arcsetvalue, decaytime,
retrydata);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "ArcOn写入失败，结果为:" << result << std::endl;
}
```

2.8.15 写ArcOff语句

函数	int SDKSimulateArcOffProCmd(arcoffdata, linenum);	
输入	arcoffdata	SDKInstructArcOff，收弧变量
输入	linenum	int，语句行号，无输入时默认为 1

返回值(int)	0：正常；-1：执行失败

SDKInstructArcOff 说明：

int	m_BurnbackTime;	回烧时间
int	m_PostflowTime;	滞后送气时间
double	m_ArcOffCurrent;	收弧电流
double	m_ArcOffVoltage;	收弧电压
int	m_ArcOffWaitTime;	等待焊机息弧成功的时间
int	m_DecayTime;	渐变时间
int	m_StickCheckDelayTime;	默认粘丝检测延迟时间，单位秒

示例：

```
int result = 0;
//新建收弧变量SDKInstructArcOff
SDKInstructArcOff arcoffdata;
//给收弧变量赋值
arcoffdata.m_BurnbackTime = 0;
arcoffdata.m_PostflowTime = 500;
arcoffdata.m_ArcOffCurrent = 120;
arcoffdata.m_ArcOffVoltage = 0;
arcoffdata.m_ArcOffWaitTime = 0;
arcoffdata.m_DecayTime = 0;
arcoffdata.m_ArcOffWaitTime = 0;
//写ArcOff语句
result = robotClient.SDKSimulateArcOffProCmd(arcoffdata);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "ArcOff写入失败，结果为:" << result << std::endl;
}
```

2.8.16 写ArcSet语句

函数	int SDKSimulateArcSetProCmd(arcsetdata, linenum);	
输入	arcsetdata	SDKInstructArcSet，焊接变量
输入	linenum	int，语句行号，无输入时默认为 1
返回值(int)	0：正常；-1：执行失败	

SDKInstructArcSet 说明：

double	m_WeldCurrent;	焊接电流
double	m_WeldVoltage;	焊接电压
double	m_WeldSpeed;	机器人焊接速度
double	m_EndSpeed;	机器人焊接结束速度 2021.9.23
int	m_StartEnd;	渐变参考点，START:0;END:1;NULL:-1
double	m_Length;	渐变距离，NULL:0

示例：

```
int result = 0;
//新建焊接变量SDKInstructArcSet
SDKInstructArcSet arcsetdata;
//给焊接变量赋值
arcsetdata.m_WeldCurrent = 60;
arcsetdata.m_WeldVoltage = 0;
arcsetdata.m_WeldSpeed = 150;
arcsetdata.m_EndSpeed = 0;
arcsetdata.m_StartEnd = -1;
arcsetdata.m_Length = 0;
//写ArcSet语句
result = robotClient.SDKSimulateArcSetProCmd(arcsetdata);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "ArcSet写入失败，结果为:" << result << std::endl;
}
```

2.8.17 写WLin语句

函数	int SDKSimulateAWLinProCmd(endpos,dyn,ovl,ori,weave,seamtrack,arconset, toolname,refname, linenum);	
输入	endpos	HMIPos，目标点
输入	dyn	int，动态特性，只可以设置位置速度，其他参数利用 Dyn 语句设置;NULL 设置-1
输入	ovl	SDKOverlap，圆滑数据，NULL 时为-1
输入	ori	ORITYPE，姿态参数，枚举，NULL 为 0
输入	weave	WeaveParam,摆弧参数
输入	seamtrack	SeamTrackParam,电弧跟踪
输入	toolname	Std::string,工具名，必须赋值
输入	refname	Std::string,用户名，必须赋值
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

HMIPos 说明：

HMIPOSTYPE	m_PosType	点位类型
	eAxisPos = 1,	关节坐标类型（含附加轴）
	eCartPos,	笛卡尔坐标类型（含附加轴）
	eRobotAxisPos,	机器人关节坐标类型
	eRobotCartPos,	机器人笛卡尔坐标类型
int	m_Port	未使用，无需赋值，默认为 0
RcPos	m_Pos	点位数据
	CartPos m_CartPos;	笛卡尔坐标，xyzabc+mode
	AuxPos m_AuxPos;	附加轴信息
	JointsPos m_Joint;	关节部分

	POSTYPE m_postyp;	位置类型，未使用，无需赋值

SDKOverlap 说明：

double	m_tcp_distance	绝对距离
double	m_Percent	%百分比
OVERLAPTYPE	m_OveTyp	圆滑给定方式
	eOVLABS = 1	绝对值给定
	eOVLREL	百分比给定

ORITYPE 说明：

ORITYPE	eOriNULL = 0,	NULL
	eVar = 1,	根据示教的姿态信息进行四元数姿态插补
	eConst,	恒定，始终保持起始位置的姿态
	eWrist,	腕关节

WeaveParam 说明：

int	m_WeaveNULL	摆弧变量是否为NULL，1为NULL
WeaveType	m_WeaveType;	摆动类型：平面摆，关节摆等
	eGeometricWeave = 0,	eTCPWeave
	eWristWeave,	56关节参与的摆动
WeaveShape	m_WeaveShape;	摆动形状：无摆动，三角摆等
	eNoWeave = 0,	
	eSin,	正弦摆
	eSpiral,	螺旋摆
	eObliqueTriangle,	斜三角摆
	eSpaceTriangle,	空间三角形
	eLTriangle,	空间L型摆
	eBackForward,	前后摆
	eConstPoint	定点摆
double	m_WeaveLength;	摆动长度
double	m_WeaveAmplitude;	摆动幅值
double	m_Weavehigh;	平面摆动的高度
int	m_QPauseTime;	摆弧1/4处停留时间，需界面传入
int	m_ThreeQPauseTime;	摆弧3/4处停留时间，需界面传入
double	m_SwingDirection;	新增，摆弧倾斜角（对sin摆无效）
double	m_WeaveAngle;	新增，摆弧平面倾斜角
double	m_SpaceAngle;	空间摆弧夹角（对L型摆和空间三角摆有效）
int	m_PauseTime_1;	1/4处停留时间
int	m_PauseTime_2;	2/4处停留时间
int	m_PauseTime_3;	3/4处停留时间
int	m_PauseTime_4;	4/4处停留时间
int	m_PauseContinue;	摆弧连续，0-完全停止，1-移动模式
double	m_EndLength;	结束长度（sin摆有效）
double	m_EndWidth;	结束宽度（sin摆有效）

double	m_CenterHigh;	中心高度（sin 摆有效）

SeamTrackParam 说明：

int	m_TrackNULL	跟踪变量是否为 NULL，1 为 NULL
int	m_flag;	是否做电弧跟踪
double	Kpl;	横向比例增益　不是比例增益百分比
double	m_LeftCoefficient;	横向左边面积系数
double	m_RightCoefficient;	横向右边面积系数
double	Kpv;	纵向比例增益　不是比例增益百分比
double	Iref;	电流参考值
int	carry;	是否加到下一个周期中
int	m_LateralBeginCycle;	横向纠偏开始周期（>=3）
int	m_VerticalModeFlag;	为0则是常数模式,为1是反馈模式 //CONST,SAMPLE
double	m_Iref_Vertical;	纵向纠偏基准电流（常数模式）
int	m_VerticalBeginCycle;	纵向取样开始周期1~999
int	m_VerticalSustainCycle;	纵向取样持续周期1~999
double	m_VerticalLength;	纵向周期长度
int	m_TypeofTimeOrDist;	1 == time,0 == distance
unsigned int	m_TimeInterval;	100-500ms，默认200ms
unsigned int	m_DistInterval;	距离 1-30mm,默认：2mm
double	m_YBias;	
double	m_ZBias;	only for lasertrack
SeamTrackComp	m_SeamTrackComp	

SeamTrackComp 说明：

double	m_L_MinComp;	横向最小补偿（单周期）
double	m_L_MaxComp;	横向最大补偿（单周期）
double	m_L_TotalMaxComp;	横向最大补偿（总）
double	m_L_RightComp;	横向不对称调整系数
double	m_L_Comp6;	备用
double	m_L_Comp5;	备用
double	m_L_Comp4;	备用
double	m_L_Comp3;	备用
double	m_L_Comp2;	备用
double	m_L_Comp1;	备用
double	m_V_MinComp;	纵向最小补偿（单周期）
double	m_V_MaxComp;	纵向最大补偿（单周期）
double	m_V_TotalMaxComp;	纵向最大补偿（总）
double	m_V_TopComp;	纵向不对称调整系数
double	m_V_Comp6;	备用
double	m_V_Comp5;	备用
double	m_V_Comp4;	备用
double	m_V_Comp3;	备用

double	m_V_Comp2;	备用
double	m_V_Comp1;	备用

示例：

```
int result = 0;
//新建目标点
HMIPos endpos;
//设置点位类型为机器人笛卡尔坐标
endpos.m_PosType = eRobotCartPos;
//给机器人笛卡尔点位赋值
endpos.m_Pos.m_CartPos.m_X = 900;
endpos.m_Pos.m_CartPos.m_Y = 0;
endpos.m_Pos.m_CartPos.m_Z = 800;
endpos.m_Pos.m_CartPos.m_A = -180;
endpos.m_Pos.m_CartPos.m_B = 0;
endpos.m_Pos.m_CartPos.m_C = 0;
endpos.m_Pos.m_CartPos.m_Mode = 0;
//设置动态特性为NULL
int dyn = -1;
//新建圆滑特性
SDKOverlap ovl;
//设置圆滑特性类型为相对圆滑
ovl.m_OveTyp;
//设置圆滑参数为100%
ovl.m_Percent = 100;
//新建姿态参数
ORITYPE ori;
//设置姿态参数为NULL
ori = eOriNULL;
//设置工具为“Default”
std::string tool1 = "Default";
//设置工具为“WORLD”
std::string ref1 = "WORLD";
//新建摆弧参数
WeaveParam weave;
//设置摆弧类型为无摆弧
weave.m_WeaveNULL = 1;
weave.m_WeaveShape = eNoWeave;
//新建电弧跟踪参数
SeamTrackParam seamtrack;
//设置电弧跟踪标志位为0，无跟踪
seamtrack.m_TrackNULL = 1;
seamtrack.m_flag = 0;
//写WLin语句
result = robotClient.SDKSimulateAWLinProCmd(endpos, dyn, ovl, ori, weave,
```

```
seamtrack, arconset, tool1, ref1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "WLin写入失败，结果为:" << result << std::endl;
}
```

2.8.18 写WCirc语句

函数	int SDKSimulateAWLinProCmd(endpos,helppos,dyn,ovl,ori,weave,seamtrack, arconset,toolname,refname, linenum);	
输入	endpos	HMIPos，目标点
输入	helppos	HMIPos，过度点
输入	dyn	int，动态特性，只可以设置位置速度，其他参数利用 Dyn 语句设置;NULL 设置-1
输入	ovl	SDKOverlap，圆滑数据，NULL 时为-1
输入	ori	ORITYPE，姿态参数，枚举，NULL 为 0
输入	weave	WeaveParam,摆弧参数
输入	seamtrack	SeamTrackParam,电弧跟踪
输入	toolname	Std::string,工具名，必须赋值
输入	refname	Std::string,用户名，必须赋值
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

HMIPos 说明：

HMIPOSTYPE	m_PosType	点位类型
	eAxisPos = 1,	关节坐标类型（含附加轴）
	eCartPos,	笛卡尔坐标类型（含附加轴）
	eRobotAxisPos,	机器人关节坐标类型
	eRobotCartPos,	机器人笛卡尔坐标类型
int	m_Port	未使用，无需赋值，默认为 0
RcPos	m_Pos	点位数据
	CartPos m_CartPos;	笛卡尔坐标，xyzabc+mode
	AuxPos m_AuxPos;	附加轴信息
	JointsPos m_Joint;	关节部分
	POSTYPE m_postyp;	位置类型，未使用，无需赋值

SDKOverlap 说明：

double	m_tcp_distance	绝对距离
double	m_Percent	%百分比
OVERLAPTYPE	m_OveTyp	圆滑给定方式
	eOVLABS = 1	绝对值给定
	eOVLREL	百分比给定

ORITYPE 说明：

ORITYPE	eOriNULL = 0,	NULL

	eVar = 1,	根据示教的姿态信息进行四元数姿态插补
	eConst,	恒定，始终保持起始位置的姿态
	eWrist,	腕关节

WeaveParam 说明：

int	m_WeaveNULL	摆弧变量是否为NULL，1为NULL
WeaveType	m_WeaveType;	摆动类型：平面摆，关节摆等
	eGeometricWeave = 0,	eTCPWeave
	eWristWeave,	56关节参与的摆动
WeaveShape	m_WeaveShape;	摆动形状：无摆动，三角摆等
	eNoWeave = 0,	
	eSin,	正弦摆
	eSpiral,	螺旋摆
	eObliqueTriangle,	斜三角摆
	eSpaceTriangle,	空间三角形
	eLTriangle,	空间L型摆
	eBackForward,	前后摆
	eConstPoint	
double	m_WeaveLength;	摆动长度
double	m_WeaveAmplitude;	摆动幅值
double	m_Weavehigh;	平面摆动的高度
int	m_QPauseTime;	摆弧1/4处停留时间，需界面传入
int	m_ThreeQPauseTime;	摆弧3/4处停留时间，需界面传入
double	m_SwingDirection;	新增，摆弧倾斜角（对sin摆无效）
double	m_WeaveAngle;	新增，摆弧平面倾斜角
double	m_SpaceAngle;	空间摆弧夹角（对L型摆和空间三角摆有效）
int	m_PauseTime_1;	1/4处停留时间
int	m_PauseTime_2;	2/4处停留时间
int	m_PauseTime_3;	3/4处停留时间
int	m_PauseTime_4;	4/4处停留时间
int	m_PauseContinue;	摆弧连续，0-完全停止，1-移动模式
double	m_EndLength;	结束长度（sin摆有效）
double	m_EndWidth;	结束宽度（sin摆有效）
double	m_CenterHigh;	中心高度（sin 摆有效）

SeamTrackParam 说明：

int	m_TrackNULL	跟踪变量是否为 NULL，1 为 NULL
int	m_flag;	是否做电弧跟踪
double	Kpl;	横向比例增益　不是比例增益百分比
double	m_LeftCoefficient;	横向左边面积系数
double	m_RightCoefficient;	横向右边面积系数
double	Kpv;	纵向比例增益　不是比例增益百分比
double	Iref;	电流参考值

int	carry;	是否加到下一个周期中
int	m_LateralBeginCycle;	横向纠偏开始周期（>=3）
int	m_VerticalModeFlag;	为0则是常数模式,为1是反馈模式 //CONST,SAMPLE
double	m_Iref_Vertical;	纵向纠偏基准电流（常数模式）
int	m_VerticalBeginCycle;	纵向取样开始周期1~999
int	m_VerticalSustainCycle;	纵向取样持续周期1~999
double	m_VerticalLength;	纵向周期长度
int	m_TypeofTimeOrDist;	1 == time,0 == distance
unsigned int	m_TimeInterval;	100-500ms，默认200ms
unsigned int	m_DistInterval;	距离 1-30mm,默认：2mm
double	m_YBias;	
double	m_ZBias;	
SeamTrackComp	m_SeamTrackComp	

SeamTrackComp 说明：

double	m_L_MinComp;	横向最小补偿（单周期）
double	m_L_MaxComp;	横向最大补偿（单周期）
double	m_L_TotalMaxComp;	横向最大补偿（总）
double	m_L_RightComp;	横向不对称调整系数
double	m_L_Comp6;	备用
double	m_L_Comp5;	备用
double	m_L_Comp4;	备用
double	m_L_Comp3;	备用
double	m_L_Comp2;	备用
double	m_L_Comp1;	备用
double	m_V_MinComp;	纵向最小补偿（单周期）
double	m_V_MaxComp;	纵向最大补偿（单周期）
double	m_V_TotalMaxComp;	纵向最大补偿（总）
double	m_V_TopComp;	纵向不对称调整系数
double	m_V_Comp6;	备用
double	m_V_Comp5;	备用
double	m_V_Comp4;	备用
double	m_V_Comp3;	备用
double	m_V_Comp2;	备用
double	m_V_Comp1;	备用

示例：

```
int result = 0;
//新建目标点
HMIPos endpos;
//设置点位类型为机器人笛卡尔坐标
endpos.m_PosType = eRobotCartPos;
//给机器人笛卡尔点位赋值
endpos.m_Pos.m_CartPos.m_X = 800;
```

```
endpos.m_Pos.m_CartPos.m_Y = 0;
endpos.m_Pos.m_CartPos.m_Z = 800;
endpos.m_Pos.m_CartPos.m_A = -180;
endpos.m_Pos.m_CartPos.m_B = 0;
endpos.m_Pos.m_CartPos.m_C = 0;
endpos.m_Pos.m_CartPos.m_Mode = 0;
//新建过度点
HMIPos helppos;
//设置点位类型为机器人笛卡尔坐标
helppos.m_PosType = eRobotCartPos;
//给机器人笛卡尔点位赋值
helppos.m_Pos.m_CartPos.m_X = 850;
helppos.m_Pos.m_CartPos.m_Y = 200;
helppos.m_Pos.m_CartPos.m_Z = 800;
helppos.m_Pos.m_CartPos.m_A = -180;
helppos.m_Pos.m_CartPos.m_B = 0;
helppos.m_Pos.m_CartPos.m_C = 0;
helppos.m_Pos.m_CartPos.m_Mode = 0;
//设置动态特性为NULL
int dyn = -1;
//新建圆滑特性
SDKOverlap ovl;
//设置圆滑特性类型为相对圆滑
ovl.m_OveTyp = eOVLREL;
//设置圆滑参数为100%
ovl.m_Percent = 100;
//新建姿态参数
ORITYPE ori;
//设置姿态参数为NULL
ori = eOriNULL;
//设置工具为“Default”
std::string tool1 = "Default";
//设置工具为“WORLD”
std::string ref1 = "WORLD";
//新建摆弧参数
WeaveParam weave;
//设置摆弧类型为无摆弧
weave.m_WeaveNULL = 1;
weave.m_WeaveShape = eNoWeave;
//新建电弧跟踪参数
SeamTrackParam seamtrack;
//设置电弧跟踪标志位为0，无跟踪
seamtrack.m_TrackNULL = 1;
seamtrack.m_flag = 0;
```

```
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "WCirc写入失败，结果为:" << result << std::endl;
}
```

2.8.19 写ArcConfig语句

函数	int SDKSimulateArcConfigProCmd(arcconfig, linenum);	
输入	arcconfig	ArcConfigData
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

ArcConfigData 说明：

double	ArcConfigData[14];	焊接配置变量

示例：

```
int result = 0;
ArcConfigData arcconfig;
arcconfig[0] = 10;
result = robotClient.SDKSimulateArcConfigProCmd(arcconfig);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "ArcConfig写入失败，结果为:{0}" << result << std::endl;
}
```

2.8.20 写ArcJob语句

函数	int SDKSimulateArcJobProCmd(arcjog, linenum);	
输入	arcjog	int
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

示例：

```
int result = 0;
int arcjog = 1;
result = robotClient.SDKSimulateArcJobProCmd(arcjog);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "ArcJob写入失败，结果为:{0}" << result << std::endl;
}
```

2.8.21 写WireSet语句

函数	int SDKSimulateWireSetProCmd(wireset, linenum);

输入	wireset	SDKInstructWireSet
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructWireSet 说明：

int	m_WireSetType;	操作方式，0 为送丝，1 为回丝
int	m_WireTime;	时间

示例：

```
int result = 0;
SDKInstructWireSet wireset;
wireset.m_WireSetType = 1;
wireset.m_WireTime = 100;
result = robotClient.SDKSimulateWireSetProCmd(wireset);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "WireSet写入失败，结果为:{0}" << result << std::endl;
}
```

2.8.22 写DOSet语句

函数	int SDKSimulateDOSetProCmd(dostruct, linenum);	
输入	dostruct	SDKInstructDOSet
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructDOSet 说明：

int	m_DOPort;	端口号，从 1 开始
bool	m_DOValue;	值

示例：

```
int result = 0;
//设置DO
SDKInstructDOSet do1;
do1.m_DOPort = 1;
do1.m_DOValue = true;
//写DOSet语句
result = robotClient.SDKSimulateDOSetProCmd(do1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "DOSet写入失败，结果为:" << result << std::endl;
}
```

2.8.23 写DOSetSyncPath语句

函数	int SDKSimulateDOSetSyncPathProCmd(dospstruct, linenum);	
输入	dospstruct	SDKInstructDOSetSyncPath
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructDOSetSyncPath 说明：

int	m_DOPort;	端口号，从 1 开始
bool	m_DOValue;	值
int	m_StartEnd;	方向,0 为 start，1 为 end
double	m_Distance;	距离
int	m_MemberType;	轴/XYZ，0 为 null，1~6 为 A1~A6

示例：

```
int result = 0;
//同步距离输出DO信号
SDKInstructDOSetSyncPath do1;
do1.m_DOPort = 1;
do1.m_DOValue = true;
do1.m_StartEnd = 0;//方向,0为start，1为end
do1.m_Distance = 10;
do1.m_MemberType = 1;//A1/X方向
//写DOSetSyncPath语句
result = robotClient.SDKSimulateDOSetSyncPathProCmd(do1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "DOSetSyncPath写入失败，结果为: " << result << std::endl;
}
```

2.8.24 写DOSetSyncTime语句

函数	int SDKSimulateDOSetSyncTimeProCmd(doststruct, linenum);	
输入	doststruct	SDKInstructDOSetSyncTime
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructDOSetSyncTime 说明：

int	m_DOPort;	端口号，从 1 开始
bool	m_DOValue;	值
int	m_StartEnd;	方向,0 为 start，1 为 end
int	m_Time;	时间

示例：

```
int result = 0;
//同步时间输出DO信号
SDKInstructDOSetSyncTime do1;
do1.m_DOPort = 1;
```

```
do1.m_DOValue = true;
do1.m_StartEnd = 0;//方向,0为start，1为end
do1.m_Time = 10;
//写DOSetSyncTime语句
result = robotClient.SDKSimulateDOSetSyncTimeProCmd(do1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "DOSetSyncTime写入失败，结果为: " << result << std::endl;
}
```

2.8.25 写DOPulse语句

函数	int SDKSimulateDOPulseProCmd(dopulse, linenum);	
输入	dopulse	SDKInstructDOPulse
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructDOPulse 说明：

int	m_DOPort;	端口号，从 1 开始
bool	m_DOValue;	值
int	m_PulseTime	脉冲时间

示例：

```
int result = 0;
//设置DOPulse
SDKInstructDOPulse dop1;
dop1.m_DOPort = 1;
dop1.m_DOValue = true;
dop1.m_PulseTime = 500;
//写DOPulse语句
result = robotClient.SDKSimulateDOPulseProCmd(dop1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "DOPulse写入失败，结果为: " << result << std::endl;
}
```

2.8.26 写DOPulseSetSyncPath语句

函数	int SDKSimulateDOPulseSyncPathProCmd (dopulsesp, linenum);	
输入	dopulsesp	SDKInstructDOPulseSyncPath
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructDOPulseSyncPath 说明：

int	m_DOPort;	端口号，从 1 开始

bool	m_DOValue;	值
int	m_PulseTime	脉冲时间
int	m_StartEnd;	方向,0 为 start，1 为 end
double	m_Distance;	距离

示例：

```
int result = 0;
//同步距离输出DOPulse信号
SDKInstructDOPulseSyncPath dop1;
dop1.m_DOPort = 1;
dop1.m_DOValue = true;
dop1.m_PulseTime = 500;
dop1.m_StartEnd = 0;//方向,0为start，1为end
dop1.m_Distance = 10;
//写DOPulseSetSyncPath语句
result = robotClient.SDKSimulateDOPulseSyncPathProCmd(dop1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "DOPulseSetSyncPath写入失败，结果为: " << result << std::endl;
}
```

2.8.27 写DOPulseSetSyncTime语句

函数	int SDKSimulateDOPulseSyncTimeProCmd(dopulsest, linenum);	
输入	doststruct	SDKInstructDOPulseSyncTime
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructDOPulseSyncTime 说明：

int	m_DOPort;	端口号，从 1 开始
bool	m_DOValue;	值
int	m_PulseTime	脉冲时间
int	m_StartEnd;	方向,0 为 start，1 为 end
int	m_Time;	时间

示例：

```
int result = 0;
//同步时间输出DOPulse信号
SDKInstructDOPulseSyncTime dop1;
dop1.m_DOPort = 1;
dop1.m_DOValue = true;
dop1.m_PulseTime = 500;
dop1.m_StartEnd = 0;//方向,0为start，1为end
dop1.m_Time = 10;
//写DOPulseSyncTime语句
```

```
result = robotClient.SDKSimulateDOPulseSyncTimeProCmd(dop1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "DOPulseSyncTime写入失败，结果为: " << result << std::endl;
}
```

2.8.28 写GDOSet语句

函数	int SDKSimulateGDOSetProCmd(gdostruct, linenum);	
输入	gdostruct	SDKInstructGDOSet
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructGDOSet 说明：

int	m_GDOStartPort;	起始端口号（最低位）
int	m_GDOEndPort;	结束端口号（最高位）
int	m_GDOValue;	值

示例：

```
int result = 0;
//设置GDO
SDKInstructGDOSet gdo1;
gdo1.m_GDOStartPort = 1;
gdo1.m_GDOEndPort = 10;
gdo1.m_GDOValue = 500;
//写GDOSet语句
result = robotClient.SDKSimulateGDOSetProCmd(gdo1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "GDOSet写入失败，结果为: " << result << std::endl;
}
```

2.8.29 写GDOSetSyncPath语句

函数	int SDKSimulateGDOSetSyncPathProCmd(gdospstruct, linenum);	
输入	gdospstruct	SDKInstructGDOSetSyncPath
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructGDOSetSyncPath 说明：

int	m_GDOStartPort;	起始端口号（最低位）
int	m_GDOEndPort;	结束端口号（最高位）
int	m_GDOValue;	值
int	m_StartEnd;	方向,0 为 start，1 为 end
double	m_Distance;	距离

int	m_MemberType;	轴/XYZ，0 为 null，1~6 为 A1~A6

示例：

```
int result = 0;
//同步距离输出GDO信号
SDKInstructGDOSetSyncPath gdo1;
gdo1.m_GDOStartPort = 1;
gdo1.m_GDOEndPort = 10;
gdo1.m_GDOValue = 500;
gdo1.m_StartEnd = 0;//方向,0为start，1为end
gdo1.m_Distance = 10;
gdo1.m_MemberType = 1;//A1/X方向
//写GDOSetSyncPath语句
result = robotClient.SDKSimulateGDOSetSyncPathProCmd(gdo1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "GDOSetSyncPath写入失败，结果为: " << result << std::endl;
}
```

2.8.30 写GDOSetSyncTime语句

函数	int SDKSimulateGDOSetSyncTimeProCmd(gdoststruct, linenum);	
输入	gdoststruct	SDKInstructGDOSetSyncTime
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructGDOSetSyncTime 说明：

int	m_GDOStartPort;	起始端口号（最低位）
int	m_GDOEndPort;	结束端口号（最高位）
int	m_GDOValue;	值
int	m_StartEnd;	方向,0 为 start，1 为 end
int	m_Time;	时间

示例：

```
int result = 0;
//同步时间输出GDO信号
SDKInstructGDOSetSyncTime gdo1;
gdo1.m_GDOStartPort = 1;
gdo1.m_GDOEndPort = 10;
gdo1.m_GDOValue = 500;
gdo1.m_StartEnd = 0;//方向,0为start，1为end
gdo1.m_Time = 10;
//写GDOSetSyncTime语句
result = robotClient.SDKSimulateGDOSetSyncTimeProCmd(gdo1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
```

```
if (result != 0)
{
    std::cout << "GDOSetSyncTime写入失败，结果为: " << result << std::endl;
}
```

2.8.31 写AOSet语句

函数	int SDKSimulateAOSetProCmd(aostruct, linenum);	
输入	aostruct	SDKInstructAOSet
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructAOSet 说明：

int	m_AOPort;	端口号，从 1 开始
int	m_AOValue;	值

示例：

```
int result = 0;
//设置AO
SDKInstructAOSet ao1;
ao1.m_AOPort = 1;
ao1.m_AOValue = 1;
//写AOSet语句
result = robotClient.SDKSimulateAOSetProCmd(ao1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "AOSet写入失败，结果为:" << result << std::endl;
}
```

2.8.32 写AOSetSyncPath语句

函数	int SDKSimulateAOSetSyncPathProCmd(aospstruct, linenum);	
输入	aospstruct	SDKInstructAOSetSyncPath
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructAOSetSyncPath 说明：

int	m_AOPort;	端口号，从 1 开始
int	m_AOValue;	值
int	m_StartEnd;	方向,0 为 start，1 为 end
double	m_Distance;	距离
int	m_MemberType;	轴/XYZ，0 为 null，1~6 为 A1~A6

示例：

```
int result = 0;
//同步距离输出AO信号
```

```
SDKInstructAOSetSyncPath ao1;
ao1.m_AOPort = 1;
ao1.m_AOValue = 1;
ao1.m_StartEnd = 0;//方向,0为start，1为end
ao1.m_Distance = 10;
ao1.m_MemberType = 1;//A1/X方向
//写AOSetSyncPath语句
result = robotClient.SDKSimulateAOSetSyncPathProCmd(ao1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "AOSetSyncPath写入失败，结果为: " << result << std::endl;
}
```

2.8.33 写AOSetSyncTime语句

函数	int SDKSimulateAOSetSyncTimeProCmd(aoststruct, linenum);	
输入	aoststruct	SDKInstructAOSetSyncTime
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructAOSetSyncTime 说明：

int	m_AOPort;	端口号，从 1 开始
int	m_AOValue;	值
int	m_StartEnd;	方向,0 为 start，1 为 end
int	m_Time;	时间

示例：

```
int result = 0;
//同步时间输出AO信号
SDKInstructAOSetSyncTime ao1;
ao1.m_AOPort = 1;
ao1.m_AOValue = 1;
ao1.m_StartEnd = 0;//方向,0为start，1为end
ao1.m_Time = 10;
//写AOSetSyncTime语句
result = robotClient.SDKSimulateAOSetSyncTimeProCmd(ao1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "AOSetSyncTime写入失败，结果为: " << result << std::endl;
}
```

2.8.34 写DIWait语句

函数	int SDKSimulateDIWaitProCmd(diwait, linenum);

输入	diwait	SDKInstructDIWait
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructDIWait 说明：

int	m_DIPort;	端口号，从 1 开始
bool	m_DIValue;	值

示例：

```
int result = 0;
//设置DI
SDKInstructDIWait di1;
di1.m_DIPort = 1;
di1.m_DIValue = true;
//写DIWait语句
result = robotClient.SDKSimulateDIWaitProCmd(di1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "DIWait写入失败，结果为:" << result << std::endl;
}
```

2.8.35 写DIWaitSyncPath语句

函数	int SDKSimulateDIWaitSyncPathProCmd(diwaitsp, linenum);	
输入	diwaitsp	SDKInstructDIWaitSyncPath
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructDIWaitSyncPath 说明：

int	m_DIPort;	端口号，从 1 开始
bool	m_DIValue;	值
int	m_StartEnd;	方向,0 为 start，1 为 end
double	m_Distance;	距离

示例：

```
int result = 0;
//同步距离输出DI信号
SDKInstructDIWaitSyncPath di1;
di1.m_DIPort = 1;
di1.m_DIValue = true;
di1.m_StartEnd = 0;//方向,0为start，1为end
di1.m_Distance = 10;
//写DIWaitSyncPath语句
result = robotClient.SDKSimulateDIWaitSyncPathProCmd(di1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
```

```
{
    std::cout << "DIWaitSyncPath写入失败，结果为: " << result << std::endl;
}
```

2.8.36 写DIWaitSyncTime语句

函数	int SDKSimulateDIWaitSyncTimeProCmd(diwaitst, linenum);	
输入	diwaitst	SDKInstructDIWaitSyncTime
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructDIWaitSyncTime 说明：

int	m_DIPort;	端口号，从 1 开始
bool	m_DIValue;	值
int	m_StartEnd;	方向,0 为 start，1 为 end
int	m_Time;	时间

示例：

```
int result = 0;
//同步时间输出DI信号
SDKInstructDIWaitSyncTime di1;
di1.m_DIPort = 1;
di1.m_DIValue = true;
di1.m_StartEnd = 0;//方向,0为start，1为end
di1.m_Time = 10;
//写DIWaitSyncTime语句
result = robotClient.SDKSimulateDIWaitSyncTimeProCmd(di1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "DIWaitSyncTime写入失败，结果为: " << result << std::endl;
}
```

2.8.37 写GDIWait语句

函数	int SDKSimulateGDIWaitProCmd(gdiwait, linenum);	
输入	gdiwait	SDKInstructGDIWait
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructGDIWait 说明：

int	m_GDIStartPort;	起始端口号（最低位）
int	m_GDIEndPort;	结束端口号（最高位）
int	m_GDIValue;	值

示例：

```
int result = 0;
```

```
//设置GDI
SDKInstructGDIWait gdi1;
gdi1.m_GDIStartPort = 1;
gdi1.m_GDIEndPort = 10;
gdi1.m_GDIValue = 500;
//写GDIWait语句
result = robotClient.SDKSimulateGDIWaitProCmd(gdi1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "GDIWait写入失败，结果为: " << result << std::endl;
}
```

2.8.38 写GDIWaitSyncPath语句

函数	int SDKSimulateGDIWaitSyncPathProCmd(gdiwaitsp, linenum);	
输入	gdiwaitsp	SDKInstructGDIWaitSyncPath
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructGDIWaitSyncPath 说明：

int	m_GDIStartPort;	起始端口号（最低位）
int	m_GDIEndPort;	结束端口号（最高位）
int	m_GDIValue;	值
int	m_StartEnd;	方向,0 为 start，1 为 end
double	m_Distance;	距离

示例：

```
int result = 0;
//同步距离输出GDI信号
SDKInstructGDIWaitSyncPath gdi1;
gdi1.m_GDIStartPort = 1;
gdi1.m_GDIEndPort = 10;
gdi1.m_GDIValue = 500;
gdi1.m_StartEnd = 0;//方向,0为start，1为end
gdi1.m_Distance = 10;
//写GDIWaitSyncPath语句
result = robotClient.SDKSimulateGDIWaitSyncPathProCmd(gdi1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "GDIWaitSyncPath写入失败，结果为: " << result << std::endl;
}
```

2.8.39 写GDIWaitSyncTime语句

函数	int SDKSimulateGDIWaitSyncTimeProCmd(gdiwaitst, linenum);	
输入	gdiwaitst	SDKInstructGDIWaitSyncTime
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructGDIWaitSyncTime 说明：

int	m_GDIStartPort;	起始端口号（最低位）
int	m_GDIEndPort;	结束端口号（最高位）
int	m_GDIValue;	值
int	m_StartEnd;	方向,0 为 start，1 为 end
int	m_Time;	时间

示例：

```
int result = 0;
//同步时间输出GDI信号
SDKInstructGDIWaitSyncTime gdi1;
gdi1.m_GDIStartPort = 1;
gdi1.m_GDIEndPort = 10;
gdi1.m_GDIValue = 500;
gdi1.m_StartEnd = 0;//方向,0为start，1为end
gdi1.m_Time = 10;
//写GDIWaitSyncTime语句
result = robotClient.SDKSimulateGDIWaitSyncTimeProCmd(gdi1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "GDIWaitSyncTime写入失败，结果为: " << result << std::endl;
}
```

2.8.40 写AIWait语句

函数	int SDKSimulateAIWaitProCmd(aiwait, linenum);	
输入	aiwait	SDKInstructAIWait
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructAIWait 说明：

int	m_AIPort;	端口号，从 1 开始
int	m_AIValue;	值

示例：

```
int result = 0;
//设置AI
SDKInstructAIWait ai1;
ai1.m_AIPort = 1;
ai1.m_AIValue = 1;
//写AIWait语句
```

```
result = robotClient.SDKSimulateAIWaitProCmd(ai1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "AIWait写入失败，结果为:" << result << std::endl;
}
```

2.8.41 写AIWaitSyncPath语句

函数	int SDKSimulateAIWaitSyncPathProCmd(aiwaitsp, linenum);	
输入	aiwaitsp	SDKInstructAIWaitSyncPath
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructAIWaitSyncPath 说明：

int	m_AIPort;	端口号，从 1 开始
int	m_AIValue;	值
int	m_StartEnd;	方向,0 为 start，1 为 end
double	m_Distance;	距离

示例：

```
int result = 0;
//同步距离输出AI信号
SDKInstructAIWaitSyncPath ai1;
ai1.m_AIPort = 1;
ai1.m_AIValue = 1;
ai1.m_StartEnd = 0;//方向,0为start，1为end
ai1.m_Distance = 10;
//写AIWaitSyncPath语句
result = robotClient.SDKSimulateAIWaitSyncPathProCmd(ai1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "AIWaitSyncPath写入失败，结果为: " << result << std::endl;
}
```

2.8.42 写AIWaitSyncTime语句

函数	int SDKSimulateAIWaitSyncTimeProCmd(aiwaitst, linenum);	
输入	aiwaitst	SDKInstructAIWaitSyncTime
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructAIWaitSyncTime 说明：

int	m_AIPort;	端口号，从 1 开始
int	m_AIValue;	值
int	m_StartEnd;	方向,0 为 start，1 为 end

int	m_Time;	时间

示例：

```
int result = 0;
//同步时间输出AI信号
SDKInstructAIWaitSyncTime ai1;
ai1.m_AIPort = 1;
ai1.m_AIValue = 1;
ai1.m_StartEnd = 0;//方向,0为start，1为end
ai1.m_Time = 10;
//写AIWaitSyncTime语句
result = robotClient.SDKSimulateAIWaitSyncTimeProCmd(ai1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "AIWaitSyncTime写入失败，结果为: " << result << std::endl;
}
```

2.8.43 写CORR_Tool语句

函数	int SDKSimulateCORR_ToolProCmd(corrtool, linenum);	
输入	corrtool	SDKInstructCORR_Tool
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructCORR_Tool 说明：

CartSys	m_CartDist	偏差量

CartSys 说明：

double	m_X;	mm
double	m_Y;	mm
double	m_Z;	mm
double	m_A;	°
double	m_B;	°
double	m_C;	°

示例：

```
int result = 0;
std::string ref1 = "ref10";
//设置CORR_Tool
SDKInstructCORR_Tool cpd_tool1;
cpd_tool1.m_CartDist.m_X = 20;
cpd_tool1.m_CartDist.m_Y = 0;
cpd_tool1.m_CartDist.m_Z = 0;
cpd_tool1.m_CartDist.m_A = 0;
cpd_tool1.m_CartDist.m_B = 0;
cpd_tool1.m_CartDist.m_C = 0;
```

```
//写CORR_Tool语句
result = robotClient.SDKSimulateCORR_ToolProCmd(cpd_tool1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "CORR_Tool写入失败，结果为:{0}" << result << std::endl;
}
```

2.8.44 写CORR_Ref语句

函数	int SDKSimulateCORR_RefProCmd(corrref, linenum);	
输入	corrref	SDKInstructCORR_Ref
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructCORR_Ref 说明：

CartSys	m_CartDist;	偏差量
char	m_RefName[32];	坐标系

CartSys 说明：

double	m_X;	mm
double	m_Y;	mm
double	m_Z;	mm
double	m_A;	°
double	m_B;	°
double	m_C;	°

示例：

```
int result = 0;
//设置CORR_Ref
SDKInstructCORR_Ref cpd_ref1;
cpd_ref1.m_CartDist.m_X = 20;
cpd_ref1.m_CartDist.m_Y = 0;
cpd_ref1.m_CartDist.m_Z = 0;
cpd_ref1.m_CartDist.m_A = 0;
cpd_ref1.m_CartDist.m_B = 0;
cpd_ref1.m_CartDist.m_C = 0;
//用户坐标赋值
strcpy_s(cpd_ref1.m_RefName, ref1.c_str());
result = robotClient.SDKSimulateCORR_RefProCmd(cpd_ref1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "CORR_Ref写入失败，结果为:{0}" << result << std::endl;
}
```

2.8.45 写CORR_OFF语句

函数	int SDKSimulateCORR_OffProCmd(corrref, linenum);	
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

示例：

```
int result = 0;
//需与前面的打开配合使用
//CORR_Off
result = robotClient.SDKSimulateCORR_OffProCmd();
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "CORR_Off写入失败，结果为:{0}" << result << std::endl;
}
```

2.8.46 写Ovl语句

函数	int SDKSimulateOvlProCmd(ovl, linenum);	
输入	ovl	SDKInstructOvl
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructOvl 说明：

int	m_OvlRel;	圆滑百分比
OverlapABS	m_OvlABS;	TCP 位置圆滑距离
OVERLAPTYPE	m_OveTyp;	圆滑类型

OverlapABS 说明：

double	m_Zone_tcp;	TCP 位置圆滑距离
double	m_Zone_ori;	= m_Zone_tcp/10
double	m_Zone_leax;	= m_Zone_tcp
double	m_Zone_reax;	= m_Zone_tcp/10

OVERLAPTYPE 说明：

m_OveTyp	eOVLABS = 1	绝对值给定
	eOVLREL	百分比给定

示例：

```
int result = 0;
SDKInstructOvl ovl1;
ovl1.m_OveTyp = eOVLREL;
ovl1.m_OvlRel = 50;
result = robotClient.SDKSimulateOvlProCmd(ovl1);
// 检查语句写入是否成功，若返回 - 1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
```

```
    std::cout << "Ovl写入失败，结果为:{0}" << result << std::endl;
}
```

2.8.47 写AxisSpaceActivate语句

函数	int SDKSimulateAxisSpaceActivateProCmd(axisspaceact, linenum);	
输入	axisspaceact	SDKInstructAxisSpaceActivate
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructAxisSpaceActivate 说明：

int	m_SpaceNum;	工作空间
bool	m_Active;	是否激活

示例：

```
int result = 0;
SDKInstructAxisSpaceActivate axisspaceact;
axisspaceact.m_SpaceNum = 1;
axisspaceact.m_Active = true;
result = robotClient.SDKSimulateAxisSpaceActivateProCmd(axisspaceact);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "AxisSpaceActivate写入失败，结果为:{0}" << result << std::endl;
}
```

2.8.48 写CartSpaceActivate语句

函数	int SDKSimulateCartSpaceActivateProCmd(cartspaceact, linenum);	
输入	cartspaceact	SDKInstructCartSpaceActivate
输入	linenum	int，语句行号，无输入时默认为 1
返回值	(int) 0：正常；-1：执行失败	

SDKInstructCartSpaceActivate 说明：

int	m_SpaceNum;	工作空间
bool	m_Active;	是否激活

示例：

```
int result = 0;
SDKInstructCartSpaceActivate cartspaceact;
cartspaceact.m_SpaceNum = 1;
cartspaceact.m_Active = true;
result = robotClient.SDKSimulateCartSpaceActivateProCmd(cartspaceact);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
{
    std::cout << "AxisSpaceActivate写入失败，结果为:{0}" << result << std::endl;
}
```

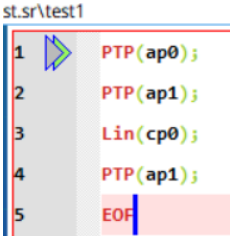
3 SDK使用案例

3.1 如何发送点位让机器人运行

思路：通过更改示教器程序中的点位数据实现外部点位运行。

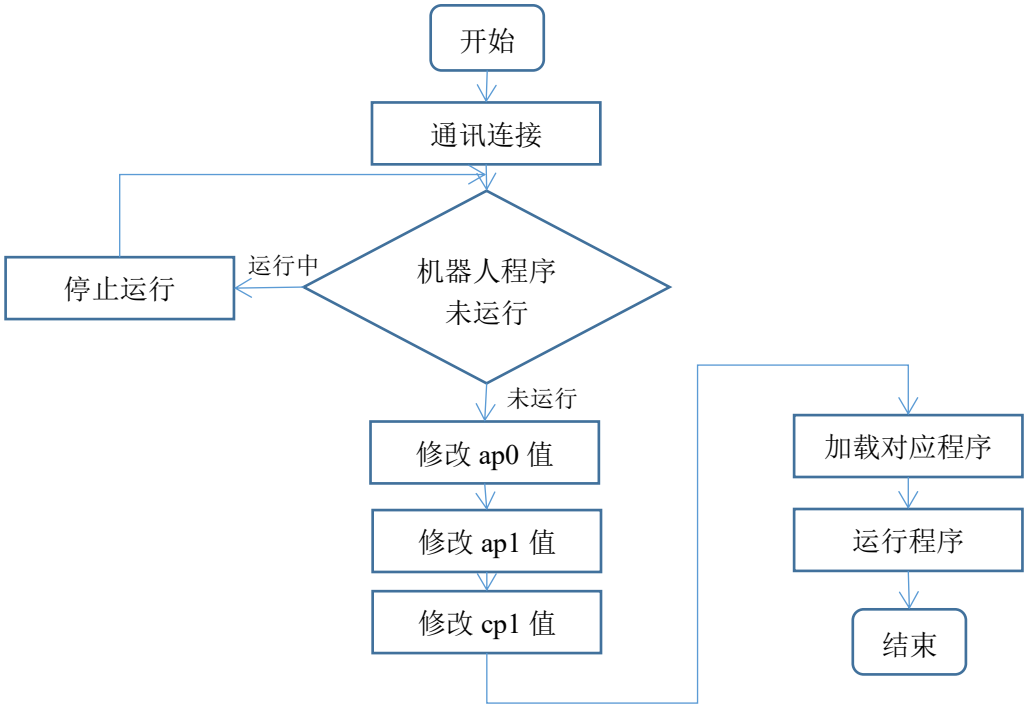
场景：适用于**固定工艺**的点位运行，使用前需先编写好示教器程序，即为固定的点位数量、运行方式。

示教器程序示例：



PTP(ap0); //关节运动到 ap0
PTP(ap1); //关节运动到 ap1
Lin(cp0); //直线运动到 cp0
PTP(ap1); //关节运动到 ap1

运行流程：



使用到的接口：

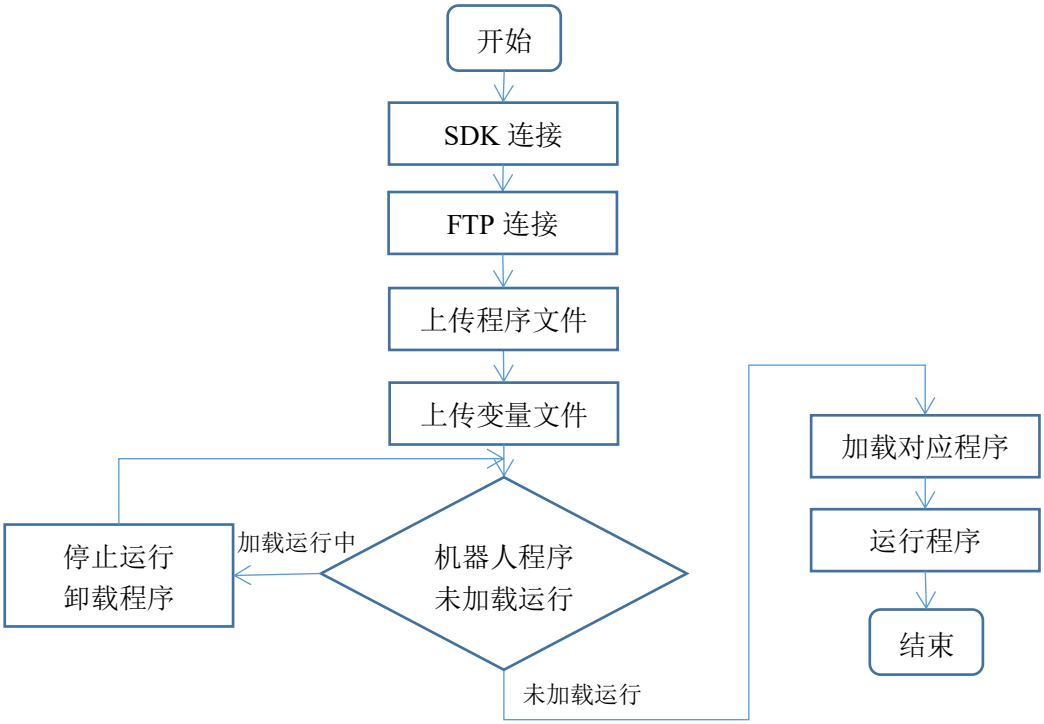
1. 初始化，连接机器人控制器；
2. 获取机器人 DO，获取机器人程序运行状态（需配置系统 DO）；
3. 修改 Axispos 值；
4. 修改 Cartpos 值；
5. 加载程序；
6. 设置 DI，运行程序（需配置系统 DI）；

3.2 如何发送多个点位让机器人运行

思路：通过 FTP 将写好的示教器程序写到机器人控制器中。

场景：适用于点位较多、数量不定、运行方式不定，但在运行前点位信息已确认的点位运行。

运行流程：



使用到的接口：

1. 初始化，连接机器人控制器；
2. 获取机器人 DO，获取机器人程序运行状态（需配置系统 DO），或者获取当前状态；
3. 获取当前工程名；
4. 获取当前程序名；
5. 卸载程序；
6. 加载程序；
7. 设置 DI，运行程序（需配置系统 DI）；

3.3 如何实时发送语句让机器人运行

思路：通过语句控制接口，在后台加载一个模拟示教器程序，实时发送语句，

机器人实时运行语句。

场景：适用于工况比较复杂、灵活性要求高的场景。

示例：

```
#include "RobotCom.hpp"
using namespace STEPROBOTSDK;
// 定义客户端（必须）
RobotComClient robotClient;
int main() {
  int result = 0;
  std::string sdkversion;  //定义版本号变量
  std::cout << "当前 SDK 版本号为:" << sdkversion << std::endl;
  //可以检查一下版本是否和机器人版本匹配
  //初始化，连接服务器
  result = robotClient.init("192.168.39.220", 30312); //仿真软件时为“127.0.0.1”
  //检查初始化是否成功
  if (result != 0)
  {
    std::cout <<"初始化失败，结果为: "<< result << std::endl;
  }
  else
  {
    std::cout << "初始化完成！" << std::endl;
    sdkversion = robotClient.SDKVersion();  //获取当前版本号
  }
  //可以先清个错
 result = robotClient.AllAlarmConfirmCmd();    //清错
 if (result != 0) //检查是否成功
 {
     std::cout << "清错失败，结果为:" << result << std::endl;
 }

  //卸载，加载程序
  if (robotClient.getProgramName() != "SDK")//若当前程序不是“SDK”
  {
      if (robotClient.getProgramName() != "")//若当前程序不是“”（无程序）
      { //kill
          result = robotClient.ProgramKillCmd(robotClient.getProjectName(),
robotClient.getProgramName());//卸载当前程序
          if (result != 0)
          {//若卸载失败，打印卸载结果
              std::cout << "卸载失败，卸载结果:" << result << std::endl;
          }
          while (robotClient.getProgramName() != "") //等待卸载成功
          {
```

```
                Sleep(10);
            }
        }
        result = robotClient.SDKSimulateProgramLoad();//加载模拟程序
        if (result != 0)
        {//若加载失败，打印加载结果
            std::cout << "加载失败，加载结果:" << result << std::endl;
        }
        while (robotClient.getProgramName() != "SDK")//等待加载成功
        {
            Sleep(10);
        }
    }
    else
    { //卸载
        result = robotClient.SDKSimulateProgramKill();
        if (result != 0)
        {
            std::cout << "卸载结果:" << result << std::endl;
            return;
        }
        while (robotClient.getProgramName() != "")
        {
            Sleep(10);
        }
        //加载
        result = robotClient.SDKSimulateProgramLoad();
        if (result != 0)
        {
            std::cout << "加载结果:" << result << std::endl;
            return;
        }
        while (robotClient.getProgramName() != "SDK")
        {
            Sleep(10);
        }
    }
    //启动程序
if (robotClient.getOperationMode() != eAutomaticExternal)  //机器人非外部状态
{
    if (robotClient.getMotorEnableState() == 1)  //机器人使能状态
    {
        result = robotClient.SetModeCmd(START); //启动加载的程序
        if (result != 0) //检查是否成功
```

```
        {
            std::cout << "程序启动执行失败，结果为:" << result << std::endl;
        }
        else
        {
            for (int i = 0; i < 20; i++)
            {
                if (robotClient.getProgramState() == eRun)
                {
                        break; //如果为运行状态，跳出for循环
                }
                Sleep(50); //50ms后再次获取状态
            }
        }
    }
  }
//写入PTP语句
//新建目标点
HMIPos endpos ;
//设置点位类型为机器人关节坐标
endpos.m_PosType = eRobotAxisPos;
//给机器人6个关节赋值
endpos.m_Pos.m_Joint[0] = 0;
endpos.m_Pos.m_Joint[1] = 0;
endpos.m_Pos.m_Joint[2] = 0;
endpos.m_Pos.m_Joint[3] = 0;
endpos.m_Pos.m_Joint[4] = 90;
endpos.m_Pos.m_Joint[5] = 0;
//设置动态特性为NULL
int dyn = -1;
//新建圆滑特性
SDKOverlap ovl;
//设置圆滑特性类型为相对圆滑
ovl.m_OveTyp = eOVLREL;
//设置圆滑参数为100%
ovl.m_Percent = 100;
//设置工具为“Default”
std::string tool1 = "Default";
//设置工具为“WORLD”
std::string ref1 = "WORLD";
//写PTP语句
result = robotClient.SDKSimulatePTPProCmd(endpos, dyn, ovl, tool1, ref1);
//检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
if (result != 0)
```

```
    {
        std::cout << "PTP写入失败，结果为:" << result << std::endl;
    }
    //写EOF语句
    result = robotClient.SDKSimulateEOFCmd();
    //检查语句写入是否成功，若返回-1，一般为参数输入错误，或超过最大可写入语句条数
    if (result != 0)
    {
        std::cout << "EOF写入失败，结果为:" << result << std::endl;
    }
    //发送语句
    //新建错误行号，并赋初始值
    int linenumber = 0;
    //发送语句
    result = robotClient.SDKSimulateProgramSend(linenumber);
    //检查语句是否发送成功
    if (result != 0)
    {
        std::cerr << "语句写入失败，结果为:" << result << std::endl;
    }

    //等待机器人运行完成，之后可以卸载程序
    result = robotClient.SDKSimulateProgramKill();
    if (result != 0)
    {
        std::cout << "卸载结果:" << result << std::endl;
        return;
    }
    while (robotClient.getProgramName() != "")
    {
        Sleep(10);
    }

    //省略其他操作

    //使用完成，关闭连接
    result = robotClient.close();
    std::cout << "关闭连接完成。" << std::endl;
    //需要使用时，重新初始化
}
```